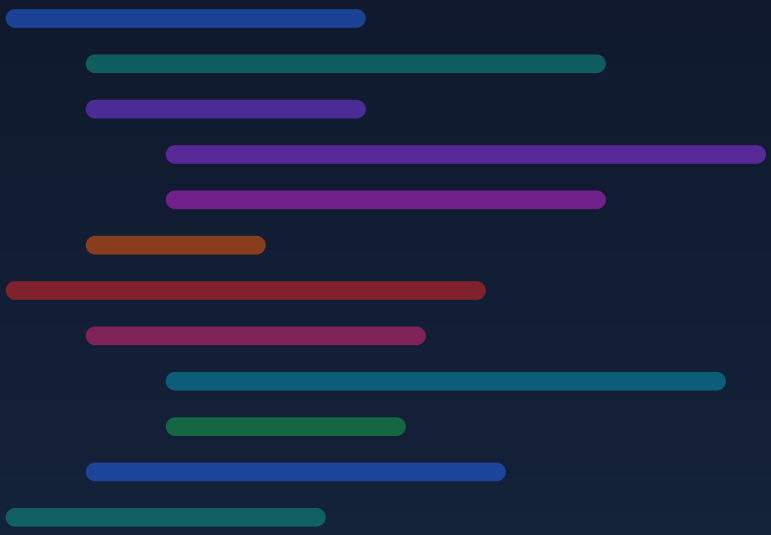


```
>>> print("Hello, world!")
Hello, world!
```

Learn to Code

A step-by-step guide from complete beginner to developer

- Python from zero
- Websites: HTML, CSS, JavaScript
- Mobile, desktop & web apps
- Git, databases, APIs & the terminal
- 20 programming languages explained
- Career paths, roadmap & 100+ exercises

Start slow and small. Build up one step at a time.

Edition 1 · October 2026

CONTENTS

Table of contents

Start here · How to use this guide 5

Part 0: Welcome to coding 7

Lesson 0.1 · What is coding? . . . 7
Lesson 0.2 · What is inside a computer? . . . 8
Lesson 0.3 · How your code becomes something the computer can run . . . 9
Lesson 0.4 · The programmer mindset . . . 10
Lesson 0.5 · Which language should I learn first? . . . 11

Part 1: Your tools 13

Lesson 1.1 · What tools does a programmer need? . . . 13
Lesson 1.2 · Beginner tools: start here . . . 14
Lesson 1.3 · Developer tools: what professionals use . . . 15
Lesson 1.4 · Setting up your computer, step by step . . . 16
Lesson 1.5 · Coding on a phone or tablet . . . 19

Part 2: Python, Stage 1: the basics 20

Lesson 1 · Your first program . . . 20
Lesson 2 · Variables: storing information . . . 22
Lesson 3 · Data types and maths . . . 24
Lesson 4 · Talking to the user: input . . . 26
Lesson 5 · Working with text (strings) . . . 27
Lesson 6 · Making decisions: if, elif, else . . . 29
Lesson 7 · Repeating things: while loops . . . 31
Lesson 8 · for loops and range . . . 33
Project 1 · Checkpoint project: number-guessing game . . . 36
Checkpoint · Quiz 1 and Stage 1 checklist . . . 38

Part 3: Python, Stage 2: organising data and code 39

Lesson 9 · Lists: many values in one variable . . . 39
Lesson 10 · Tuples and sets . . . 42
Lesson 11 · Dictionaries: looking things up by name . . . 43
Lesson 12 · Functions: reusable blocks of code . . . 45
Lesson 13 · Strings in depth . . . 48
Lesson 14 · Errors and exceptions . . . 50

Lesson 15 · Working with files . . . 52
Lesson 16 · Modules and libraries . . . 54
Project 2 · Checkpoint project: to-do list app . . . 57
Checkpoint · Quiz 2 and Stage 2 checklist . . . 59

Part 4: Python, Stage 3: thinking like a developer 61

Lesson 17 · Classes and objects . . . 61
Lesson 18 · Inheritance and the four ideas of OOP . . . 64
Lesson 19 · Pythonic shortcuts . . . 66
Lesson 20 · Debugging like a pro . . . 68
Lesson 21 · Testing your code . . . 69
Lesson 22 · Writing clean code . . . 71
Lesson 23 · Algorithms and Big-O . . . 72
Lesson 24 · Data structures you will hear about . . . 75
Project 3 · Checkpoint project: contact book . . . 77
Checkpoint · Quiz 3 and Stage 3 checklist . . . 79

Part 5: Developer essentials 80

Lesson 25 · The terminal (command line) . . . 80
Lesson 26 · Git and GitHub . . . 82
Lesson 27 · How the internet and websites work . . . 84
Lesson 28 · APIs and JSON . . . 85
Lesson 29 · Databases and SQL . . . 87
Lesson 30 · Security basics every developer must know . . . 89
Lesson 31 · How software is built in teams . . . 90
Lesson 32 · The cloud, servers and Docker . . . 91

Part 6: Building websites 92

Lesson 33 · What does a website need? . . . 92
Lesson 34 · HTML: the structure . . . 94
Lesson 35 · CSS: the style . . . 97
Lesson 36 · JavaScript: the behaviour . . . 100
Lesson 37 · Project 4 · Project: your portfolio website . . . 106
Lesson 38 · Publish your website for free . . . 112
Lesson 39 · Backends: making websites dynamic . . . 113
Lesson 40 · Levelling up as a web developer . . . 115

Part 7: Building apps 117

Lesson 41 · Websites vs apps: what's the difference? . . . 118
Lesson 42 · Mobile apps: native vs cross-platform . . . 119

Lesson 43 · One app, four frameworks . . . 120
Lesson 44 · Your first mobile app, step by step . . . 123
Lesson 45 · Desktop apps . . . 124
Lesson 46 · Web apps and PWAs . . . 126
Lesson 47 · How games are made . . . 127
Lesson 48 · Everything an app needs . . . 129
Lesson 49 · Publishing your app . . . 130
Lesson 50 · AI-powered apps . . . 131

Part 8: A tour of programming languages 132

Lesson 51 · The big picture . . . 133
Lesson 52 · The web languages . . . 134
Lesson 53 · General-purpose languages . . . 137
Lesson 54 · Mobile languages . . . 140
Lesson 55 · Systems languages: close to the machine . . . 142
Lesson 56 · Data and scripting languages . . . 144
Lesson 57 · Block-based coding and other languages you'll hear about . . . 146
Lesson 58 · Which language for which goal? . . . 147

Part 9: Your path forward 148

Lesson 59 · Career paths in software . . . 149
Lesson 60 · Your 12-month roadmap . . . 150
Lesson 61 · 30 project ideas, from easy to advanced . . . 151
Lesson 62 · The best free learning resources . . . 153
Lesson 63 · Using AI as a learning partner . . . 154
Lesson 64 · Your portfolio and your first opportunity . . . 155
You made it · Final words . . . 156

Appendices: Answers, references and checklists 157

Appendix A · Answers to exercises and quizzes . . . 157
Appendix B · Glossary . . . 176
Appendix C · Python cheat sheet . . . 179
Appendix D · HTML, CSS and JavaScript cheat sheet . . . 181
Appendix E · Troubleshooting: when things don't work . . . 183
Appendix F · Keyboard shortcuts . . . 184
Appendix G · Frequently asked questions . . . 185
Appendix H · Your progress checklist . . . 186

START HERE

How to use this guide

This guide is a ladder. Every step is small, and every step stands on the one before it. You start with **no knowledge at all**. By the end you will have written real programs, built and published a website, and understand how apps are made. You will also know how the professional world of software works: which languages and tools developers use, and which path fits you.

The guide has ten parts, from very easy to advanced:

Part	What it covers	Level
0 Welcome to coding	What coding is, how computers and code work, the right mindset	Absolute beginner
1 Your tools	Which apps beginners use, which developers use, and how to set them up	Absolute beginner
2 Python, Stage 1	Printing, variables, maths, input, text, decisions, loops	Beginner
3 Python, Stage 2	Lists, dictionaries, functions, errors, files, libraries	Beginner+
4 Python, Stage 3	Classes, debugging, testing, clean code, algorithms	Intermediate
5 Developer essentials	Terminal, Git and GitHub, the internet, APIs, databases, security, teamwork	Intermediate
6 Building websites	HTML, CSS, JavaScript, a full portfolio site, publishing it, backends	Intermediate
7 Building apps	Websites vs apps, mobile, desktop, web apps, games, app stores	Intermediate+
8 Tour of languages	20 languages: what each one is for, with the same program in each	All levels
9 Your path forward	Careers, a 12-month roadmap, project ideas, resources, using AI well	All levels

How each lesson works

1. **Explanation.** A short, plain-language explanation of one new idea.
2. **Example code.** A small program that shows the idea. Type it yourself.
3. **Output.** The dark box shows exactly what the program prints, so you can check your result.
4. **Try it yourself.** Short exercises. Do them *before* looking at the answers in Appendix A.
5. **Checkpoints.** After each Python stage there is a project and a quiz to prove you are ready to move on.

The boxes you will see

Box	What it means
Key idea	The single most important idea of the section. Remember this.
Tip	A trick or habit that makes your life easier.
Watch out	A common mistake that almost every beginner makes.
Good to know	Extra background. Interesting, but not required.
In the real world	How professionals actually do this at work.
Try it yourself	Exercises. This is where the real learning happens.

Six rules for learning fast

1. **Type every example yourself.** Copy-pasting teaches your eyes; typing teaches your brain and fingers.
2. **Change the examples.** After a program works, change a number or a word and predict what will happen. Then run it.
3. **Little and often.** 30 to 45 minutes every day beats 5 hours once a week.
4. **Errors are normal.** Professional developers see error messages all day long. An error is a clue, not a failure.
5. **The 20-minute rule.** Stuck for 20 minutes? Take a short break, re-read the lesson, then ask for help.
6. **Build things.** Reading alone will not make you a programmer. The projects in this guide matter most.

A suggested pace

Time	Parts	Goal
Week 1	Parts 0 and 1	Understand the basics and set up your tools
Weeks 2 to 4	Part 2	Write small programs on your own; finish the guessing game
Weeks 5 to 8	Part 3	Organise data and code; finish the to-do app
Weeks 9 to 12	Part 4	Think like a developer; finish the contact book
Month 4	Part 5	Use the terminal, Git, GitHub, APIs and databases
Months 5 to 6	Part 6	Build and publish your own website
Month 7	Part 7	Understand apps and build your first one
Any time	Parts 8 and 9	Explore languages and plan your path

TIP

Going slower than this is completely fine. Understanding matters more than speed. If a lesson feels hard, do it twice. The second time is always easier.

PART 0

Welcome to coding

What coding really is, how computers run your code, and how to think like a programmer.

In this part you will learn

- ✓ What a program is, using everyday examples
- ✓ What is inside a computer and how it runs code
- ✓ The difference between compiled and interpreted languages
- ✓ The mindset and habits that make people good at coding
- ✓ Which language to learn first, and why

LESSON 0.1

What is coding?

A computer is incredibly fast. It can do billions of calculations every second. But it has **no common sense at all**. It cannot guess what you mean. It does exactly what it is told: nothing more, nothing less.

Coding (also called **programming**) means writing exact instructions in a language the computer understands. A list of instructions like that is called a **program**. A program can also be called **code**, **software**, an **app** or a **script**.

A program is like a recipe

Here is a recipe a human can follow, next to a program a computer can follow:

Recipe for a human	Program for a computer (Python)
1. Ask the guest their name	`name = input("What is your name? ")`
2. Say hello to them by name	`print("Hello,", name)`
3. Ask how many cups of tea they want	`cups = int(input("How many cups? "))`
4. Tell them how much sugar you need	`print("Sugar needed:", cups * 2, "spoons")`

Both are steps that run **in order, from top to bottom, one at a time**. The difference is that the computer needs every step written in a very precise way. A human understands "make tea". A computer

needs every tiny step spelled out.

> **KEY IDEA**
> A program is a list of instructions that the computer follows exactly, from top to bottom, one line at a time.

Code is everywhere

- **Phone apps** like WhatsApp, TikTok and Instagram are code.
- **Websites** like YouTube, Google and Wikipedia are code.
- **Games** like Minecraft, Fortnite and Roblox are code.
- **Everyday machines**: cars, traffic lights, washing machines, ATMs and smart TVs all run code.
- **AI assistants**, like the one helping you right now, are built with code.

Everything you will ever build, from a tiny calculator to a huge app, is made of a few simple building blocks: **storing values**, **making decisions**, **repeating things**, and **grouping steps** into reusable pieces. This guide teaches those blocks one by one.

LESSON 0.2

What is inside a computer?

You do not need to be a hardware expert to code. But knowing the four main parts will help you understand error messages and why some programs are slow.

Part	What it does	Everyday comparison
CPU (processor)	Runs the instructions. The "brain".	A chef who follows the recipe
RAM (memory)	Short-term storage for programs that are running now. Very fast. Emptied when the power goes off.	The kitchen counter you are working on
Storage (SSD or hard drive)	Long-term storage for files and apps. Slower than RAM, but it keeps everything when turned off.	The fridge and cupboards
Operating system (Windows, macOS, Linux, Android, iOS)	The main program that manages everything else: files, apps, screen, keyboard and internet.	The restaurant manager

Everything is numbers: binary

Inside, a computer only understands two states: electricity **on** (1) and **off** (0). This is called **binary**. A single 0 or 1 is a **bit**. Eight bits make a **byte**. Letters, photos, music and videos are all stored as huge collections of 0s and 1s.

Normal number	Binary	How it works
0	`0000`	nothing switched on
1	`0001`	1
2	`0010`	2
5	`0101`	4 + 1
10	`1010`	8 + 2

GOOD TO KNOW

Sizes you will hear about: 1 kilobyte (KB) is about 1,000 bytes, a page of text. 1 megabyte (MB) is about a million bytes, a photo. 1 gigabyte (GB) is about a billion bytes, a film. 1 terabyte (TB) is about 1,000 GB, a whole computer's storage.

LESSON 0.3

How your code becomes something the computer can run

Computers only understand **machine code**: binary instructions for the CPU. Humans are terrible at writing that, so we invented **programming languages**. These are human-friendly languages that get *translated* into machine code. The code you write is called **source code**. There are two main ways to translate it:

	Compiled languages	Interpreted languages
How	The whole program is translated *before* it runs, into a separate file that can be run directly.	The program is translated *while* it runs, line by line, by a program called an interpreter.
Like	Translating a whole book before publishing it	A live interpreter at a meeting
Good	Very fast programs	Easy to start, quick to test changes
Examples	C, C++, Rust, Go, Swift	Python, JavaScript, Ruby, PHP

Some languages, like **Java**, **Kotlin** and **C#**, sit in between. They are compiled into an in-between form called *bytecode*, which a *virtual machine* then runs. This lets the same program run on Windows, Mac and Linux.

GOOD TO KNOW

You do not need to memorise this. Just remember that a tool always sits between your code and the computer and translates it. For Python, that tool is the **Python interpreter**.

Words you will hear all the time

Word	Meaning
Source code	The text of a program that a human writes
Run / execute	Tell the computer to start following the program
Syntax	The grammar rules of a programming language
Bug	A mistake in a program that makes it behave wrongly
Debugging	Finding and fixing bugs
Editor / IDE	The app you write code in (IDE means Integrated Development Environment)
Terminal	A window where you type commands instead of clicking
Library / package	Code someone else wrote that you can reuse
Framework	A big library that gives your project a ready-made structure (for example React or Django)
Frontend / backend	The part users see / the part on the server that users don't see

There is a full glossary of more than 90 words in Appendix B.

LESSON 0.4

The programmer mindset

Good programmers are not geniuses who never make mistakes. They are people who **stay calm, break problems into small pieces, and keep trying**. These habits matter more than talent.

1. Errors are clues, not failures

Your code will break. A lot. That is normal for everyone, including people with 20 years of experience. An error message tells you **what** went wrong and **where**. Learning to read error messages is one of the most valuable skills you can have.

2. Break big problems into small steps

"Make a game" is too big to start. Break it down until every step is easy:

1. Show a message: "Guess my number"
2. Let the player type a number
3. Check if the number is right
4. If it is wrong, say "too high" or "too low"
5. Repeat until the player is right
6. Count the attempts and show the score

Each step is easy on its own. Developers call this **decomposition**. You will build this exact game in Part 2.

3. Practise a little every day

Your brain learns through repetition. Short, daily practice builds skill much faster than rare, long sessions.

4. Avoid "tutorial hell"

Tutorial hell is when you watch video after video but cannot build anything on your own. The cure is simple: **after every lesson, build something small without help**, even if it is ugly.

5. Ask good questions

When you ask someone (a teacher, a forum or an AI assistant) for help, include:

- What you are trying to do
- Your code (the actual code, not a description of it)
- The **full** error message
- What you expected, and what actually happened
- What you have already tried

> **TIP**
> Often, just writing a clear question makes you spot the answer yourself. Developers even explain their code out loud to a rubber duck to find bugs. This really works, and it's called *rubber duck debugging*.

6. Do I need to be good at maths?

For most programming, **no**. Basic arithmetic and logical thinking are enough for websites, apps and automation. More maths helps in specific areas: games (geometry), AI and data science (statistics, algebra) and graphics. You can learn it when you need it.

LESSON 0.5

Which language should I learn first?

There are hundreds of programming languages. As a beginner, three names come up most often:

	Python	JavaScript	Java
Easy to learn?	Easiest. Reads almost like English.	Medium. Has some odd quirks.	Harder. Lots of extra code.
Hello world	`print("Hi")`	`console.log("Hi")`	5 lines of setup code
Best for	Learning, automation, data, AI, back-end servers	Websites, web apps, some mobile and desktop apps	Android, big company systems, school courses
Runs in a web browser?	No	Yes, it is the only language browsers run	No
Speed	Slower	Fast	Fast

Here is the same tiny program in all three. It counts from 1 to 3:

PYTHON

```python
for i in range(1, 4):
    print(i)
```

JAVASCRIPT

```javascript
for (let i = 1; i <= 3; i++) {
    console.log(i);
}
```

JAVA

```java
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            System.out.println(i);
        }
    }
}
```

All three print the same thing. Python just needs the least extra code, so you can focus on *thinking* instead of fighting the syntax.

KEY IDEA

Start with **Python**. The ideas you learn (variables, decisions, loops, functions, objects) work in *every* language. After Python, JavaScript or Java takes weeks to learn instead of months.

WATCH OUT

There are two exceptions. **(1)** If your school or exam uses Java or JavaScript, learn that one first, because passing comes first. **(2)** If you *only* want to build websites, start learning JavaScript after Part 2 of this guide.

GOOD TO KNOW

Java and JavaScript are **not related**, despite the similar names. JavaScript got its name in 1995 as a marketing trick, because Java was popular at the time. Part 8 explains 20 languages and what each is used for.

The plan for this guide

1. Learn to **think like a programmer** with Python (Parts 2 to 4)
2. Learn the **tools every developer uses**: terminal, Git, databases (Part 5)
3. Build **websites** with HTML, CSS and JavaScript (Part 6)
4. Understand and build **apps**: mobile, desktop and games (Part 7)
5. Explore **other languages** and choose your path (Parts 8 and 9)

PART 1

Your tools

Which apps beginners use, which apps professional developers use, and how to set up your computer step by step.

In this part you will learn

- ✓ What tools a programmer needs, and why
- ✓ The best tools for beginners (including ones that need no install)
- ✓ The tools professional developers use every day
- ✓ How to install Python and VS Code, step by step
- ✓ How to write and run your very first program file
- ✓ What you can do on a phone or tablet

LESSON 1.1

What tools does a programmer need?

To write and run code you need just two things:

1. **A place to write code**: a *code editor* or an *IDE*.
2. **Something that runs your code**: for Python, the *Python interpreter*.

As you grow, you will add more tools:

Tool	What it is for	When you need it
Code editor	A text editor made for code. It colours your code, spots typos and runs programs.	Day 1
IDE	A code editor with everything built in: running, debugging, testing and project management.	Day 1 or later
Interpreter / compiler	Turns your code into something the computer runs (see Lesson 0.3).	Day 1
Web browser	Runs websites and JavaScript. Has developer tools built in.	Websites (Part 6)
Terminal	Type commands to run programs, install packages and use Git.	Part 5
Git + GitHub	Saves the history of your code and lets you share it online.	Part 5
Package manager	Installs code other people wrote (`pip` for Python, `npm` for JavaScript).	Part 3

GOOD TO KNOW

Code editor vs IDE. A code editor (like VS Code) is light and flexible. You add features with extensions. An IDE (like PyCharm) is heavier but has everything built in. Both are fine, and many developers use both.

LESSON 1.2

Beginner tools: start here

Option A: Nothing to install (start in 30 seconds)

These run in your web browser. They are perfect for your first lessons, or for practising on a school computer.

Tool	What it is	Account?	Good for
Programiz Online Compiler (programiz.com/python-programming/online-compiler)	Write Python on the left, click Run, see the result on the right.	No	Quick practice, the simplest option
Replit (replit.com)	An online coding workspace for many languages. It saves your projects.	Yes (free plan with limits)	Small projects, sharing code
Google Colab (colab.research.google.com)	"Notebooks" that mix code and notes. Saves to Google Drive.	Google account	Python practice, data and AI later
Python's online shell (python.org/shell)	A tiny live Python prompt on the official website.	No	Testing a single line

Option B: Install on your computer (best long-term)

Tool	What it is	Why beginners like it
Python (python.org)	The Python language itself. Free.	Required to run Python on your computer.
Thonny (thonny.org)	A simple Python editor made for learners. Free. Comes with Python built in.	Big Run button. Shows your variables live. Can run your code one step at a time so you see exactly what happens.
IDLE	A basic editor that comes with Python.	Already installed with Python. Very simple.
VS Code (code.visualstudio.com)	The most popular code editor in the world. Free.	Works for every language. You will use it for years.

KEY IDEA

Our recommendation: start with **Programiz** or **Thonny** for the first 2 to 4 weeks. Then move to **VS Code**, which is what most professionals use.

Option C: Block coding (optional warm-up)

Scratch (scratch.mit.edu) lets you snap blocks together instead of typing. It is great for understanding loops and decisions in a fun, visual way. If typed code feels scary, spend a few hours in Scratch first. Then come back.

LESSON 1.3

Developer tools: what professionals use

You do not need these on day one, but you should know they exist. You will grow into them.

Job	Beginner uses	Professional developer uses
Writing code	Programiz, Thonny, IDLE	**VS Code** (most popular), **PyCharm** (Python), **IntelliJ IDEA** (Java/Kotlin), **Visual Studio** (C#, C++ on Windows), **Android Studio** (Android apps), **Xcode** (iPhone and Mac apps, Mac only), **Cursor** and other AI-assisted editors
Running code	The Run button	The **terminal** (command line), plus run and debug settings
Finding bugs	`print()`	**Debuggers** with breakpoints, logging, error tracking services
Saving versions	Saving files	**Git** for history, **GitHub** or GitLab for sharing and teamwork
Installing libraries	Nothing yet	**pip** / **uv** (Python), **npm** / **pnpm** (JavaScript), and others
Testing websites	Opening the page	**Browser DevTools** (F12), responsive device mode, Lighthouse
Talking to servers	Nothing yet	**Postman**, **curl** for testing APIs
Databases	Nothing yet	**DB Browser for SQLite**, **pgAdmin**, **DBeaver**
Running apps anywhere	Nothing yet	**Docker** (packs an app with everything it needs)
Designing screens	Paper and pencil	**Figma** (designing apps and websites before coding)
Working in a team	Nothing yet	**Slack**/**Teams** (chat), **Jira**/**Linear**/**GitHub Issues** (task tracking)
Help and learning	This guide, teachers	Official docs, Stack Overflow, AI assistants (Claude, GitHub Copilot)

Minimum computer you need

- **Any** Windows, Mac or Linux computer from the last 8 to 10 years can learn to code.
- **RAM:** 8 GB is enough to learn. 16 GB is comfortable for bigger projects and mobile app development.
- **Storage:** an SSD makes everything feel much faster. Keep at least 20 GB free.
- **Chromebook:** you can use online tools (Replit, Colab), or turn on the built-in Linux option to install real tools.
- **Building iPhone apps** needs a Mac, because Apple's Xcode only runs on macOS (see Part 7).

LESSON 1.4

Setting up your computer, step by step

Follow these steps in order. It takes about 20 minutes. Do it once, and you are set for months.

Step 1: Install Python

On Windows

1. Open your browser and go to **python.org/downloads**.
2. Click the big yellow **Download Python 3.x** button. The newest version is fine.
3. Open the downloaded file.
4. **Important:** if you see a checkbox called **"Add python.exe to PATH"**, tick it. This lets you run Python from anywhere.
5. Click **Install Now** and wait until it finishes.

On Mac

1. Go to **python.org/downloads** and download the macOS installer.
2. Open it and click **Continue** until it is installed.
3. On a Mac, the command is usually `python3` (not `python`). Remember this!

On Linux

Python 3 is almost always installed already. Check with `python3 --version`.

Step 2: Check that Python works

Open the **terminal**. This is a window where you type commands:

- **Windows:** press the Windows key, type `cmd` or `Terminal`, press Enter.
- **Mac:** press Cmd + Space, type `Terminal`, press Enter.

Type this and press Enter (use `python3` on Mac and Linux):

TERMINAL

```
python --version
```

YOU SHOULD SEE SOMETHING LIKE

```
Python 3.14.0
```

WATCH OUT

If you see *"'python' is not recognized"* on Windows, try `py --version`. If that also fails, run the installer again and make sure **"Add python.exe to PATH"** is ticked.

Step 3: Install VS Code

1. Go to **code.visualstudio.com** and download VS Code for your system. Install it.
2. Open VS Code. Click the **Extensions** icon on the left (four little squares), or press Ctrl + Shift + X (Cmd + Shift + X on Mac).
3. Search for **Python** and install the one made by **Microsoft**.
4. Optional: turn on **File > Auto Save** so you never lose work.

TIP

Prefer something simpler for now? Install **Thonny** from thonny.org instead. It includes Python, so you can even skip Step 1.

Step 4: Make a folder for all your code

Organisation matters. Create one folder for everything you code while learning:

FOLDER LAYOUT

```
Documents/
└── coding/
    ├── part2-basics/
    │   ├── lesson01_hello.py
    │   └── lesson02_variables.py
    ├── projects/
    │   └── guessing_game.py
    └── website/
```

Step 5: Write and run your first program file

1. In VS Code, click **File > Open Folder** and open your `coding` folder.
2. Click **File > New File**, and save it as `hello.py`. The `.py` ending tells the computer it is Python.
3. Type the code below. Then save it.
4. Click the **Run** button (a triangle in the top right), or type `python hello.py` in the terminal.

PYTHON

```python
print("Hello, world!")
print("I just ran my first Python file!")
```

OUTPUT

```
Hello, world!
I just ran my first Python file!
```

KEY IDEA

Every Python program is just a text file that ends in `.py`. You write it in an editor, and the Python interpreter runs it from top to bottom.

LESSON 1.5

Coding on a phone or tablet

You can learn the basics on a phone or tablet. It is slower to type code there, but it is great for practising on the go:

Device	Apps	Notes
Any phone or tablet	Programiz, Replit or Google Colab in the web browser	Nothing to install
Android	**Pydroid 3** (Python), Termux (advanced)	Pydroid 3 runs real Python offline
iPhone / iPad	**Pyto**, **a-Shell** (free), **Pythonista** (paid)	Pythonista is polished. a-Shell is a free terminal
Learning apps	Sololearn, Mimo, Grasshopper (JavaScript)	Good for daily practice; not for building real projects

WATCH OUT

A phone is fine for Parts 0 to 3. For websites, apps, Git and real projects, you will want a laptop or desktop computer. A keyboard and a bigger screen make a huge difference.

Checklist before Part 2

- ✓ I can open an editor (online or installed)
- ✓ I can run `print("Hello, world!")` and see the result
- ✓ I have a folder where I will save all my practice code

PART 2

Python, Stage 1: the basics

Your first real programs. Printing, variables, maths, input, text, decisions and loops, then your first game.

In this part you will learn

- ✓ How to show things on the screen with `print()`
- ✓ How to store information in variables
- ✓ Numbers, text and true/false values, plus doing maths
- ✓ How to ask the user for input
- ✓ How to make decisions with `if`, `elif` and `else`
- ✓ How to repeat things with `while` and `for` loops
- ✓ How to build a complete number-guessing game

LESSON 1

Your first program

Open your editor and type this program. Do not copy and paste it. Typing it is part of learning.

PYTHON

```
print("Hello, world!")
print("I am learning Python.")
print(5 + 3)
```

OUTPUT

```
Hello, world!
I am learning Python.
8
```

Let's look at what happened:

- `print()` is a **function**: a command that does a job. Its job is to show things on the screen.
- Whatever you put **inside the brackets** is what gets printed.
- Text must be inside **quotes**. Text in quotes is called a **string**.
- Numbers do not need quotes. `5 + 3` is calculated first, then the answer `8` is printed.
- The program runs **from top to bottom**, one line at a time.

Comments: notes for humans

Anything after a `#` symbol is a **comment**. Python ignores it completely. Comments explain your code to people, including future you.

PYTHON
```
# This whole line is a comment. Python skips it.
print("Comments help humans understand code")  # a comment at the end of a line
```

OUTPUT
```
Comments help humans understand code
```

Your first error (on purpose!)

Let's break something, so the first error you see doesn't scare you. Remove the closing bracket:

PYTHON
```
print("Hello"
```

ERROR
```
  File "hello.py", line 1
    print("Hello"
          ^
SyntaxError: '(' was never closed
```

Python tells you **the file**, **the line number**, points with `^` to where it got confused, and gives the **type of error**. A `SyntaxError` means you broke the grammar rules, like a missing bracket or quote.

Now try a capital letter:

PYTHON
```
Print("Hello")
```

ERROR
```
NameError: name 'Print' is not defined. Did you mean: 'print'?
```

WATCH OUT

Python is **case-sensitive**. `print`, `Print` and `PRINT` are three different words to Python. Only `print` exists.

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Print your name, your age and your city on three separate lines.
2. Print the result of `12 * 7`. (In Python, `*` means multiply.)
3. Delete one quote mark from a `print` line and read the error message. Then fix it.
4. Print this box using three `print` lines: `*****` then `*  *` then `*****`

LESSON 2

Variables: storing information

A **variable** is a name that stores a value, like a labelled box. You create one with the `=` sign. Then you can use the name instead of the value.

PYTHON

```
name = "Amira"
age = 16
print(name)
print(age)

age = 17          # put a new value in the box
print(age)
```

OUTPUT

```
Amira
16
17
```

KEY IDEA

In Python, `=` does **not** mean "equals" like in maths. It means **"store the value on the right in the name on the left"**. Read `age = 17` as "age becomes 17".

Rules for naming variables

Name	Allowed?	Why
`user_name`	✓ Yes, and good	Clear, lowercase, words joined with `_`
`score2`	✓ Yes	Numbers are fine, just not at the start
`2score`	✗ No	Cannot start with a number
`user name`	✗ No	No spaces allowed
`for`	✗ No	`for` is a Python keyword (a reserved word)
`x`	Allowed, but unclear	Fine for maths. Elsewhere, use names that explain what they hold
`Age` vs `age`	Both allowed	But they are **two different** variables!

TIP

Python programmers write variable names in **snake_case**: all lowercase, with words joined by underscores, like `high_score` or `first_name`.

Using variables together

PYTHON

```
first = "Ada"
last = "Lovelace"
full = first + " " + last
print(full)
```

OUTPUT

```
Ada Lovelace
```

Using `+` on strings joins them together. This is called **concatenation**. Notice the `" "`: a string with just a space, so the names don't stick together.

Updating a variable

PYTHON

```
score = 0
score = score + 10   # take the old value, add 10, store it back
score += 5           # a shortcut for: score = score + 5
print(score)
```

OUTPUT

```
15
```

The shortcuts `+=`, `-=`, `*=` and `/=` are used all the time. For example, `lives -= 1` means "lose one life".

GOOD TO KNOW

Ada Lovelace wrote what many people call the first computer program, back in the 1840s, for a machine that was never fully built. The Ada programming language is named after her.

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Make variables `city` and `country` and print a sentence like `I live in Cairo, Egypt`.
2. Make variables `price = 4` and `quantity = 3`. Print the total cost.
3. Start with `coins = 50`. Spend 20 coins, then find 35 more, using `-=` and `+=`. Print `coins` (should be 65).
4. Swap the values of `a = 1` and `b = 2` so `a` is 2 and `b` is 1. Hint: use a third variable.

LESSON 3

Data types and maths

Every value in Python has a **type**. The four basic types are:

Type	Name	Examples	Used for
`int`	integer	`42`, `-7`, `0`	Whole numbers: ages, counts, scores
`float`	floating-point number	`3.14`, `-0.5`, `2.0`	Decimal numbers: prices, measurements
`str`	string	`"hello"`, `'42'`, `""`	Text of any kind
`bool`	boolean	`True`, `False`	Yes/no answers (note the capital letter)

You can ask Python for the type of any value with `type()`:

PYTHON
```
print(type(42))
print(type(3.14))
print(type("hello"))
print(type(True))
```

OUTPUT
```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Maths operators

PYTHON
```
print(10 + 3)    # add
print(10 - 3)    # subtract
print(10 * 3)    # multiply
print(10 / 3)    # divide (always gives a float)
print(10 // 3)   # floor division: divide and drop the decimals
print(10 % 3)    # modulo: the remainder after dividing
print(10 ** 3)   # power: 10 to the power of 3
```

OUTPUT
```
13
7
30
3.3333333333333335
3
1
1000
```

GOOD TO KNOW

Why `3.3333333333333335` and not `...333`? Computers store decimals in binary, so tiny rounding differences happen. Use `round(10 / 3, 2)` to get `3.33`.

The amazing `%` (modulo)

Modulo gives the **remainder**. It sounds boring, but it is everywhere in programming:

- `n % 2 == 0` checks if a number is **even** (no remainder when divided by 2).
- `total_minutes % 60` gives the minutes left over after taking out full hours.
- Games use it to wrap around, like going from the last level back to the first.

Order of operations

Python follows the same order as maths: brackets first, then powers, then `* / // %`, then `+ -`.

PYTHON
```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

OUTPUT
```
14
20
```

Numbers and strings are different!

PYTHON
```
print("5" + "5")   # two strings: joined together
print(5 + 5)       # two numbers: added
```

OUTPUT
```
55
10
```

PYTHON
```
print("5" + 5)
```

ERROR
```
TypeError: can only concatenate str (not "int") to str
```

Python refuses to add text to a number because it cannot know what you want. You will learn to convert between types in the next lesson.

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Calculate how many seconds there are in one day (60 seconds × 60 minutes × 24 hours).
2. Convert 100 °C to Fahrenheit with the formula `c * 9 / 5 + 32`.
3. Use `//` and `%` to turn 135 minutes into hours and minutes (should be 2 hours and 15 minutes).
4. Predict the output of `print(2 ** 10)` and `print(17 % 5)`, then check.

LESSON 4

Talking to the user: input

Programs get interesting when they react to the user. `input()` shows a message, waits for the user to type something and press Enter, then gives you what they typed.

PYTHON

```
name = input("What is your name? ")
print("Nice to meet you, " + name + "!")
```

OUTPUT (the user typed Sara)

```
What is your name? Sara
Nice to meet you, Sara!
```

KEY IDEA

`input()` **always** gives you a **string**, even if the user types a number. To do maths with it, you must convert it first.

Converting between types

Function	Converts to	Example	Result
`int()`	whole number	`int("16")`	`16`
`float()`	decimal number	`float("4.5")`	`4.5`
`str()`	string	`str(99)`	`"99"`
`int()`	whole number (cuts off the decimals)	`int(3.9)`	`3`

PYTHON

```
age = input("How old are you? ")
age = int(age)                        # "16" becomes 16
print("Next year you will be", age + 1)
```

OUTPUT

```
How old are you? 16
Next year you will be 17
```

TIP

`print()` can take several values separated by commas. It puts a space between them automatically and can mix text and numbers, so `print("Age:", 16)` works without converting.

A shorter way, which you will see everywhere:

PYTHON

```
age = int(input("How old are you? "))
```

Mini program: a tip calculator

PYTHON

```
bill = float(input("Bill amount: "))
tip_percent = float(input("Tip percent: "))

tip = bill * tip_percent / 100
total = bill + tip

print("Tip:", round(tip, 2))
print("Total:", round(total, 2))
```

OUTPUT

```
Bill amount: 40
Tip percent: 15
Tip: 6.0
Total: 46.0
```

WATCH OUT

If the user types `hello` when you call `int()`, the program crashes with a `ValueError`. That's normal for now. In Lesson 14 you will learn how to handle bad input politely.

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Ask the user for two numbers. Print their sum, difference, product and division.
2. Ask for the year someone was born and print how old they turn this year (use 2026).
3. Ask for a number of minutes and print it as hours and minutes (use `//` and `%`).
4. Ask for the user's name and favourite food, then print a funny sentence using both.

LESSON 5

Working with text (strings)

You will work with text constantly: names, messages, passwords, web pages. Python has great tools for it.

f-strings: the best way to mix text and values

Put an `f` before the opening quote. Then anything inside `{curly brackets}` is replaced by its value:

PYTHON

```
name = "Leo"
age = 15
print(f"{name} is {age} years old.")
print(f"Next year {name} will be {age + 1}.")
```

OUTPUT

```
Leo is 15 years old.
Next year Leo will be 16.
```

TIP

f-strings are cleaner than joining with `+`, and you don't need to convert numbers with `str()`. Use them by default.

Useful string tools

PYTHON

```python
word = "Python"
print(len(word))      # length: how many characters
print(word.upper())   # UPPERCASE
print(word.lower())   # lowercase
print(word[0])        # the first character
print(word[-1])       # the last character
```

OUTPUT

```
6
PYTHON
python
P
n
```

Counting starts at zero

Each character has a position number called an **index**. In programming, counting starts at **0**, not 1. Negative numbers count from the end:

Character	P	y	t	h	o	n
Index	0	1	2	3	4	5
Negative index	-6	-5	-4	-3	-2	-1

WATCH OUT

Asking for an index that doesn't exist, like `word[10]`, causes an `IndexError: string index out of range`.

Special characters

PYTHON

```python
print("Line one\nLine two")     # \n means new line
print("Name:\tSara")            # \t means tab
print("It's easy")               # an apostrophe inside double quotes is fine
print("=" * 20)                 # repeat a string
```

OUTPUT

```
Line one
Line two
Name:   Sara
It's easy
====================
```

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Ask for the user's name. Print it in UPPERCASE, and how many letters it has.
2. Print the first and the last letter of any word the user types.
3. Print a banner: a line of 30 `=` signs, a title, then another line of 30 `=` signs.
4. Use an f-string to print: `Sara scored 91 points out of 100 (91%)` using variables.

LESSON 6

Making decisions: if, elif, else

Programs need to make choices: *if* the password is right, let them in; *otherwise*, show an error. First we need a way to compare things.

Comparison operators

Operator	Meaning	Example	Result
`==`	is equal to	`5 == 5`	`True`
`!=`	is not equal to	`5 != 3`	`True`
`>`	is greater than	`5 > 8`	`False`
`<`	is less than	`5 < 8`	`True`
`>=`	is greater than or equal to	`5 >= 5`	`True`
`<=`	is less than or equal to	`4 <= 3`	`False`

WATCH OUT

`=` **stores** a value. `==` **compares** two values. Mixing them up is one of the most common beginner mistakes.

if and else

PYTHON

```
age = 16

if age >= 18:
    print("You can vote.")
else:
    print("You cannot vote yet.")

print("Done.")
```

OUTPUT

```
You cannot vote yet.
Done.
```

Three important details:

- The `if` line ends with a **colon** `:`.
- The lines that belong to the `if` are **indented** (pushed right) by **4 spaces**. Indentation is how Python knows which lines are inside the `if`.
- `print("Done.")` is not indented, so it runs no matter what.

elif: more than two choices

PYTHON

```python
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(f"Your grade is {grade}")
```

OUTPUT

```
Your grade is B
```

Python checks each condition **from top to bottom** and runs **only the first one that is true**. 85 is not ≥ 90, but it is ≥ 80, so the grade is B, and Python skips the rest.

Combining conditions: and, or, not

Operator	True when...	Example
`and`	**both** sides are true	`age >= 13 and age <= 19` (a teenager)
`or`	**at least one** side is true	`day == "Sat" or day == "Sun"` (the weekend)
`not`	the condition is false	`not is_raining`

PYTHON

```python
age = 20
has_ticket = True

if age >= 18 and has_ticket:
    print("Welcome to the concert!")
elif not has_ticket:
    print("You need a ticket.")
else:
    print("Sorry, adults only.")
```

OUTPUT

```
Welcome to the concert!
```

Even or odd?

PYTHON
```python
number = 7
if number % 2 == 0:
    print(f"{number} is even")
else:
    print(f"{number} is odd")
```

OUTPUT
```
7 is odd
```

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Ask for a number and say whether it is positive, negative or zero.
2. Ask for a password. If it is `"open123"` print `Access granted`, otherwise `Access denied`.
3. Ask for three numbers and print the biggest one (don't use `max()`, use `if`).
4. Leap year: a year is a leap year if it is divisible by 4, **except** years divisible by 100 that are **not** divisible by 400. Check 2024, 1900 and 2000.

LESSON 7

Repeating things: while loops

A **loop** repeats code. A `while` loop repeats **as long as a condition is true**.

PYTHON
```python
count = 1
while count <= 5:
    print(count)
    count += 1
print("Finished!")
```

OUTPUT
```
1
2
3
4
5
Finished!
```

Step by step: `count` starts at 1. Is `1 <= 5`? Yes, so print 1 and add 1. Is `2 <= 5`? Yes... When `count` becomes 6, the condition is false and the loop stops.

WATCH OUT

If you forget `count += 1`, `count` stays 1 forever and the loop **never ends**. This is an **infinite loop**. Press **Ctrl + C** in the terminal (or the Stop button) to stop it.

Countdown

PYTHON

```python
n = 3
while n > 0:
    print(n)
    n -= 1
print("Liftoff!")
```

OUTPUT

```
3
2
1
Liftoff!
```

Repeat until the user gets it right

PYTHON

```python
password = ""
while password != "python123":
    password = input("Enter password: ")
print("Access granted")
```

OUTPUT

```
Enter password: hello
Enter password: python
Enter password: python123
Access granted
```

Adding up numbers

PYTHON

```python
total = 0
number = 1
while number <= 100:
    total += number
    number += 1
print(total)
```

OUTPUT

```
5050
```

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Print all even numbers from 2 to 20 with a `while` loop.
2. Keep asking the user for numbers until they type `0`. Then print the total of all numbers typed.
3. Print the 7 times table up to 7 × 10.
4. A bank account starts at 1000 and grows 5% every year. How many years until it is over 2000?

LESSON 8

for loops and range

A `for` loop goes through a sequence of items **one at a time**. Combined with `range()`, it is the easiest way to repeat something a set number of times.

PYTHON
```
for i in range(5):
    print(i)
```

OUTPUT
```
0
1
2
3
4
```

`range(5)` gives the numbers 0, 1, 2, 3, 4. That's five numbers, starting at 0 and **stopping before 5**.

Code	Numbers you get	Meaning
`range(5)`	0 1 2 3 4	from 0, stop before 5
`range(1, 6)`	1 2 3 4 5	from 1, stop before 6
`range(0, 11, 2)`	0 2 4 6 8 10	from 0, stop before 11, step 2
`range(10, 0, -1)`	10 9 8 7 6 5 4 3 2 1	count down, step -1

Looping through a string

PYTHON
```
for letter in "cat":
    print(letter)
```

OUTPUT
```
c
a
t
```

A times table

PYTHON

```python
number = 7
for i in range(1, 11):
    print(f"{number} x {i} = {number * i}")
```

OUTPUT

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

break and continue

`break` **stops** the loop immediately. `continue` **skips** to the next round.

PYTHON

```python
for i in range(1, 10):
    if i == 5:
        break
    print(i)
```

OUTPUT

```
1
2
3
4
```

PYTHON

```python
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

OUTPUT

```
1
2
4
5
```

Loops inside loops

PYTHON

```python
for row in range(1, 5):
    print("*" * row)
```

OUTPUT

```
*
**
***
****
```

PYTHON

```python
for row in range(1, 4):
    for col in range(1, 4):
        print(row * col, end=" ")
    print()
```

OUTPUT

```
1 2 3
2 4 6
3 6 9
```

`end=" "` tells `print` to end with a space instead of a new line, and an empty `print()` moves to the next line.

KEY IDEA

Use a `for` **loop** when you know how many times to repeat, or want to go through items. Use a `while` **loop** when you repeat *until something happens*, like a correct answer.

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Print the numbers from 10 down to 1, then `Happy New Year!`
2. Ask for a word and count how many vowels (a, e, i, o, u) it has.
3. **FizzBuzz** (a famous interview question): print 1 to 30, but print `Fizz` for multiples of 3, `Buzz` for multiples of 5, and `FizzBuzz` for multiples of both.
4. Ask for a number `n` and print a right-angled triangle of `#` that is `n` rows tall.

PROJECT 1

Checkpoint project: number-guessing game

Time to combine everything into a real game. The computer picks a secret number and the player guesses until they find it. We will build it in **three versions**, adding a little each time, just like real developers do.

Version 1: one guess

PYTHON

```python
secret = 7
guess = int(input("Guess my number (1-10): "))

if guess == secret:
    print("Correct!")
else:
    print("Wrong! It was", secret)
```

Version 2: keep guessing until correct

PYTHON

```python
secret = 7
guess = 0

while guess != secret:
    guess = int(input("Guess my number (1-10): "))
    if guess != secret:
        print("Nope, try again.")

print("Correct!")
```

Version 3: random number, hints and score

Python has a built-in **module** called `random` (more about modules in Lesson 16). `random.randint(1, 100)` picks a random whole number from 1 to 100.

PYTHON · guessing_game.py

```python
import random

secret = random.randint(1, 100)
attempts = 0

print("I'm thinking of a number between 1 and 100.")

while True:
    guess = int(input("Your guess: "))
    attempts += 1

    if guess < secret:
        print("Too low!")
    elif guess > secret:
        print("Too high!")
    else:
        print(f"Correct! You got it in {attempts} attempts.")
        break
```

EXAMPLE GAME

```
I'm thinking of a number between 1 and 100.
Your guess: 50
Too high!
Your guess: 25
Too low!
Your guess: 37
Too high!
Your guess: 31
Correct! You got it in 4 attempts.
```

`while True:` loops forever **on purpose**. We leave the loop with `break` when the guess is correct. This is a very common pattern.

TIP

Smart strategy: always guess the middle of what's left (50, then 25 or 75, and so on). You will never need more than 7 guesses for 1 to 100. This idea is called **binary search**, and you will meet it again in Lesson 23.

CHALLENGES

Upgrade your game. Each one is a real feature request:

1. Give the player only **7 attempts**. If they run out, show the secret number.
2. After winning, ask **"Play again? (y/n)"** and restart if they type `y`.
3. Add **difficulty levels**: easy (1–10), medium (1–100), hard (1–1000).
4. Keep track of the **best score** (fewest attempts) across games.

CHECKPOINT

Quiz 1 and Stage 1 checklist

Answer without running code first. Then check with Python, and with the answers in Appendix A. **Aim for 8 out of 10** before moving on.

1. What does `print(7 // 2)` show?
2. What is the type of `"42"`?
3. What is wrong with `if x = 5:`?
4. What does `for i in range(2, 8, 2): print(i)` print?
5. What type of value does `input()` always return?
6. What is `x` after: `x = 10`, then `x += 5`, then `x *= 2`?
7. What is the difference between `=` and `==`?
8. What does `"ab" * 3` give?
9. How do you stop a program stuck in an infinite loop?
10. What does `print(len("hello world"))` show?

Can you do these on your own?

- ✓ Write and run a `.py` file
- ✓ Create variables and update them with `+=`
- ✓ Use `int`, `float`, `str` and `bool`, and convert between them
- ✓ Ask the user for input and use it in a calculation
- ✓ Use f-strings to build messages
- ✓ Write `if` / `elif` / `else` with `and` / `or`
- ✓ Write `while` and `for` loops, and use `break`
- ✓ Build the guessing game without looking at the code

PART 3

Python, Stage 2: organising data and code

Lists, dictionaries, functions, errors, files and libraries. These are the tools for writing real, useful programs.

In this part you will learn

- ✓ How to store many values with lists, tuples, sets and dictionaries
- ✓ How to write your own functions so code can be reused
- ✓ Advanced string tricks: slicing, splitting and formatting
- ✓ How to read error messages and handle errors without crashing
- ✓ How to save and load data with files
- ✓ How to use Python's built-in modules and install new packages
- ✓ How to build a to-do list app that remembers your tasks

LESSON 9

Lists: many values in one variable

So far each variable held one value. A **list** holds many values, in order. You write it with square brackets `[ ]` and commas.

PYTHON

```python
fruits = ["apple", "banana", "cherry"]
print(fruits)
print(fruits[0])        # first item (index 0!)
print(len(fruits))      # how many items

fruits.append("mango")  # add to the end
fruits[1] = "blueberry" # change an item
print(fruits)
```

OUTPUT

```
['apple', 'banana', 'cherry']
apple
3
['apple', 'blueberry', 'cherry', 'mango']
```

The most useful list operations

Code	What it does
`items.append(x)`	Add `x` to the end
`items.insert(0, x)`	Insert `x` at position 0 (the start)
`items.remove(x)`	Remove the first item equal to `x`
`items.pop()`	Remove **and return** the last item (`items.pop(0)` removes the first)
`items.sort()`	Sort the list itself (A to Z or smallest to biggest)
`sorted(items)`	Return a **new** sorted list, leaving the original alone
`items.reverse()`	Reverse the order
`x in items`	`True` if `x` is in the list
`items.index(x)`	The position of `x`
`items.count(x)`	How many times `x` appears
`len(items)`, `sum(items)`, `min(items)`, `max(items)`	Length, total, smallest, biggest

Looping through a list

PYTHON
```python
scores = [72, 88, 95, 61]
total = 0
for score in scores:
    total += score
print("Average:", total / len(scores))
```

OUTPUT
```
Average: 79.0
```

Python has built-in shortcuts for this kind of thing:

PYTHON
```python
scores = [72, 88, 95, 61]
print(sum(scores), max(scores), min(scores))
print(sorted(scores))
```

OUTPUT
```
316 95 61
[61, 72, 88, 95]
```

Slicing: taking part of a list

`items[start:stop]` gives the items from `start` up to, **but not including**, `stop`:

PYTHON

```
letters = ["a", "b", "c", "d", "e"]
print(letters[1:3])   # positions 1 and 2
print(letters[:2])    # from the start, stop before 2
print(letters[-2:])   # the last two
```

OUTPUT

```
['b', 'c']
['a', 'b']
['d', 'e']
```

Getting the position too: enumerate

PYTHON

```
for number, fruit in enumerate(["apple", "banana", "cherry"], start=1):
    print(number, fruit)
```

OUTPUT

```
1 apple
2 banana
3 cherry
```

Checking if something is in a list

PYTHON

```
guests = ["Sara", "Omar", "Lina"]
if "Omar" in guests:
    print("Omar is invited")
```

OUTPUT

```
Omar is invited
```

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Make a shopping list program: keep asking for items, add each to a list, stop at `done`, then print the list with numbers.
2. Find the biggest number in `[4, 19, 7, 33, 2]` **without** using `max()`.
3. Given `scores = [55, 82, 71, 90, 48, 67]`, count how many are 70 or more.
4. Remove duplicates: from `[1, 2, 2, 3, 1, 4]` make `[1, 2, 3, 4]` with a loop and a new list.

LESSON 10

Tuples and sets

Tuples: lists that cannot change

A **tuple** is like a list, but once created it cannot be changed. Use round brackets. Tuples are great for values that belong together and should stay fixed, like coordinates or a date.

PYTHON
```
point = (3, 4)
x, y = point          # "unpacking": take the values out
print(x, y)
print(point[0])
```

OUTPUT
```
3 4
3
```

PYTHON
```
point[0] = 10
```

ERROR
```
TypeError: 'tuple' object does not support item assignment
```

Sets: no duplicates allowed

A **set** is a collection where every item is **unique**, and there is no fixed order. Use curly brackets.

PYTHON
```
names = ["sam", "ali", "sam", "mia", "ali"]
unique = set(names)
print(len(unique))
print(sorted(unique))
```

OUTPUT
```
3
['ali', 'mia', 'sam']
```

Sets are perfect for comparing groups:

PYTHON
```
a = {1, 2, 3}
b = {2, 3, 4}
print(a & b)   # in both (intersection)
print(a | b)   # in either (union)
print(a - b)   # in a but not in b (difference)
```

OUTPUT
```
{2, 3}
{1, 2, 3, 4}
{1}
```

Which collection should I use?

Type	Looks like	Ordered?	Can change?	Duplicates?	Use it for
list	`[1, 2, 3]`	Yes	Yes	Yes	Most things: a to-do list, scores
tuple	`(1, 2, 3)`	Yes	No	Yes	Fixed data: coordinates, RGB colours
set	`{1, 2, 3}`	No	Yes	No	Unique items, fast "is it in here?" checks
dict	`{"a": 1}`	Yes	Yes	Keys: no	Looking things up by name (next lesson!)

LESSON 11

Dictionaries: looking things up by name

A **dictionary** (`dict`) stores **key: value** pairs. It is like a real dictionary: you look up a word (the key) to find its meaning (the value).

PYTHON
```
student = {
    "name": "Sara",
    "age": 16,
    "grade": "A",
}

print(student["name"])     # look up a value by its key
student["age"] = 17        # change a value
student["city"] = "Cairo"  # add a new pair
print(student)
```

OUTPUT
```
Sara
{'name': 'Sara', 'age': 17, 'grade': 'A', 'city': 'Cairo'}
```

Safe lookups and checking keys

PYTHON
```
student = {"name": "Sara", "age": 16}
print(student.get("phone", "not given"))   # a default if the key is missing
print("age" in student)
```

OUTPUT
```
not given
True
```

WATCH OUT

`student["phone"]` crashes with a `KeyError` if the key doesn't exist. Use `.get()` when you are not sure a key is there.

Looping through a dictionary

PYTHON

```python
prices = {"apple": 0.5, "bread": 2.25, "milk": 1.2}
for item, price in prices.items():
    print(f"{item}: ${price:.2f}")
```

OUTPUT

```
apple: $0.50
bread: $2.25
milk: $1.20
```

`.keys()` gives just the keys, `.values()` gives just the values, and `.items()` gives both as pairs.

A classic: counting words

PYTHON

```python
text = "the cat and the dog and the bird"
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

OUTPUT

```
{'the': 3, 'cat': 1, 'and': 2, 'dog': 1, 'bird': 1}
```

Lists of dictionaries: real-world data

Most real data looks like this. Think of each dictionary as a row in a spreadsheet:

PYTHON

```python
students = [
    {"name": "Sara", "score": 91},
    {"name": "Omar", "score": 78},
    {"name": "Lina", "score": 85},
]

for s in students:
    print(f"{s['name']} scored {s['score']}")
```

OUTPUT

```
Sara scored 91
Omar scored 78
Lina scored 85
```

IN THE REAL WORLD

Data from websites and apps (called JSON, see Lesson 28) looks almost exactly like Python lists and dictionaries. Learn these well and you will understand most of the data you meet.

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Make a phone book dict with 3 names and numbers. Ask for a name and print the number, or `Not found`.
2. Ask the user for a sentence and count how often each word appears.
3. A shop has `stock = {"pens": 10, "books": 4}` and `prices = {"pens": 1.5, "books": 12}`. Print the total value of all stock.
4. From the `students` list above, print the name of the student with the highest score.

LESSON 12

Functions: reusable blocks of code

A **function** is a named block of code that does one job. You **define** it once, then **call** it as many times as you want. You have used functions already: `print()`, `len()`, `input()`. Now you'll make your own.

PYTHON

```
def greet():
    print("Hello!")
    print("Welcome to Python.")

greet()
greet()
```

OUTPUT

```
Hello!
Welcome to Python.
Hello!
Welcome to Python.
```

- `def` means **define**. Then the function name, brackets and a colon.
- The indented lines are the function's **body**.
- Defining a function does **not** run it. Calling it, with `greet()`, does.

Parameters: giving a function information

PYTHON

```
def greet(name):
    print(f"Hello, {name}!")

greet("Sara")
greet("Omar")
```

OUTPUT

```
Hello, Sara!
Hello, Omar!
```

return: getting an answer back

PYTHON

```python
def add(a, b):
    return a + b

result = add(3, 4)
print(result)
print(add(10, 20) * 2)
```

OUTPUT

```
7
60
```

KEY IDEA

`print()` **shows** a value on the screen. `return` **gives a value back** to the code that called the function, so you can store it or use it in calculations. Most functions should `return`, not `print`.

Default values

PYTHON

```python
def power(base, exponent=2):
    return base ** exponent

print(power(5))       # uses the default exponent of 2
print(power(2, 10))
```

OUTPUT

```
25
1024
```

Scope: variables live inside their function

PYTHON

```python
def make_message():
    message = "inside the function"
    return message

print(make_message())
```

OUTPUT

```
inside the function
```

`message` only exists **inside** `make_message`. If you tried `print(message)` outside the function, you would get a `NameError`. This is called **scope**, and it keeps functions from interfering with each other.

A complete example: temperature converter

PYTHON

```python
def c_to_f(celsius):
    """Convert Celsius to Fahrenheit."""
    return celsius * 9 / 5 + 32


def f_to_c(fahrenheit):
    """Convert Fahrenheit to Celsius."""
    return (fahrenheit - 32) * 5 / 9


print(c_to_f(100))
print(f_to_c(32))
print(round(f_to_c(100), 1))
```

OUTPUT

```
212.0
0.0
37.8
```

The text in triple quotes right under `def` is a **docstring**. It describes what the function does. Editors show it when you hover over the function.

Rules for good functions

- **One function, one job.** If you need the word "and" to describe it, split it.
- **Name it with a verb:** `calculate_total`, `is_valid_email`, `send_message`.
- **Keep it short.** Under about 20 lines is a good target.
- **Don't Repeat Yourself (DRY).** If you copy-paste code, it probably belongs in a function.

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Write `is_even(n)` that returns `True` or `False`.
2. Write `rectangle_area(width, height)` and `circle_area(radius)` (use `3.14159`).
3. Write `count_vowels(text)` that returns how many vowels a string has.
4. Write `is_prime(n)` that returns `True` if `n` is a prime number. Use it to print all primes below 50.

LESSON 13

Strings in depth

Slicing strings

PYTHON

```
text = "Hello, World"
print(text[0:5])    # characters 0 to 4
print(text[7:])     # from 7 to the end
print(text[::-1])   # reversed!
```

OUTPUT

```
Hello
World
dlroW ,olleH
```

The string methods you will use most

PYTHON

```
sentence = "  learning python is fun  "
clean = sentence.strip()          # remove spaces at both ends
print(clean)

words = clean.split()             # break into a list of words
print(words)
print("-".join(words))            # glue a list together
print(clean.replace("fun", "awesome"))
print(clean.title())
```

OUTPUT

```
learning python is fun
['learning', 'python', 'is', 'fun']
learning-python-is-fun
learning python is awesome
Learning Python Is Fun
```

Method	What it does	Example result
`s.strip()`	Remove spaces/new lines from both ends	`" hi "` becomes `"hi"`
`s.split(",")`	Break into a list at each comma	`"a,b"` becomes `['a', 'b']`
`", ".join(list)`	Join a list into one string	`['a', 'b']` becomes `"a, b"`
`s.replace(a, b)`	Replace every `a` with `b`	
`s.find("x")`	Position of `"x"` (or `-1` if not found)	
`s.startswith("A")`	Does it start with `"A"`?	`True` / `False`
`s.isdigit()`	Is it all digits?	`"123"` gives `True`
`s.count("a")`	How many times `"a"` appears	

WATCH OUT

Strings **cannot be changed** in place. `name.upper()` gives you a *new* string. To keep it, store it: `name = name.upper()`.

Formatting numbers in f-strings

PYTHON

```python
price = 4.5
population = 1234567
print(f"Price: ${price:.2f}")        # 2 decimal places
print(f"Population: {population:,}")  # thousands separator
print(f"[{'left':<10}]")              # pad to 10 characters
print(f"[{'right':>10}]")
```

OUTPUT

```
Price: $4.50
Population: 1,234,567
[left      ]
[     right]
```

Example: is it a palindrome?

PYTHON

```python
def is_palindrome(word):
    word = word.lower()
    return word == word[::-1]

print(is_palindrome("Racecar"))
print(is_palindrome("Python"))
```

OUTPUT

```
True
False
```

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Ask for a sentence and print how many words it has.
2. Make an acronym maker: `"portable network graphics"` should become `PNG`.
3. Improve `is_palindrome` so `"Never odd or even"` counts as a palindrome (ignore spaces).
4. Ask for an email address and print the part before the `@` (the username).

LESSON 14

Errors and exceptions

There are three kinds of errors:

Kind	When	Example
Syntax error	Before the program runs. The code breaks Python's grammar.	A missing bracket or colon
Runtime error (exception)	While the program runs. It crashes.	Dividing by zero, a missing file
Logic error	The program runs, but gives the **wrong answer**. No message at all!	Using + where you meant -

How to read an error message (a traceback)

PYTHON

```
number = 0
print(10 / number)
```

ERROR

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(10 / number)
          ~~~^~~~~~~~
ZeroDivisionError: division by zero
```

KEY IDEA

Read tracebacks **from the bottom up**. The **last line** tells you the type of error and what happened. The lines above tell you **where**: the file and the line number.

The errors you will meet most often

Error	What it means	Usual fix
`SyntaxError`	Python can't understand the code	Check brackets, quotes and colons
`IndentationError`	Spaces at the start of a line are wrong	Use 4 spaces consistently
`NameError`	You used a name that doesn't exist	Check spelling and capitals; define it first
`TypeError`	Wrong type for the operation	Convert with `int()` or `str()`
`ValueError`	Right type, but a bad value	e.g. `int("hello")`. Check the input
`IndexError`	List position doesn't exist	Remember indexes start at 0; check `len()`
`KeyError`	Dictionary key doesn't exist	Check the key, or use `.get()`
`AttributeError`	That type has no such method	e.g. calling `.append()` on a string
`ZeroDivisionError`	Divided by zero	Check before dividing
`FileNotFoundError`	The file isn't there	Check the file name and folder
`ModuleNotFoundError`	A library isn't installed	`pip install` it (Lesson 16)

Handling errors with try and except

Instead of crashing, you can **catch** an error and do something sensible:

PYTHON

```
try:
    age = int(input("Age: "))
    print("In 10 years you will be", age + 10)
except ValueError:
    print("Please type a number, like 16.")
```

OUTPUT

```
Age: sixteen
Please type a number, like 16.
```

Python **tries** the code in the `try` block. If a `ValueError` happens, it jumps to the `except` block instead of crashing.

The pattern: keep asking until the input is valid

PYTHON

```
while True:
    try:
        age = int(input("Age: "))
        break                    # valid, so leave the loop
    except ValueError:
        print("That is not a number. Try again.")

print("Thanks! You are", age)
```

Functions that handle errors

PYTHON

```python
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return None

print(safe_divide(10, 2))
print(safe_divide(5, 0))
```

OUTPUT

```
5.0
None
```

`None` is Python's special value for "nothing" or "no value".

WATCH OUT

Don't write a bare `except:` that catches **every** error. It also hides real bugs. Always name the error you expect, like `except ValueError:`.

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Write `get_number(prompt)` that keeps asking until the user types a valid decimal number, then returns it.
2. Ask for a list position and print `fruits[position]`, catching `IndexError` and `ValueError`.
3. Find the logic error: `def average(nums): total = 0` / `for n in nums: total = n` / `return total / len(nums)`. Why does `average([2, 4, 6])` give 2.0?

LESSON 15

Working with files

Variables disappear when your program ends. To **remember** things, like high scores, notes or settings, save them in a **file**.

PYTHON

```python
# write to a file ("w" = write)
with open("notes.txt", "w") as file:
    file.write("First line\n")
    file.write("Second line\n")

# read it back ("r" = read)
with open("notes.txt", "r") as file:
    content = file.read()

print(content)
```

OUTPUT

```
First line
Second line
```

- `open(name, mode)` opens a file. `with` makes sure it is **closed automatically** afterwards, even if something goes wrong.
- `\n` is the new-line character. `write()` does not add new lines for you.
- The file is created in the **same folder** your program runs from.

Mode	Meaning	Careful!
`"r"`	Read (the default)	Error if the file doesn't exist
`"w"`	Write	**Erases** everything already in the file!
`"a"`	Append: add to the end	Safe: keeps the existing content

Reading line by line

PYTHON

```python
with open("notes.txt") as file:
    for line in file:
        print(line.strip())    # strip() removes the \n at the end
```

Saving structured data with JSON

For lists and dictionaries, use the built-in `json` module. JSON is the most common format for data on the internet.

PYTHON
```python
import json

data = {"name": "Sara", "scores": [90, 85]}

with open("data.json", "w") as file:
    json.dump(data, file)        # save

with open("data.json") as file:
    loaded = json.load(file)     # load

print(loaded["scores"])
```

OUTPUT
```
[90, 85]
```

GOOD TO KNOW

Other common formats: **CSV** (spreadsheet tables, use the `csv` module), **TXT** (plain text), and databases for big data (Lesson 29).

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Make a diary: ask for an entry and **append** it to `diary.txt` with today's date (see Lesson 16 for dates, or just type it).
2. Write a program that counts how many lines are in a text file.
3. Save a high score to `highscore.txt`. When the program starts, load it, and only overwrite it when the new score is higher.

LESSON 16

Modules and libraries

A **module** is a file of Python code that you can **import** to use its functions. Python comes with hundreds of modules built in. This is called the **standard library**, and it's why people say Python comes with "batteries included".

PYTHON
```python
import math

print(math.sqrt(16))
print(math.pi)
print(math.floor(3.7), math.ceil(3.2))
```

OUTPUT
```
4.0
3.141592653589793
3 4
```

Three ways to import

```python
import random                  # use as random.randint(...)
from random import randint     # use as randint(...)
import datetime as dt          # a short nickname: dt.date.today()
```

```python
import random
from datetime import date

print(random.randint(1, 6))               # a dice roll
print(random.choice(["rock", "paper", "scissors"]))
print(date.today())                       # today's date
```

```
OUTPUT (yours will be different: it's random!)
4
paper
2026-10-06
```

Standard library highlights

Module	What it is for
`math`	Square roots, pi, rounding up/down, trigonometry
`random`	Random numbers and choices (games, simulations)
`datetime`	Dates and times
`time`	Pausing (`time.sleep(1)`) and measuring time
`json`, `csv`	Reading and writing data files
`os`, `pathlib`	Working with files and folders
`statistics`	Mean, median, mode
`sqlite3`	A real database built into Python (Lesson 29)
`turtle`	Drawing pictures with code. Great fun for beginners!
`tkinter`	Making desktop windows with buttons (Part 7)

Fun break: drawing with turtle

```python
import turtle

pen = turtle.Turtle()
for side in range(4):
    pen.forward(100)   # move 100 steps
    pen.right(90)      # turn 90 degrees

turtle.done()          # keep the window open
```

A window opens and draws a square. Change `4` to `5` and `90` to `72` to draw a pentagon. Can you draw a star?

Installing packages other people made

Beyond the standard library, there are **over 600,000 free packages** on PyPI (the Python Package Index, pypi.org). You install them with **pip** in the terminal:

TERMINAL
```
python -m pip install requests
```

On Mac and Linux, use `python3 -m pip install requests`.

Package	What it does
`requests`	Download web pages and talk to APIs
`pandas`	Work with tables of data, like a super-powered spreadsheet
`numpy`	Fast maths with large amounts of numbers
`matplotlib`	Draw charts and graphs
`flask`, `django`, `fastapi`	Build websites and web APIs
`pygame`	Make 2D games
`pytest`	Test your code
`beautifulsoup4`	Pull information out of web pages
`scikit-learn`, `pytorch`	Machine learning and AI

Your own modules

Any `.py` file is a module. If you have `helpers.py` with a function `shout(text)`, another file in the same folder can use it:

PYTHON · helpers.py
```
# helpers.py
def shout(text):
    return text.upper() + "!"
```

PYTHON · main.py
```
# main.py
from helpers import shout

print(shout("hello"))
```

OUTPUT
```
HELLO!
```

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Make a dice roller: ask how many dice to roll, then print each roll and the total.
2. Make a random password generator using `random.choice` and the `string` module (`string.ascii_letters`, `string.digits`).
3. Use `datetime` to print how many days are left until your next birthday.
4. Use `turtle` to draw a star (hint: 5 sides, turn 144 degrees).

PROJECT 2

Checkpoint project: to-do list app

Let's build a real, useful program: a to-do list that **remembers your tasks** even after you close it. It uses functions, lists, loops, `if`, files and error handling: everything from Parts 2 and 3.

Plan first, code second

Professional developers plan before they code. Our app needs to:

1. Load saved tasks from a file when it starts (or start empty)
2. Show a menu: show tasks, add a task, remove a task, quit
3. Save the tasks to the file after every change
4. Never crash, even if the user types something strange

Each job becomes a **function**. Then `main()` ties them together.

PYTHON · todo.py

```
FILENAME = "todo.txt"


def load_tasks():
    """Read tasks from the file. Return an empty list if there is no file yet."""
    tasks = []
    try:
        with open(FILENAME) as file:
            for line in file:
                if line.strip():
                    tasks.append(line.strip())
    except FileNotFoundError:
        pass  # first run: no file yet, that's fine
    return tasks


def save_tasks(tasks):
    with open(FILENAME, "w") as file:
        for task in tasks:
            file.write(task + "\n")


def show_tasks(tasks):
    if not tasks:
        print("No tasks yet!")
        return
    for number, task in enumerate(tasks, start=1):
        print(f"{number}. {task}")


def main():
    tasks = load_tasks()
    while True:
        print("\n=== TO-DO LIST ===")
        print("1) Show  2) Add  3) Remove  4) Quit")
        choice = input("Choose 1-4: ").strip()
```

```
        if choice == "1":
            show_tasks(tasks)
        elif choice == "2":
            task = input("New task: ").strip()
            if task:
                tasks.append(task)
                save_tasks(tasks)
                print("Added!")
        elif choice == "3":
            show_tasks(tasks)
            try:
                number = int(input("Number to remove: "))
            except ValueError:
                print("Please type a number.")
                continue
            if 1 <= number <= len(tasks):
                removed = tasks.pop(number - 1)
                save_tasks(tasks)
                print(f"Removed: {removed}")
            else:
                print("No task with that number.")
        elif choice == "4":
            print("Goodbye!")
            break
        else:
            print("Please choose 1, 2, 3 or 4.")


main()
```

EXAMPLE RUN

```
=== TO-DO LIST ===
1) Show  2) Add  3) Remove  4) Quit
Choose 1-4: 2
New task: Finish Lesson 16
Added!

=== TO-DO LIST ===
1) Show  2) Add  3) Remove  4) Quit
Choose 1-4: 1
1. Finish Lesson 16
```

Notice the check `if 1 <= number <= len(tasks)`. Without it, typing `0` would remove the **last** task, because `tasks.pop(-1)` removes the last item. Thinking about strange input like this is called handling **edge cases**, and it's what separates working code from *good* code.

CHALLENGES

Make it yours:

1. Add option **5) Mark as done**, which puts `[x]` in front of a task.
2. Save tasks as **JSON** with a `done` field: `{"task": "...", "done": false}`.
3. Add **priorities** (high/medium/low) and show high-priority tasks first.
4. Add a **due date** to each task and warn about tasks that are overdue.

Bonus project: a quiz game

PYTHON · quiz.py

```python
questions = [
    {"q": "What does len(\"hi\") return?", "a": "2"},
    {"q": "Which keyword defines a function?", "a": "def"},
    {"q": "What is 2 ** 3?", "a": "8"},
]

score = 0
for item in questions:
    answer = input(item["q"] + " ").strip().lower()
    if answer == item["a"]:
        print("Correct!")
        score += 1
    else:
        print(f"Wrong, the answer is {item['a']}")

print(f"You scored {score}/{len(questions)}")
```

CHECKPOINT

Quiz 2 and Stage 2 checklist

1. What does `[10, 20, 30][1]` give?
2. What is the difference between `items.sort()` and `sorted(items)`?
3. Which collection automatically removes duplicates?
4. How do you safely get a dictionary value that might be missing?
5. What is the difference between `print` and `return` in a function?
6. What does `"a,b,c".split(",")` return?
7. What does `"Python"[::-1]` give?
8. Which file mode erases a file's content: `"r"`, `"w"` or `"a"`?
9. In which order should you read a traceback?
10. What command installs the package `requests`?

Can you do these on your own?

- ✓ Create, change, loop over and slice lists
- ✓ Choose between a list, tuple, set and dictionary
- ✓ Store and look up data in dictionaries, including lists of dictionaries
- ✓ Write functions with parameters and `return` values
- ✓ Use string methods: `split`, `join`, `strip`, `replace`
- ✓ Read a traceback and fix the error
- ✓ Catch errors with `try` / `except`
- ✓ Read and write text and JSON files
- ✓ Import modules and install packages with pip
- ✓ Build the to-do app with at least one challenge feature

PART 4

Python, Stage 3: thinking like a developer

Objects, debugging, testing, clean code and algorithms. These are the skills that turn code that works into good code.

In this part you will learn

- ✓ Object-oriented programming: classes, objects and inheritance
- ✓ Pythonic shortcuts: comprehensions, `zip`, `lambda`
- ✓ How to debug step by step with a real debugger
- ✓ How to test your code automatically
- ✓ How to write clean code that others can read
- ✓ Algorithms, Big-O and the data structures interviewers ask about
- ✓ How to build a contact book app with classes and JSON

LESSON 17

Classes and objects

So far, data (variables) and actions (functions) have lived separately. **Object-oriented programming (OOP)** puts them together into **objects**.

- A **class** is a **blueprint**, like a cookie cutter or the plan for a house.
- An **object** is a **thing built from the blueprint**: a cookie, or an actual house. You can make many objects from one class.
- Objects have **attributes** (data they know, like `name`) and **methods** (things they can do, like `bark()`).

PYTHON

```python
class Dog:
    def __init__(self, name, age):
        self.name = name      # attribute
        self.age = age        # attribute

    def bark(self):           # method
        print(f"{self.name} says Woof!")

    def have_birthday(self):  # method
        self.age += 1


rex = Dog("Rex", 3)       # create an object
bella = Dog("Bella", 5)   # create another one

rex.bark()
bella.bark()
rex.have_birthday()
print(rex.age)
```

OUTPUT

```
Rex says Woof!
Bella says Woof!
4
```

What do __init__ and self mean?

- `__init__` (two underscores on each side) runs **automatically** when you create an object. It sets up the starting attributes. It is called the **constructor**.
- `self` means **"this particular object"**. When you call `rex.bark()`, `self` is `rex`. So `self.name` is Rex's name, not Bella's.
- Every method needs `self` as its first parameter, but you don't pass it yourself. Python does that for you.

A more useful example: a bank account

PYTHON

```python
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit must be positive.")
            return
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            print("Not enough money!")
            return
        self.balance -= amount

    def __str__(self):
        return f"{self.owner}: ${self.balance:.2f}"


account = BankAccount("Sara", 100)
account.deposit(50)
account.withdraw(500)
account.withdraw(30)
print(account)
```

OUTPUT

```
Not enough money!
Sara: $120.00
```

`__str__` is a special method that decides what `print(account)` shows. Without it, you would see something unhelpful like `<__main__.BankAccount object at 0x7f...>`.

KEY IDEA

The object **protects its own data**: the only way to change the balance is through `deposit` and `withdraw`, which check the rules. This is the main reason OOP exists: it keeps data and the rules for that data together.

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Write a `Rectangle` class with `width` and `height`, and methods `area()` and `perimeter()`.
2. Write a `Student` class with a name and a list of grades, a method `add_grade(g)`, and a method `average()`.
3. Add a `history` list to `BankAccount` that records every deposit and withdrawal, and a method that prints it.

LESSON 18

Inheritance and the four ideas of OOP

Inheritance lets a new class **reuse** everything from an existing class and then add or change things. The new class is the **child** (subclass). The original is the **parent** (superclass).

PYTHON

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "..."

    def introduce(self):
        print(f"I am {self.name} and I say {self.speak()}")


class Cat(Animal):          # Cat inherits from Animal
    def speak(self):
        return "Meow"


class Cow(Animal):
    def speak(self):
        return "Moo"


for animal in [Cat("Tom"), Cow("Daisy"), Animal("Mystery")]:
    animal.introduce()
```

OUTPUT

```
I am Tom and I say Meow
I am Daisy and I say Moo
I am Mystery and I say ...
```

`Cat` and `Cow` never define `introduce()` or `__init__`. They **inherit** them from `Animal`. They only **override** (replace) `speak()`.

Adding attributes in a child class with super()

PYTHON

```python
class Person:
    def __init__(self, name):
        self.name = name


class Student(Person):
    def __init__(self, name, school):
        super().__init__(name)   # let the parent set up the name
        self.school = school     # then add the new attribute


s = Student("Lina", "City High")
print(s.name, "-", s.school)
```

OUTPUT

```
Lina - City High
```

The four pillars of OOP (a favourite exam question!)

Pillar	Simple meaning	In our examples
Encapsulation	Keep data and the methods that use it together, and protect the data from outside changes.	Balance only changes through `deposit` / `withdraw`
Abstraction	Hide complicated details behind a simple interface.	You call `account.withdraw(30)` without knowing how it works inside
Inheritance	A class can reuse another class's code.	`Cat` gets `introduce()` from `Animal`
Polymorphism	Different objects respond to the same method call in their own way.	`speak()` gives Meow or Moo depending on the object

TIP

Inheritance means "**is a**" (a Cat *is an* Animal). If the relationship is "**has a**" (a Car *has an* Engine), store one object inside another instead. That is called **composition**, and it is often the better choice.

LESSON 19

Pythonic shortcuts

Experienced Python developers use a few shortcuts all the time. You should be able to read them, and use them when they make code **clearer**.

List comprehensions: build a list in one line

PYTHON

```python
# the long way
squares = []
for n in range(1, 6):
    squares.append(n * n)

# the short way: a list comprehension
squares = [n * n for n in range(1, 6)]
print(squares)

evens = [n for n in range(10) if n % 2 == 0]
print(evens)
```

OUTPUT

```
[1, 4, 9, 16, 25]
[0, 2, 4, 6, 8]
```

Read it as: "**n times n**, **for each n** in range 1 to 5". There is also a dictionary version:

PYTHON

```python
names = ["sara", "omar"]
lengths = {name: len(name) for name in names}
print(lengths)
```

OUTPUT

```
{'sara': 4, 'omar': 4}
```

zip: loop over two lists together

PYTHON

```python
names = ["Sara", "Omar", "Lina"]
scores = [91, 78, 85]
for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

OUTPUT

```
Sara: 91
Omar: 78
Lina: 85
```

lambda and sorting by a key

A `lambda` is a tiny function without a name. It is mostly used to tell `sorted()` **what to sort by**:

PYTHON
```python
students = [("Sara", 91), ("Omar", 78), ("Lina", 85)]
by_score = sorted(students, key=lambda s: s[1], reverse=True)
print(by_score)
```

OUTPUT
```
[('Sara', 91), ('Lina', 85), ('Omar', 78)]
```

One-line if (conditional expression)

PYTHON
```python
age = 20
status = "adult" if age >= 18 else "minor"
print(status)
```

OUTPUT
```
adult
```

any() and all()

PYTHON
```python
scores = [91, 78, 45]
print(any(s < 50 for s in scores))   # is at least one below 50?
print(all(s >= 40 for s in scores))  # are all at least 40?
```

OUTPUT
```
True
True
```

The if __name__ == "__main__" line

You will see this at the bottom of many Python files:

PYTHON
```python
def main():
    print("Running the program")


if __name__ == "__main__":
    main()
```

OUTPUT
```
Running the program
```

It means: "only run `main()` if this file was run **directly**, not when another file **imports** it". This lets a file be both a program and a module.

> **WATCH OUT**
> Shortcuts are only good when they make code **easier** to read. If a comprehension needs more than one `if` or `for`, a normal loop is usually clearer.

LESSON 20

Debugging like a pro

Debugging is not guessing. It is a **process**:

1. **Reproduce** the bug: find the exact steps that make it happen.
2. **Read** the error message, from the bottom up.
3. **Locate** the problem: which line, which function?
4. **Form a guess**: "I think `total` is wrong after the loop."
5. **Test the guess** with a `print` or the debugger.
6. **Fix** it, then **re-test**, including the cases that already worked.

Example: hunt the bug

PYTHON
```python
def average(numbers):
    total = 0
    for n in numbers:
        total = n
    return total / len(numbers)

print(average([2, 4, 6]))   # should be 4.0
```

OUTPUT
```
2.0
```

No error message, just a wrong answer: a **logic error**. Let's add a debug print inside the loop:

PYTHON
```python
def average(numbers):
    total = 0
    for n in numbers:
        total = n
        print(f"DEBUG: n={n}, total={total}")
    return total / len(numbers)

print(average([2, 4, 6]))
```

OUTPUT
```
DEBUG: n=2, total=2
DEBUG: n=4, total=4
DEBUG: n=6, total=6
2.0
```

Now it's obvious: `total` is **replaced** each time instead of **added to**. The fix is `total += n`. Remove the debug print afterwards!

Using a real debugger

A **debugger** pauses your program so you can look inside it while it runs.

- **In Thonny:** click the bug icon (Debug), then use **Step over** (F6) to run one line at a time. Variables appear in the Variables panel.
- **In VS Code:** click just left of a line number to add a red dot. This is a **breakpoint**. Press **F5** to start debugging. The program stops at the red dot. Press **F10** to step to the next line and watch the Variables panel on the left.

More debugging techniques

- **Rubber duck debugging:** explain your code line by line, out loud, to anything (even a toy duck). You will often hear yourself say the mistake.
- **Simplify:** cut the problem down to the smallest piece of code that still shows the bug.
- **Search the error:** paste the last line of the error into a search engine. Someone has almost certainly had it before.
- **Take a break:** a 10-minute walk solves an amazing number of bugs.
- **Ask an AI assistant:** share the code and the full error, and ask it to *explain* the problem, not just fix it.

LESSON 21

Testing your code

How do you know your code works? You could test it by hand every time. But professionals write **automated tests**: code that checks your code. When you change something, the tests tell you instantly if you broke anything.

The simplest test: assert

`assert` checks that something is true. If it isn't, the program stops with an `AssertionError`.

PYTHON

```
def add(a, b):
    return a + b

assert add(2, 3) == 5
assert add(-1, 1) == 0
assert add(0, 0) == 0
print("All tests passed!")
```

OUTPUT

```
All tests passed!
```

Real testing with pytest

pytest is the most popular testing tool for Python. Install it with `python -m pip install pytest`. Put your code in one file and your tests in another file whose name starts with `test_`:

PYTHON · school.py

```python
def is_even(n):
    return n % 2 == 0


def grade(score):
    if score >= 90:
        return "A"
    if score >= 80:
        return "B"
    return "C"
```

PYTHON · test_school.py

```python
from school import is_even, grade


def test_is_even():
    assert is_even(4)
    assert not is_even(7)


def test_grade_boundaries():
    assert grade(90) == "A"
    assert grade(89) == "B"
    assert grade(80) == "B"
    assert grade(79) == "C"
```

Then run `pytest` in the terminal, in that folder:

TERMINAL

```
pytest
```

OUTPUT

```
============ test session starts ============
collected 2 items

test_school.py ..                      [100%]

============= 2 passed in 0.01s =============
```

What should you test?

- **Normal cases:** typical inputs, like `grade(85)`.
- **Boundaries:** the exact values where behaviour changes, like 89, 90 and 80. Bugs love boundaries!
- **Edge cases:** empty lists, zero, negative numbers, very big numbers, empty strings.
- **Bad input:** what *should* happen with `None` or the wrong type?

IN THE REAL WORLD

Many teams use **Test-Driven Development (TDD)**: write a failing test first (**red**), write just enough code to pass it (**green**), then tidy up the code (**refactor**). At most companies, code without tests isn't allowed to be merged.

LESSON 22

Writing clean code

Code is read far more often than it is written: by teammates, by reviewers, and by you six months later. **Clean code** is code that is easy to read and change.

Before and after

PYTHON · hard to read

```python
def f(l):
    t = 0
    for i in l:
        t = t + i
    return t / len(l)
```

PYTHON · clean

```python
def average(numbers):
    """Return the average of a list of numbers (0 for an empty list)."""
    if not numbers:
        return 0
    return sum(numbers) / len(numbers)
```

The clean version has a clear name, a docstring, handles the empty-list edge case (the first version crashes!) and uses a built-in.

The rules that matter most

Rule	Example
Descriptive names	`days_until_exam`, not `d` or `x2`
Functions are verbs, variables are nouns	`calculate_tax()`, `tax_rate`
Constants in CAPITALS	`MAX_LIVES = 3`, `FILENAME = "todo.txt"`
Classes in CapWords	`BankAccount`, `ContactBook`
Small functions with one job	`load_tasks()`, `save_tasks()`, `show_tasks()`
Don't Repeat Yourself (DRY)	Copy-pasted code belongs in a function
Keep It Simple (KISS)	The simplest solution that works is usually the best
Comments explain why, not what	`# retry because the API sometimes times out`

PEP 8: Python's official style guide

- Indent with **4 spaces**
- Use `snake_case` for variables and functions
- Keep lines reasonably short (about 79 to 100 characters)
- Put two blank lines between top-level functions and classes
- Put spaces around operators: `x = a + b`, not `x=a+b`

> **TIP**
> Let tools do the boring work. A **formatter** (like **Ruff** or **Black**) fixes the layout of your code automatically. A **linter** (like **Ruff** or **Pylint**) warns you about likely bugs. In VS Code, install the Ruff extension and turn on *Format on Save*.

LESSON 23

Algorithms and Big-O

An **algorithm** is a step-by-step method for solving a problem: a recipe. Some recipes are much faster than others, and that difference matters when your data gets big.

Searching: linear search

Check every item, one by one, until you find it:

PYTHON
```python
def linear_search(items, target):
    for index, item in enumerate(items):
        if item == target:
            return index
    return -1          # -1 means "not found"

print(linear_search([7, 3, 9, 1], 9))
```

OUTPUT
```
2
```

Searching: binary search (only for sorted lists)

Remember the guessing-game strategy? Look at the **middle**. If the target is bigger, throw away the left half, otherwise the right half. Repeat.

PYTHON

```python
def binary_search(items, target):
    low = 0
    high = len(items) - 1
    while low <= high:
        middle = (low + high) // 2
        if items[middle] == target:
            return middle
        elif items[middle] < target:
            low = middle + 1      # search the right half
        else:
            high = middle - 1     # search the left half
    return -1

numbers = [3, 8, 15, 23, 42, 57, 91]
print(binary_search(numbers, 42))
print(binary_search(numbers, 5))
```

OUTPUT

```
4
-1
```

For a million sorted items, linear search may need **1,000,000** checks. Binary search needs at most **20**. Same answer, wildly different speed.

Big-O notation: how does the work grow?

Big-O describes how the number of steps grows as the input (called **n**) gets bigger:

Big-O	Name	Steps for 1,000 items	Example
O(1)	Constant	1	Getting `items[5]`, a dictionary lookup
O(log n)	Logarithmic	about 10	Binary search
O(n)	Linear	1,000	Linear search, one loop through a list
O(n log n)	Linearithmic	about 10,000	Good sorting (Python's `sorted()`)
O(n²)	Quadratic	1,000,000	A loop inside a loop over the same list

Sorting: bubble sort

Bubble sort is slow, O(n²), but it's easy to understand: go through the list, swap neighbours that are in the wrong order, and repeat. The biggest values "bubble" to the end.

PYTHON

```python
def bubble_sort(items):
    items = items[:]                # work on a copy
    n = len(items)
    for i in range(n):
        for j in range(n - 1 - i):
            if items[j] > items[j + 1]:
                items[j], items[j + 1] = items[j + 1], items[j]   # swap
    return items

print(bubble_sort([5, 2, 9, 1, 7]))
```

OUTPUT

```
[1, 2, 5, 7, 9]
```

GOOD TO KNOW

In real code, always use `sorted()` or `.sort()`. They use an algorithm called **Timsort**, which is O(n log n) and very well tested. You learn bubble sort to understand *how* sorting works, and because it comes up in exams.

Recursion: a function that calls itself

PYTHON

```python
def factorial(n):
    if n <= 1:          # base case: stop here
        return 1
    return n * factorial(n - 1)

print(factorial(5))     # 5 * 4 * 3 * 2 * 1
```

OUTPUT

```
120
```

WATCH OUT

Every recursive function needs a **base case** that stops it. Without one, it calls itself forever and crashes with `RecursionError: maximum recursion depth exceeded`.

LESSON 24

Data structures you will hear about

Structure	Idea	Real-life example	In Python
Array / list	Items in order, found by position	Seats in a row	`list`
Stack	Last In, First Out (LIFO)	A stack of plates; the Undo button	`list` with `append` / `pop`
Queue	First In, First Out (FIFO)	A line at a shop; a printer queue	`collections.deque`
Hash map	Find a value by key, instantly	A phone book	`dict`
Set	Unique items, fast membership checks	A guest list	`set`
Linked list	Each item points to the next one	A treasure hunt with clues	Build it with classes
Tree	Items with parents and children	Folders on your computer; a family tree	Build it with classes
Graph	Items (nodes) connected by links (edges)	Road maps; friends on social media	`dict` of lists

A stack

PYTHON

```python
history = []
history.append("page1")
history.append("page2")
history.append("page3")
print(history.pop())     # the Back button: the last page comes off first
print(history)
```

OUTPUT

```
page3
['page1', 'page2']
```

A queue

PYTHON

```python
from collections import deque

line = deque()
line.append("Ali")
line.append("Mia")
line.append("Sam")
print(line.popleft())    # first in line is served first
print(list(line))
```

OUTPUT

```
Ali
['Mia', 'Sam']
```

A graph

PYTHON

```python
friends = {
    "Sara": ["Omar", "Lina"],
    "Omar": ["Sara"],
    "Lina": ["Sara", "Mia"],
    "Mia": ["Lina"],
}
print("Lina's friends:", friends["Lina"])
```

OUTPUT

```
Lina's friends: ['Sara', 'Mia']
```

TIP

To practise algorithms and data structures, try **Exercism**, **Codewars**, **HackerRank** and later **LeetCode**. Start with "easy" problems. Doing one problem a day adds up fast.

PROJECT 3

Checkpoint project: contact book

This project uses **classes**, **lists**, **JSON files**, **comprehensions** and **error handling**. It is a small but properly designed program: data in one class, storage in another, and the user interface in `main()`.

PYTHON · contacts.py

```
import json

FILENAME = "contacts.json"


class Contact:
    def __init__(self, name, phone, email):
        self.name = name
        self.phone = phone
        self.email = email

    def to_dict(self):
        return {"name": self.name, "phone": self.phone, "email": self.email}

    def __str__(self):
        return f"{self.name:<15} {self.phone:<15} {self.email}"


class ContactBook:
    def __init__(self, filename):
        self.filename = filename
        self.contacts = []
        self.load()

    def load(self):
        try:
            with open(self.filename) as file:
                for item in json.load(file):
                    self.contacts.append(
                        Contact(item["name"], item["phone"], item["email"])
                    )
        except FileNotFoundError:
            pass

    def save(self):
        with open(self.filename, "w") as file:
            json.dump([c.to_dict() for c in self.contacts], file, indent=2)

    def add(self, contact):
        self.contacts.append(contact)
        self.save()

    def search(self, text):
        text = text.lower()
        return [c for c in self.contacts if text in c.name.lower()]

    def delete(self, name):
        before = len(self.contacts)
        self.contacts = [c for c in self.contacts
```

```
                        if c.name.lower() != name.lower()]
        self.save()
        return len(self.contacts) < before


def main():
    book = ContactBook(FILENAME)
    while True:
        print("\n1) List  2) Add  3) Search  4) Delete  5) Quit")
        choice = input("Choose: ").strip()
        if choice == "1":
            if not book.contacts:
                print("No contacts yet.")
            for contact in book.contacts:
                print(contact)
        elif choice == "2":
            name = input("Name: ").strip()
            phone = input("Phone: ").strip()
            email = input("Email: ").strip()
            if name:
                book.add(Contact(name, phone, email))
                print("Saved!")
            else:
                print("Name cannot be empty.")
        elif choice == "3":
            results = book.search(input("Search for: "))
            if not results:
                print("No matches.")
            for contact in results:
                print(contact)
        elif choice == "4":
            if book.delete(input("Name to delete: ")):
                print("Deleted.")
            else:
                print("Not found.")
        elif choice == "5":
            print("Goodbye!")
            break
        else:
            print("Please choose 1-5.")


if __name__ == "__main__":
    main()
```

CHALLENGES

Level it up:

1. Check that the email contains `@` and the phone contains only digits, spaces or `+`.
2. Add **edit**: change the phone or email of an existing contact.
3. Write **pytest tests** for `ContactBook.search` and `ContactBook.delete` (use a test file name like `test_contacts.json`).
4. Add a **birthday** field and an option to list birthdays in the next 30 days.

CHECKPOINT

Quiz 3 and Stage 3 checklist

1. What is the difference between a class and an object?
2. What does `self` refer to inside a method?
3. Name the four pillars of OOP.
4. Rewrite with a list comprehension: a list of the squares of the even numbers from 0 to 10.
5. What is a breakpoint?
6. What does `assert add(2, 2) == 4` do if `add` is broken?
7. What is the Big-O of binary search, and what must be true about the list?
8. Which data structure is Last In, First Out?
9. What must every recursive function have?
10. Why are boundary values (like 89 and 90 in a grading function) important to test?

Can you do these on your own?

- ✓ Write a class with `__init__`, attributes, methods and `__str__`
- ✓ Use inheritance and `super()`
- ✓ Read and write list comprehensions
- ✓ Find a bug with prints and with a debugger
- ✓ Write tests with `assert` and pytest
- ✓ Explain O(1), O(n) and O(n^2) with examples
- ✓ Implement binary search without looking
- ✓ Build the contact book with at least one challenge feature

KEY IDEA

If you finished Parts 2 to 4 and the three projects, you are **no longer a beginner**. You can now learn any area of programming. Parts 5 to 7 show you how real software (websites, apps and servers) is built.

PART 5

Developer essentials

The tools and ideas every developer uses, whatever language they write: the terminal, Git, the internet, APIs, databases, security and teamwork.

In this part you will learn

- ✓ How to use the terminal (command line) confidently
- ✓ How to save your code's history with Git and share it on GitHub
- ✓ What really happens when you open a website
- ✓ How programs talk to each other with APIs and JSON
- ✓ How databases and SQL work
- ✓ The security basics every developer must know
- ✓ How software is built by teams, and what "the cloud" really is

LESSON 25

The terminal (command line)

The **terminal** is a window where you control the computer by **typing commands** instead of clicking. It looks scary at first, but you only need about 10 commands, and developers use it every single day.

- **Windows:** *Terminal* or *PowerShell* (search for it in the Start menu)
- **Mac:** *Terminal* (Cmd + Space, type Terminal)
- **Linux:** *Terminal* (often Ctrl + Alt + T)
- **VS Code:** a terminal is built in: **View > Terminal**, or Ctrl + ` (the backtick key)

The commands you actually need

What you want	Mac / Linux	Windows (PowerShell)
Where am I?	`pwd`	`pwd`
List files here	`ls`	`ls` or `dir`
Go into a folder	`cd projects`	`cd projects`
Go up one folder	`cd ..`	`cd ..`
Make a folder	`mkdir website`	`mkdir website`
Make an empty file	`touch notes.txt`	`ni notes.txt`
Delete a file (careful: no recycle bin!)	`rm notes.txt`	`del notes.txt`
Clear the screen	`clear`	`cls` or `clear`
Run a Python file	`python3 game.py`	`python game.py`
Open this folder in VS Code	`code .`	`code .`

A typical session

TERMINAL

```
cd Documents
mkdir coding
cd coding
mkdir my-first-project
cd my-first-project
code .
```

Now you have a new project folder, and VS Code is open in it. This is how developers start every new project.

TIP

Three shortcuts that save hours: **Tab** auto-completes file and folder names. The **Up arrow** brings back your previous commands. **Ctrl + C** stops a running program.

GOOD TO KNOW

A `.` means "this folder" and `..` means "the folder above". On a Mac, if `code .` doesn't work, open VS Code, press Cmd + Shift + P, and run *Shell Command: Install 'code' command in PATH*.

LESSON 26

Git and GitHub

Git is a tool that saves **snapshots** of your project over time, like save points in a video game. If you break something, you can go back. **GitHub** is a website that stores your Git projects online, so you can back them up, show them to employers and work with other people.

	Git	GitHub
What	A program on your computer	A website (github.com)
Does	Tracks changes and saves the history of your code	Hosts your code online, plus teamwork tools
Alternatives	(Git is the standard)	GitLab, Bitbucket

Install and set up (once)

1. Install Git from **git-scm.com**. On a Mac, typing `git --version` in the terminal will offer to install it.
2. Tell Git who you are. It writes this on every save point:

TERMINAL

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

The everyday workflow

HOW GIT WORKS

```
        git add                git commit                git push
Your files  ──────►  Staging area  ──────►  Local history  ──────►  GitHub
(working folder)     (ready to save)        (save points)            (online)
```

TERMINAL

```
git init                          # start tracking this folder (once per project)
git status                        # what has changed?
git add .                         # stage all changes
git commit -m "Add guessing game" # save a snapshot with a message
git log --oneline                 # see the history
```

Put your project on GitHub

1. Create a free account at **github.com**.
2. Click **New** (the green button) to create a repository (a *repo* is a project). Give it a name, and don't add any files.
3. GitHub shows you commands like these. Run them in your project folder:

TERMINAL

```
git remote add origin https://github.com/YOUR-USERNAME/my-project.git
git branch -M main
git push -u origin main
```

Refresh the GitHub page and your code is there! After that, every time you make changes, it's just:

TERMINAL

```
git add .
git commit -m "Describe what you changed"
git push
```

GOOD TO KNOW

When you push for the first time, Git asks you to sign in. The easiest way is through the browser window that pops up. If you prefer clicking to typing, **GitHub Desktop** (desktop.github.com) and the **Source Control** panel in VS Code do all of this with buttons.

Branches: work on new features safely

A **branch** is a separate line of work. You build a new feature on a branch, and when it works, you **merge** it into `main`.

TERMINAL

```
git switch -c dark-mode      # create a new branch and move to it
# ... edit files, then add and commit as usual ...
git switch main              # go back to main
git merge dark-mode          # bring the finished feature in
```

Other commands you will use

Command	What it does
`git clone URL`	Download a copy of a repository from GitHub
`git pull`	Get the latest changes from GitHub
`git diff`	Show exactly what changed since the last commit
`git restore file.py`	Throw away your changes to a file (careful!)
`git branch`	List branches

Pull requests: how teams work

In a team, you don't push straight to `main`. You push your branch and open a **pull request (PR)** on GitHub. A PR says: "here are my changes, please review them". Teammates comment, you fix things, automated tests run, and then the PR is **merged**. This is called **code review**, and it's how almost every software company works.

WATCH OUT

Never commit passwords, API keys or secret tokens. Once they are on GitHub, assume someone has seen them. Put secret files in a file called `.gitignore` so Git ignores them.

> **TIP**
> Write commit messages that say **what and why**: *"Fix crash when task list is empty"* is great. *"stuff"* or *"update"* is not.

LESSON 27

How the internet and websites work

When you type a website address and press Enter, a lot happens in less than a second:

1. Your browser (the **client**) asks a **DNS** server: "What is the address of example.com?" DNS is the internet's phone book. It turns names into **IP addresses** like `93.184.215.14`.
2. Your browser connects to the computer at that address: a **server**. A server is just a computer that's always on, waiting for requests.
3. Your browser sends an **HTTP request**: "GET me the page `/`, please."
4. The server sends back a **response**: a status code (like `200 OK`) plus the page's **HTML**.
5. The browser reads the HTML, then asks for the **CSS**, **JavaScript** and **images** it mentions.
6. The browser draws (**renders**) the page, and runs the JavaScript to make it interactive.

The parts of a URL

URL
```
https://www.example.com/products/shoes?color=red#reviews
```

Part	Example	Meaning
Protocol	`https://`	How to talk. The **s** means the connection is encrypted (secure)
Domain	`www.example.com`	The website's name, which DNS turns into an IP address
Path	`/products/shoes`	Which page on the site
Query string	`?color=red`	Extra information sent to the server, like search filters
Fragment	`#reviews`	Jump to a section of the page (never sent to the server)

HTTP methods and status codes

Method	Meaning	Example
`GET`	Read something	Load a page, get a list of posts
`POST`	Create something	Submit a form, publish a post
`PUT` / `PATCH`	Update something	Edit your profile
`DELETE`	Delete something	Remove a comment

Code	Meaning
200 OK	Everything worked
201 Created	Something new was created
301 / **302**	The page has moved: go to another address (a redirect)
400 Bad Request	The request was wrong (for example, missing information)
401 / **403**	You are not logged in / you are not allowed
404 Not Found	The page doesn't exist (the famous one!)
500 Server Error	The server's code crashed. A bug on *their* side

KEY IDEA

Remember the pattern: 2xx = success, 3xx = go somewhere else, 4xx = **you** (the client) made a mistake, 5xx = **the server** made a mistake.

Frontend, backend and database

Layer	Runs on	Job	Built with
Frontend	The user's browser or phone	Everything the user sees and clicks	HTML, CSS, JavaScript, React, Swift, Kotlin
Backend	A server	Business logic: logins, payments, rules, sending emails	Python, JavaScript (Node.js), Java, C#, Go, PHP, Ruby
Database	A server	Stores data permanently	PostgreSQL, MySQL, SQLite, MongoDB

Full-stack developers work on all three layers.

LESSON 28

APIs and JSON

An **API** (Application Programming Interface) is a way for **programs to talk to other programs**. Think of a restaurant: you (your program) don't go into the kitchen (the other system). You give your order to the **waiter** (the API), who brings back your food (the data).

Weather apps use a weather API. Apps with a "Sign in with Google" button use Google's API. AI apps use AI model APIs. Most web APIs send data back as **JSON**:

JSON

```
{
  "city": "Cairo",
  "temperature": 31,
  "sunny": true,
  "forecast": ["sunny", "sunny", "cloudy"]
}
```

JSON looks almost exactly like Python dictionaries and lists. The differences: JSON uses `true` / `false` / `null` instead of `True` / `False` / `None`, and always uses double quotes.

Calling a real API from Python

Install the `requests` package (`python -m pip install requests`), then:

PYTHON

```python
import requests

response = requests.get("https://api.github.com/users/octocat")
print(response.status_code)

data = response.json()       # turn the JSON into a Python dict
print(data["name"])
print(data["public_repos"], "public repositories")
```

OUTPUT (the numbers may change over time)

```
200
The Octocat
8 public repositories
```

Things to know about APIs

- **API keys:** many APIs need a secret key to identify you. Keep it secret. Never put it in code that you upload, or in a website's JavaScript.
- **Rate limits:** most APIs limit how many requests you can make per minute or per day.
- **Documentation:** every good API has docs that explain its URLs (called **endpoints**) and what they return. Reading API docs is a core developer skill.
- **Free APIs to practise with:** search for "public APIs list" on GitHub. There are free APIs for weather, Pokémon, jokes, space photos and more.

TIP

You can test APIs without writing code with tools like **Postman** or **Bruno**, or by simply opening a GET URL in your browser.

LESSON 29

Databases and SQL

Files are fine for small things. But real apps with thousands of users need a **database**: software that stores huge amounts of data safely and finds things in it fast. Most databases store data in **tables**, like spreadsheets: each **row** is one record, and each **column** is one field.

id	name	age	score
1	Sara	16	91
2	Omar	17	78
3	Lina	16	85

SQL: the language of databases

SQL (Structured Query Language, said "sequel" or "S-Q-L") is how you talk to most databases. It reads almost like English:

```sql
-- create a table
CREATE TABLE students (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER,
    score INTEGER
);

-- add rows
INSERT INTO students (name, age, score) VALUES ('Sara', 16, 91);
INSERT INTO students (name, age, score) VALUES ('Omar', 17, 78);

-- read rows
SELECT * FROM students;
SELECT name, score FROM students WHERE score > 80 ORDER BY score DESC;

-- change rows
UPDATE students SET score = 82 WHERE name = 'Omar';

-- delete rows
DELETE FROM students WHERE id = 1;
```

Almost everything apps do with data is one of four actions, known as **CRUD**:

CRUD	SQL	HTTP (from Lesson 27)	Example
Create	`INSERT`	`POST`	Sign up, write a post
Read	`SELECT`	`GET`	View your feed
Update	`UPDATE`	`PUT` / `PATCH`	Edit your profile
Delete	`DELETE`	`DELETE`	Delete a message

Using a database from Python

Python comes with **SQLite**, a small but real database stored in one file. No installation needed:

PYTHON

```python
import sqlite3

connection = sqlite3.connect(":memory:")   # use "school.db" to save to a file
cursor = connection.cursor()

cursor.execute("CREATE TABLE students (name TEXT, score INTEGER)")
cursor.executemany(
    "INSERT INTO students VALUES (?, ?)",
    [("Sara", 91), ("Omar", 78), ("Lina", 85)],
)

query = "SELECT name, score FROM students WHERE score > 80 ORDER BY score DESC"
for row in cursor.execute(query):
    print(row)

connection.close()
```

OUTPUT

```
('Sara', 91)
('Lina', 85)
```

WATCH OUT

Always use `?` placeholders for values, like above. **Never** build SQL by gluing in user input with f-strings or `+`. Attackers can type SQL code into your form and steal or delete your data. This attack is called **SQL injection**, and it's one of the most common security holes on the web.

Connecting tables: JOIN

Real databases have many tables linked by IDs. A **JOIN** combines them:

SQL

```sql
SELECT students.name, classes.title
FROM students
JOIN classes ON students.class_id = classes.id;
```

Types of databases

Database	Type	Good for
SQLite	SQL, in a single file	Learning, small apps, phone apps
PostgreSQL	SQL server	Most professional web apps. Powerful and free
MySQL / **MariaDB**	SQL server	Very common, especially with PHP and WordPress
MongoDB	NoSQL (stores JSON-like documents)	Flexible data that doesn't fit neat tables
Redis	Key-value, kept in memory	Super-fast caching, sessions, leaderboards
Firebase / **Supabase**	Hosted backend with a database	Mobile and web apps without writing your own server

> **TIP**
> To look inside an SQLite file with a friendly interface, install **DB Browser for SQLite** (sqlitebrowser.org). It's free.

LESSON 30

Security basics every developer must know

Security is not just for specialists. Most hacks happen because of simple mistakes by ordinary developers. Learn these habits from the start:

Rule	Why
Never trust user input	Check and clean everything users type. Treat it as possibly dangerous.
Use parameterised queries (`?` placeholders)	Prevents SQL injection (Lesson 29).
Escape output in web pages	Prevents **XSS** (cross-site scripting): attackers putting JavaScript into your pages through comments or names.
Never store plain-text passwords	Store a *hash* made by a password library (like bcrypt or Argon2). If the database leaks, the passwords stay safe.
Keep secrets out of code	Put API keys and passwords in *environment variables* or secret managers, and add secret files to `.gitignore`.
Always use HTTPS	Encrypts data between the user and the server. Free with modern hosting.
Keep dependencies updated	Old library versions often have known security holes. GitHub's Dependabot can warn you.
Give the minimum access needed	A program that only reads data shouldn't have permission to delete it.

GOOD TO KNOW

The **OWASP Top 10** (owasp.org) is a famous list of the ten most important web security risks. Read it once you start building websites with logins.

WATCH OUT

Learn hacking skills only on systems you own or have permission to test, like the practice labs on **TryHackMe**, **Hack The Box** or **PicoCTF**. Attacking real systems without permission is illegal.

LESSON 31

How software is built in teams

Professional software is rarely built by one person. Here is how a typical team works.

The software development life cycle

1. **Plan:** what problem are we solving, and for whom?
2. **Design:** sketch screens (often in **Figma**) and plan the code's structure.
3. **Build:** write the code, in small pieces.
4. **Test:** automated tests plus manual checks.
5. **Release (deploy):** put it live for users.
6. **Maintain:** fix bugs, improve, listen to feedback, and repeat.

Agile and Scrum

Most teams work in an **agile** way: they build in small steps and adjust often, instead of planning everything for a year. A popular style is **Scrum**:

- Work is split into **sprints**: 1 to 2 week cycles.
- Tasks are written as **tickets** in tools like Jira, Linear or GitHub Issues.
- A short daily **stand-up** meeting: what I did, what I'll do next, what's blocking me.
- At the end of each sprint: a demo, and a **retrospective** on how to work better.

CI/CD: robots that check and ship code

Continuous Integration (CI): every time someone pushes code, a server automatically runs all the tests. If anything fails, the team knows immediately. **Continuous Deployment (CD):** when tests pass, the new version goes live automatically. **GitHub Actions** is a popular free tool for this.

Common jobs in a software team

Role	What they do
Software engineer / developer	Designs, writes, tests and maintains code
Product manager	Decides *what* to build and why, based on users and business goals
Designer (UI/UX)	Designs how the product looks and feels
QA engineer	Tests the product to find bugs before users do
DevOps / platform engineer	Builds the servers, pipelines and tools that keep everything running
Engineering manager	Leads and supports the developers

LESSON 32

The cloud, servers and Docker

"**The cloud**" simply means **other people's computers that you rent over the internet**. Instead of buying servers, companies rent computing power, storage and databases, and pay for what they use.

Provider	Notes
AWS (Amazon Web Services)	The biggest cloud provider
Microsoft Azure	Popular with large companies and .NET developers
Google Cloud	Strong in data and AI
Vercel, Netlify, Render, Railway, Fly.io	Simpler platforms that are great for putting your first projects online

Docker and containers

Ever heard "but it works on my computer!"? **Docker** fixes that. It packs your app, plus everything it needs (the right Python version, libraries and settings), into a **container** that runs the same way on any computer or server. A recipe file called a `Dockerfile` describes the container:

DOCKERFILE

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

GOOD TO KNOW

You don't need the cloud or Docker as a beginner. But you will hear these words constantly, and they become important once you build backends or work in a team.

PART 6

Building websites

HTML for structure, CSS for style, JavaScript for behaviour. Then you'll build a real portfolio website and put it online for free.

In this part you will learn

- ✓ What a website needs, and the difference between static and dynamic sites
- ✓ HTML: headings, text, links, images, lists, tables and forms
- ✓ CSS: colours, fonts, the box model, Flexbox, Grid and responsive design
- ✓ JavaScript: variables, functions, events and changing the page
- ✓ How to build a complete personal portfolio website
- ✓ How to publish your website online for free
- ✓ How backends make websites dynamic (Python Flask and Node.js)
- ✓ What to learn next: React, TypeScript and full-stack development

LESSON 33

What does a website need?

Every website you have ever visited is built from **three languages** working together:

Language	Job	House comparison
HTML	The **structure** and content: headings, paragraphs, images, buttons	The walls, rooms and doors
CSS	The **style**: colours, fonts, spacing, layout, animations	The paint, furniture and decoration
JavaScript	The **behaviour**: what happens when you click, type or scroll	The electricity: lights, doorbell, alarm

GOOD TO KNOW

HTML and CSS are not really *programming* languages: they have no loops or decisions. HTML is a **markup** language and CSS is a **style sheet** language. JavaScript is the real programming language of the web.

What you need to build a website

Need	Tool	Cost
Somewhere to write code	**VS Code** (plus the free **Live Server** extension, which refreshes the page every time you save)	Free
Something to view it	A **web browser**: Chrome, Firefox, Edge or Safari	Free
To put it online	**Hosting**: a server that sends your files to visitors (GitHub Pages, Netlify, Vercel, Cloudflare Pages)	Free for simple sites
Your own address (optional)	A **domain name** like `sara.dev`, bought from a registrar	About $10–20 a year

Static vs dynamic websites

	Static website	Dynamic website / web app
What	The same files are sent to every visitor	Pages are built for each user, with data from a database
Examples	Portfolios, restaurant menus, landing pages, documentation	Instagram, YouTube, online shops, school portals
Needs	HTML, CSS, JavaScript	All of that, **plus** a backend language, a database and a server
Logins and accounts?	No	Yes
Hosting	Free and simple	Free tiers exist; costs grow with users

You'll build a static site first, which is the right place to start. Lesson 39 shows how backends add the dynamic part.

No-code website builders

Tools like **Wix**, **Squarespace**, **WordPress** and **Shopify** let people build websites without code. They're great for businesses that just need a site quickly. Learning to code gives you **full control**, lets you build things these tools can't, and is the skill that companies hire for.

How a website project is organised

FOLDER

```
my-website/
├── index.html      ← the home page (always called index.html)
├── style.css       ← the styles
├── script.js       ← the JavaScript
└── images/
    └── photo.jpg
```

LESSON 34

HTML: the structure

Create a folder called `my-website`, open it in VS Code, create a file called `index.html`, and type this:

```HTML
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Website</title>
</head>
<body>
    <h1>Hello, world!</h1>
    <p>This is my very first web page.</p>
</body>
</html>
```

Save it, then **double-click the file** to open it in your browser. Or, if you installed **Live Server**, right-click the file in VS Code and choose **Open with Live Server**.

Part	What it does
`<!DOCTYPE html>`	Tells the browser this is a modern HTML page
`<html lang="en">`	Wraps the whole page, and says the language is English
`<head>`	Information **about** the page, which isn't shown on it: title, character set, links to CSS
`<meta name="viewport" ...>`	Makes the page display properly on phones
`<title>`	The text on the browser tab
`<body>`	Everything you **see** on the page

Anatomy of an HTML element

```HTML
<a href="https://wikipedia.org" class="link">Visit Wikipedia</a>
```

- `<a ...>` is the **opening tag**, and `</a>` is the **closing tag** (note the `/`).
- `href="..."` and `class="..."` are **attributes**: extra settings for the element.
- `Visit Wikipedia` is the **content**: what the user sees.
- Some elements have no content and no closing tag, such as `<img>`, `<br>` and `<input>`.

The tags you'll use most

Tag	What it's for
`<h1>` to `<h6>`	Headings, from the most important (`h1`, use one per page) to the least
`<p>`	A paragraph of text
`<a href="...">`	A link to another page or site
`<img src="..." alt="...">`	An image. `alt` describes it for blind users and search engines
`<ul>`, `<ol>`, `<li>`	Bullet lists, numbered lists, and the items in them
`<strong>`, `<em>`	Important (bold) and emphasised (italic) text
`<button>`	A clickable button
`<form>`, `<input>`, `<label>`, `<textarea>`	Forms that users type into
`<table>`, `<tr>`, `<th>`, `<td>`	Tables: rows, header cells, data cells
`<div>`, `<span>`	Boxes with no meaning, used for grouping and styling
`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`	**Semantic** boxes that describe what each part of the page *is*

Text, links and images

HTML

```
<h1>My Favourite Animals</h1>
<p>Cats are <strong>amazing</strong>. They sleep about <em>15 hours</em> a day.</p>

<a href="https://en.wikipedia.org/wiki/Cat">Read more about cats</a>

<img src="images/cat.jpg" alt="An orange cat sleeping in the sun" width="300">
```

Lists

HTML

```
<h2>My skills</h2>
<ul>
    <li>Python</li>
    <li>HTML</li>
    <li>CSS</li>
</ul>

<h2>Morning routine</h2>
<ol>
    <li>Wake up</li>
    <li>Breakfast</li>
    <li>Code for 30 minutes</li>
</ol>
```

Forms

HTML

```
<form>
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" placeholder="you@example.com" required>

    <label for="level">Your level:</label>
    <select id="level" name="level">
        <option>Beginner</option>
        <option>Intermediate</option>
    </select>

    <button type="submit">Subscribe</button>
</form>
```

`type="email"` makes the browser check that it looks like an email address, and `required` stops the form being sent while the field is empty. Other useful types are `text`, `password`, `number`, `date`, `checkbox` and `radio`.

Semantic page layout

HTML

```
<body>
    <header>
        <nav>
            <a href="#about">About</a>
            <a href="#contact">Contact</a>
        </nav>
    </header>

    <main>
        <section id="about">
            <h2>About me</h2>
            <p>...</p>
        </section>
    </main>

    <footer>
        <p>&copy; 2026 Sara</p>
    </footer>
</body>
```

A link to `#about` jumps to the element with `id="about"`. `©` is an **HTML entity** that shows the © symbol.

TIP

Right-click any website and choose Inspect (or press F12). This opens the browser's **DevTools**, where you can see and even live-edit the HTML and CSS of any site. It's the best way to learn how websites are built.

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Make an "About me" page with a heading, two paragraphs, a photo and a list of your hobbies.
2. Add a table of your weekly timetable (days as columns).
3. Create a second page, `games.html`, and link the two pages to each other.
4. Build a sign-up form with name, email, password, a date of birth field and a checkbox for "I agree to the rules".

LESSON 35

CSS: the style

CSS decides how HTML looks. Put your CSS in a separate file, `style.css`, and connect it inside the `<head>` of your HTML:

```html
<link rel="stylesheet" href="style.css">
```

Anatomy of a CSS rule

```css
h1 {
    color: navy;
    font-size: 40px;
    text-align: center;
}
```

`h1` is the **selector** (which elements to style). Inside the curly brackets are **declarations**: a **property**, a colon, a **value**, and a semicolon.

Selectors: choosing what to style

Selector	Selects	HTML it matches
`p`	Every `<p>` element	`<p>`
`.card`	Every element with the **class** `card` (a class can be used many times)	`<div class="card">`
`#header`	The one element with the **id** `header` (an id is unique)	`<header id="header">`
`nav a`	Every `<a>` that is **inside** a `<nav>`	
`a:hover`	Links while the mouse is over them	
`h1, h2`	Both `h1` and `h2` elements	

Colours and fonts

CSS

```css
body {
    font-family: "Segoe UI", Arial, sans-serif;  /* first one that exists wins */
    color: #1f2937;                              /* text colour (hex code) */
    background-color: rgb(249, 250, 251);        /* the same idea, as red, green, blue */
    line-height: 1.6;                            /* space between lines */
}

.highlight {
    color: tomato;                               /* 140+ colour names exist */
    font-weight: bold;
}
```

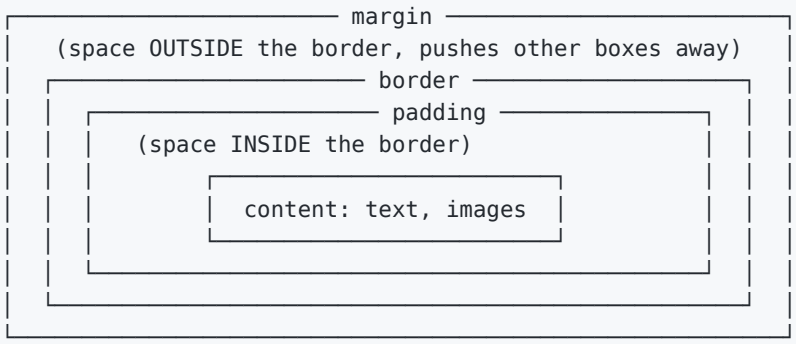
TIP

Free fonts: **Google Fonts** (fonts.google.com). Pick a font, copy the `<link>` code into your `<head>`, and use its name in `font-family`. Colour palettes: **coolors.co**.

The box model: the most important idea in CSS

Every element is a **rectangle** made of four layers:

THE BOX MODEL



CSS

```css
.card {
    padding: 20px;              /* space inside */
    border: 2px solid #e5e7eb;  /* the edge */
    margin: 16px;               /* space outside */
    border-radius: 12px;        /* rounded corners */
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}
```

TIP

Start every stylesheet with `* { box-sizing: border-box; }`. It makes widths include the padding and border, which is much easier to reason about.

Units

Unit	Meaning	Use it for
`px`	Pixels: a fixed size	Borders, small details
`rem`	Relative to the root font size (usually 1rem = 16px)	Font sizes, spacing
`%`	A percentage of the parent element	Widths
`vw` / `vh`	1% of the screen's width / height	Full-screen sections
`fr`	A fraction of the free space (only in Grid)	Grid columns

Layout with Flexbox (one direction)

Flexbox arranges items in a **row** or a **column**. It's perfect for navigation bars, buttons in a row and centring things.

```css
.navbar {
    display: flex;
    justify-content: space-between;  /* spread out along the row */
    align-items: center;             /* centre vertically */
    gap: 20px;                       /* space between items */
}

.center-everything {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
}
```

Layout with Grid (rows and columns)

```css
.gallery {
    display: grid;
    grid-template-columns: repeat(3, 1fr);   /* 3 equal columns */
    gap: 16px;
}
```

Responsive design: looking good on phones

More than half of web traffic comes from phones, so your site must adapt to small screens. A **media query** applies CSS only when the screen fits a condition:

```css
@media (max-width: 700px) {
    .gallery {
        grid-template-columns: 1fr;   /* 1 column on small screens */
    }
}
```

TIP

In DevTools (F12), click the **phone and tablet icon** to preview your site at different screen sizes.

Hover effects and animation

CSS

```css
.button {
    background: #2563eb;
    color: white;
    padding: 12px 24px;
    border-radius: 8px;
    transition: transform 0.2s, background 0.2s;   /* animate changes */
}

.button:hover {
    background: #1d4ed8;
    transform: scale(1.05);
}
```

GOOD TO KNOW

Once you're comfortable with CSS, you might try a CSS framework. **Tailwind CSS** (very popular now) and **Bootstrap** give you ready-made styles. Learn plain CSS first, though, so you understand what they do.

TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Style your About page: pick a font from Google Fonts, a background colour and a text colour.
2. Make a "card" with padding, rounded corners and a shadow, that lifts up when you hover over it.
3. Build a navigation bar with Flexbox: logo on the left, links on the right.
4. Make a gallery with 3 columns on a computer and 1 column on a phone.

LESSON 36

JavaScript: the behaviour

JavaScript makes pages **interactive**. Connect a file called `script.js` at the end of the `<body>`, just before `</body>`:

HTML

```html
<script src="script.js"></script>
```

Your first JavaScript, in the console

Open DevTools (F12) and click the **Console** tab. You can type JavaScript there and see the results instantly. `console.log()` is JavaScript's `print()`:

JAVASCRIPT

```javascript
console.log("Hello from JavaScript!");
console.log(2 + 3);
```

Variables: const and let

JAVASCRIPT

```javascript
const name = "Sara";   // const: cannot be reassigned
let score = 0;         // let: can change
score = score + 10;
console.log(`${name} has ${score} points`);   // a template string, like an f-string
```

OUTPUT

```
Sara has 10 points
```

TIP

Use `const` by default, and `let` only when the value must change. You will see `var` in old code. Don't use it in new code.

Python vs JavaScript: a translation table

You already know the ideas, so this is mostly new spelling:

Python	JavaScript
`print("hi")`	`console.log("hi");`
`name = "Sara"`	`const name = "Sara";` or `let name = "Sara";`
`f"Hi {name}"`	`` `Hi ${name}` `` (backtick quotes)
`True`, `False`, `None`	`true`, `false`, `null` (and `undefined`)
`and`, `or`, `not`	`&&`, `||`, `!`
`==`	`===` (always use three equals signs)
`if x > 5:` with indentation	`if (x > 5) { ... }` with curly brackets
`elif`	`else if`
`for item in items:`	`for (const item of items) { ... }`
`for i in range(5):`	`for (let i = 0; i < 5; i++) { ... }`
`def add(a, b):`	`function add(a, b) { ... }` or `const add = (a, b) => a + b;`
`len(items)`	`items.length`
`items.append(x)`	`items.push(x)`
list `[1, 2]`	array `[1, 2]`
dict `{"name": "Sara"}`	object `{ name: "Sara" }`
`# comment`	`// comment`

WATCH OUT

In JavaScript, `==` tries to convert types first, so `"5" == 5` is `true`! Always use `===`, which checks the type too. `"5" === 5` is `false`.

Decisions, loops and functions

JAVASCRIPT

```javascript
const age = 16;
if (age >= 18) {
    console.log("Adult");
} else {
    console.log("Not an adult yet");
}

const fruits = ["apple", "banana", "cherry"];
for (const fruit of fruits) {
    console.log(fruit);
}

function add(a, b) {
    return a + b;
}
const multiply = (a, b) => a * b;   // an "arrow function"
console.log(add(2, 3), multiply(2, 3));
```

OUTPUT

```
Not an adult yet
apple
banana
cherry
5 6
```

Objects and arrays

JAVASCRIPT

```javascript
const student = { name: "Sara", age: 16, skills: ["HTML", "CSS"] };
console.log(student.name);
student.skills.push("JavaScript");
console.log(student.skills.length);

const scores = [72, 88, 95];
const doubled = scores.map((s) => s * 2);          // like a list comprehension
const high = scores.filter((s) => s > 80);
console.log(doubled, high);
```

OUTPUT

```
Sara
3
[ 144, 176, 190 ] [ 88, 95 ]
```

The DOM: changing the page with JavaScript

The browser turns your HTML into a tree of objects called the **DOM** (Document Object Model). JavaScript can find elements in it, change them, and react to **events** like clicks and key presses.

HTML

```html
<h1 id="count">0</h1>
<button id="add-btn">Click me!</button>
<script src="script.js"></script>
```

JAVASCRIPT

```javascript
const countText = document.querySelector("#count");   // find elements
const button = document.querySelector("#add-btn");
let count = 0;

button.addEventListener("click", () => {   // run this on every click
    count++;
    countText.textContent = count;           // change the page
});
```

That's the core pattern of all interactive websites: **find an element, listen for an event, change something**.

Code	What it does
`document.querySelector(".card")`	Find the first element matching a CSS selector
`document.querySelectorAll("li")`	Find all matching elements
`el.textContent = "Hi"`	Change the text
`el.style.color = "red"`	Change a style
`el.classList.add("dark")` / `.remove` / `.toggle`	Add, remove or switch a CSS class (the best way to change styles)
`input.value`	What the user typed in an input box
`document.createElement("li")`, `list.append(li)`	Create a new element and add it to the page
`el.addEventListener("click", fn)`	Run `fn` when the event happens (also `"input"`, `"submit"`, `"keydown"`)

Getting data from an API with fetch

JAVASCRIPT

```javascript
async function loadUser() {
    const response = await fetch("https://api.github.com/users/octocat");
    const data = await response.json();
    document.querySelector("#name").textContent = data.name;
}

loadUser();
```

`async` and `await` let JavaScript **wait** for slow things, like network requests, without freezing the page.

> **GOOD TO KNOW**
> JavaScript can also run **outside** the browser, using **Node.js** (nodejs.org). With Node, you can run `node script.js` in the terminal, just like `python script.py`. Node is how JavaScript is used to build servers (Lesson 39).

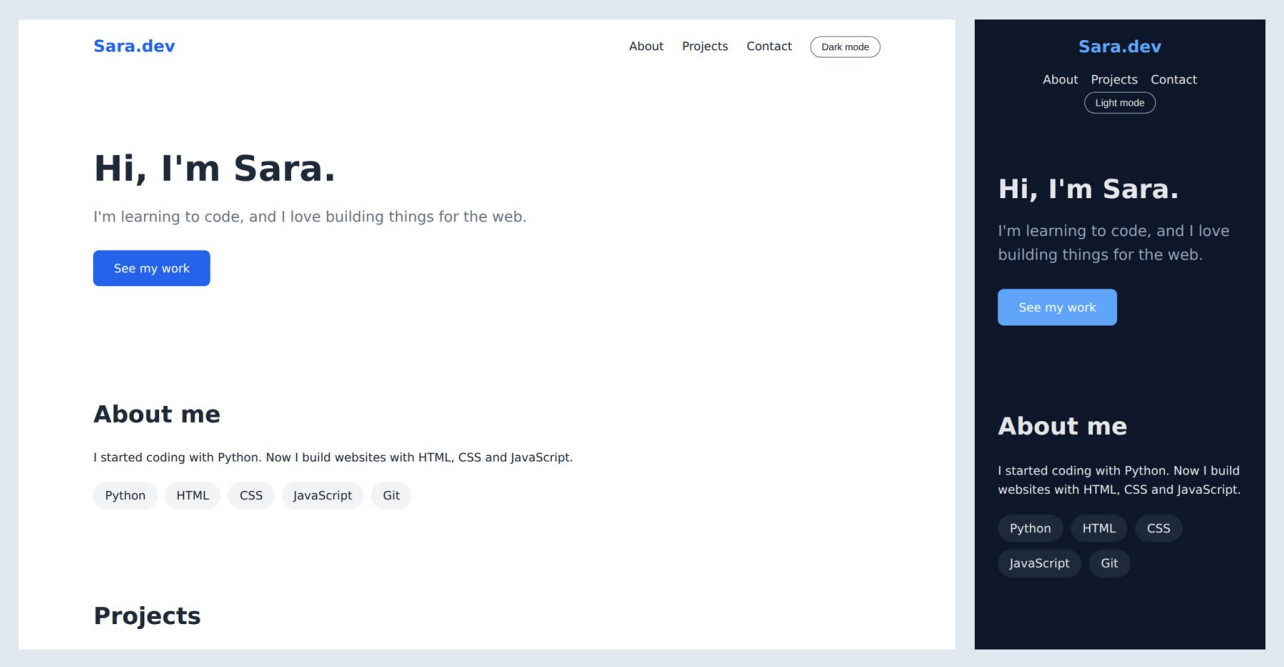
TRY IT YOURSELF (ANSWERS IN APPENDIX A)

1. Make a page with an input box and a button. When clicked, show `Hello, <name>!` on the page.
2. Make a button that changes the page's background to a random colour each time it's clicked.
3. Build a browser to-do list: type a task, click Add, and it appears in a `<ul>`. Bonus: click a task to cross it out.
4. Build a simple calculator with two inputs and buttons for +, −, × and ÷.

LESSON 37 · PROJECT 4

Project: your portfolio website

Now you'll build a real personal website that you can show to anyone: a teacher, a friend, or a future employer. It has a navigation bar, an introduction, an about section, project cards, a contact form, **dark mode** that remembers your choice, and a **responsive** layout that works on phones.



The finished site: light mode on a computer (left) and dark mode on a phone (right).

Create a folder called `portfolio` with three files. Replace Sara's details with your own!

HTML · index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sara | Developer</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header class="navbar">
        <a href="#" class="logo">Sara.dev</a>
        <nav>
            <a href="#about">About</a>
            <a href="#projects">Projects</a>
            <a href="#contact">Contact</a>
            <button id="theme-toggle">Dark mode</button>
        </nav>
    </header>

    <main>
        <section class="hero">
            <h1>Hi, I'm Sara.</h1>
```

```
            <p>I'm learning to code, and I love building things for the web.</p>
            <a href="#projects" class="button">See my work</a>
        </section>

        <section id="about">
            <h2>About me</h2>
            <p>I started coding with Python. Now I build websites with
               HTML, CSS and JavaScript.</p>
            <ul class="skills">
                <li>Python</li>
                <li>HTML</li>
                <li>CSS</li>
                <li>JavaScript</li>
                <li>Git</li>
            </ul>
        </section>

        <section id="projects">
            <h2>Projects</h2>
            <div class="grid">
                <article class="card">
                    <h3>Guessing Game</h3>
                    <p>A Python game where you guess a secret number.</p>
                    <a href="https://github.com/">View code</a>
                </article>
                <article class="card">
                    <h3>To-Do App</h3>
                    <p>A to-do list that saves your tasks to a file.</p>
                    <a href="https://github.com/">View code</a>
                </article>
                <article class="card">
                    <h3>Contact Book</h3>
                    <p>An object-oriented app with search and JSON storage.</p>
                    <a href="https://github.com/">View code</a>
                </article>
            </div>
        </section>

        <section id="contact">
            <h2>Contact</h2>
            <form id="contact-form">
                <input type="text" id="name" placeholder="Your name" required>
                <input type="email" id="email" placeholder="Your email" required>
                <textarea id="message" rows="4" placeholder="Your message" required></texta
rea>
                <button type="submit" class="button">Send</button>
            </form>
            <p id="form-status"></p>
        </section>
    </main>

    <footer>
        <p>&copy; <span id="year"></span> Sara. Built from scratch.</p>
    </footer>

    <script src="script.js"></script>
</body>
</html>
```

CSS · style.css

```css
/* ---------- reset and colours ---------- */
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

:root {
    --bg: #ffffff;
    --text: #1f2937;
    --muted: #6b7280;
    --card: #f3f4f6;
    --accent: #2563eb;
}

body.dark {
    --bg: #0f172a;
    --text: #e5e7eb;
    --muted: #94a3b8;
    --card: #1e293b;
    --accent: #60a5fa;
}

body {
    font-family: system-ui, "Segoe UI", Roboto, Arial, sans-serif;
    background: var(--bg);
    color: var(--text);
    line-height: 1.6;
    transition: background 0.3s, color 0.3s;
}

/* ---------- navigation ---------- */
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 20px 8%;
}

.logo {
    font-size: 1.4rem;
    font-weight: bold;
    color: var(--accent);
    text-decoration: none;
}

nav a {
    margin-left: 20px;
    color: var(--text);
    text-decoration: none;
}

nav a:hover {
    color: var(--accent);
}

#theme-toggle {
    margin-left: 20px;
```

```
    padding: 6px 14px;
    border: 1px solid var(--muted);
    border-radius: 20px;
    background: none;
    color: var(--text);
    cursor: pointer;
}

/* ---------- sections ---------- */
section {
    padding: 60px 8%;
}

.hero {
    padding-top: 90px;
    padding-bottom: 90px;
}

.hero h1 {
    font-size: 3rem;
}

.hero p {
    margin: 10px 0 30px;
    font-size: 1.25rem;
    color: var(--muted);
}

h2 {
    margin-bottom: 20px;
    font-size: 2rem;
}

.button {
    display: inline-block;
    padding: 12px 28px;
    border: none;
    border-radius: 8px;
    background: var(--accent);
    color: white;
    font-size: 1rem;
    text-decoration: none;
    cursor: pointer;
}

.button:hover {
    opacity: 0.85;
}

.skills {
    display: flex;
    flex-wrap: wrap;
    gap: 10px;
    margin-top: 20px;
    list-style: none;
}

.skills li {
    padding: 6px 16px;
    border-radius: 20px;
```

```
    background: var(--card);
}

/* ---------- project cards ---------- */
.grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
}

.card {
    padding: 24px;
    border-radius: 12px;
    background: var(--card);
    transition: transform 0.2s;
}

.card:hover {
    transform: translateY(-5px);
}

.card p {
    margin: 8px 0 12px;
}

.card a {
    color: var(--accent);
}

/* ---------- contact form ---------- */
form {
    display: flex;
    flex-direction: column;
    gap: 12px;
    max-width: 500px;
}

input,
textarea {
    padding: 12px;
    border: 1px solid var(--muted);
    border-radius: 8px;
    background: var(--bg);
    color: var(--text);
    font: inherit;
}

#form-status {
    margin-top: 12px;
    color: var(--accent);
}

footer {
    padding: 30px;
    text-align: center;
    color: var(--muted);
}

/* ---------- phones and small screens ---------- */
@media (max-width: 700px) {
```

```
    .navbar {
        flex-direction: column;
        gap: 12px;
    }

    nav {
        text-align: center;
    }

    nav a,
    #theme-toggle {
        margin: 4px 6px;
    }

    .hero {
        padding-top: 50px;
        padding-bottom: 50px;
    }

    .hero h1 {
        font-size: 2.2rem;
    }

    .grid {
        grid-template-columns: 1fr;
    }
}
```

JAVASCRIPT · script.js

```
// 1. Dark mode button
const toggle = document.querySelector("#theme-toggle");

function applyTheme(theme) {
    const isDark = theme === "dark";
    document.body.classList.toggle("dark", isDark);
    toggle.textContent = isDark ? "Light mode" : "Dark mode";
}

toggle.addEventListener("click", () => {
    const newTheme = document.body.classList.contains("dark") ? "light" : "dark";
    applyTheme(newTheme);
    localStorage.setItem("theme", newTheme);   // remember the choice
});

applyTheme(localStorage.getItem("theme") || "light");

// 2. Put the current year in the footer
document.querySelector("#year").textContent = new Date().getFullYear();

// 3. Contact form (there is no server yet, so we just say thanks)
const form = document.querySelector("#contact-form");
const formStatus = document.querySelector("#form-status");

form.addEventListener("submit", (event) => {
    event.preventDefault();   // stop the page from reloading
    const name = document.querySelector("#name").value;
    formStatus.textContent = `Thanks, ${name}! (This demo doesn't send messages yet.)`;
    form.reset();
});
```

What makes this code good

- **CSS variables** (`--bg`, `--text`, `--accent`) hold all the colours in one place. Dark mode just swaps the variables by adding a `dark` class to `<body>`.
- **localStorage** saves the visitor's theme choice in their browser, so it's remembered next time.
- **Semantic HTML** (`header`, `nav`, `main`, `section`, `article`, `footer`) helps search engines and screen readers.
- **Flexbox** for the navbar and skills, **Grid** for the cards, and a **media query** for phones.

CHALLENGES

Make it truly yours:

1. Add your own projects, a photo of yourself (or an avatar) and links to your GitHub.
2. Add a "Skills" bar chart made only from `div`s and CSS widths.
3. Make the navigation bar stick to the top while scrolling (look up `position: sticky`).
4. Make the contact form really send messages, using a free form service such as Formspree.

LESSON 38

Publish your website for free

Option 1: GitHub Pages

1. Create a free account on **github.com** (if you don't have one).
2. Create a new **public** repository named exactly `YOUR-USERNAME.github.io`.
3. Click **Add file > Upload files**, and drag in `index.html`, `style.css`, `script.js` and any images. Click **Commit changes**. (Or push them with Git, as in Lesson 26.)
4. Go to the repository's **Settings > Pages**. Under *Build and deployment*, choose **Deploy from a branch**, pick the `main` branch and the `/ (root)` folder, and click **Save**.
5. Wait a minute or two, then visit **https://YOUR-USERNAME.github.io**. Your site is live, for anyone in the world!

Option 2: Netlify, Vercel or Cloudflare Pages

Sign up for free, then either **drag and drop** your project folder onto the site, or connect your GitHub repository. Connecting GitHub means that every time you `git push`, your website updates automatically.

Your own domain name (optional)

Instead of `sara.github.io` you can have `sara.dev`. Buy a domain from a registrar (for example Cloudflare, Namecheap or Porkbun). Prices are usually around $10–20 a year. Then follow your host's "custom domain" instructions to connect it. HTTPS is set up for you for free.

Checklist before you share your site

- ✓ The page has a good `<title>`, and every image has `alt` text
- ✓ It looks good on a phone (check with DevTools device mode)
- ✓ Every link works, and there are no errors in the Console
- ✓ No passwords, keys or private information are in the files
- ✓ Run **Lighthouse** (in Chrome DevTools) for a score on speed, accessibility and SEO

LESSON 39

Backends: making websites dynamic

A static site sends the same files to everyone. To have **user accounts, saved data, search or payments**, you need a **backend**: code running on a server that receives requests, uses a database, and sends back responses.

HOW A DYNAMIC WEBSITE WORKS

```
  BROWSER (frontend)                SERVER (backend)                DATABASE
 HTML + CSS + JavaScript  ──request──►  Python / Node.js / ...  ──SQL──►  PostgreSQL
                          ◄─response──  (rules, logins, logic)  ◄─data──
```

A Python backend with Flask

Install Flask with `python -m pip install flask`, then create `app.py`:

PYTHON · app.py

```python
from flask import Flask
from markupsafe import escape

app = Flask(__name__)


@app.route("/")
def home():
    return "<h1>Hello from Python!</h1>"


@app.route("/hello/<name>")
def hello(name):
    return f"<h1>Hello, {escape(name)}!</h1>"


@app.route("/api/tasks")
def tasks():
    return {"tasks": ["Learn HTML", "Learn CSS", "Learn JavaScript"]}


if __name__ == "__main__":
    app.run(debug=True)
```

Run `python app.py` and open **http://127.0.0.1:5000** in your browser. Then try **/hello/Sara** and **/api/tasks**.

- `@app.route("/")` is a **decorator**. It connects a URL to a function. When someone visits that URL, the function runs and its return value is sent back.
- `<name>` in the route captures part of the URL as a variable.
- Returning a dictionary sends **JSON**. That's how you build an API that your JavaScript can `fetch()`.
- `escape(name)` is a **security** step. Without it, someone could put JavaScript into the URL and run it on your page (an **XSS** attack, see Lesson 30).

Your frontend JavaScript can then show the data from your own API:

JAVASCRIPT

```
const response = await fetch("/api/tasks");
const data = await response.json();
for (const task of data.tasks) {
    const li = document.createElement("li");
    li.textContent = task;
    document.querySelector("#task-list").append(li);
}
```

The same backend in JavaScript, with Node.js and Express

TERMINAL

```
npm init -y
npm install express
```

JAVASCRIPT · server.js

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
    res.send("<h1>Hello from Node.js!</h1>");
});

app.get("/api/tasks", (req, res) => {
    res.json({ tasks: ["Learn HTML", "Learn CSS", "Learn JavaScript"] });
});

app.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
});
```

Run it with `node server.js`, then open **http://localhost:3000**.

Backend choices

Language	Popular frameworks	Known for
Python	**Django** (everything built in), **Flask** (small and simple), **FastAPI** (fast, modern APIs)	Easy to learn; great if your app also uses data or AI
JavaScript / TypeScript	**Express**, **Next.js**, **NestJS**	One language for frontend and backend
PHP	**Laravel**, **WordPress**	Powers a huge part of the web
Java / Kotlin	**Spring Boot**	Big companies and banks
C#	**ASP.NET Core**	Microsoft-based companies
Go	Built-in `net/http`, **Gin**	Fast, simple servers and cloud tools
Ruby	**Ruby on Rails**	Building products quickly

Putting a backend online

Backends need a server that is always running. Beginner-friendly hosts include **Render**, **Railway**, **Fly.io** and **PythonAnywhere** (for Python). Many have free or very cheap plans for small projects.

LESSON 40

Levelling up as a web developer

Frontend frameworks

When websites become big apps (like Gmail or Spotify's web player), managing the DOM by hand gets messy. **Frameworks** let you build the page from reusable **components** and update it automatically when data changes.

Framework	Notes
React	The most used. Made by Meta. Huge job market
Next.js	Built on React. Adds pages, routing and a backend: a full-stack framework
Vue	Friendly and easy to learn
Angular	Large and structured. Popular in big companies
Svelte	Very simple, fast and loved by developers

A React component that counts clicks (compare it with the DOM version in Lesson 36):

JAVASCRIPT (REACT)

```
import { useState } from "react";

function Counter() {
    const [count, setCount] = useState(0);

    return (
        <button onClick={() => setCount(count + 1)}>
            Clicked {count} times
        </button>
    );
}

export default Counter;
```

To start a React project, install Node.js, then run:

TERMINAL

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

TypeScript

TypeScript is JavaScript with **types**. You say what kind of value each variable holds, and the editor catches mistakes before you run the code. Most professional JavaScript projects now use TypeScript. Learn it after you're comfortable with JavaScript.

TYPESCRIPT

```
function greet(name: string): string {
    return `Hello, ${name}!`;
}

greet(42);   // error before running: 42 is not a string
```

The web developer roadmap

1. HTML and CSS (including Flexbox, Grid and responsive design)
2. JavaScript and the DOM
3. Git and GitHub
4. Build and publish 3 to 5 projects
5. A framework: React (or Vue or Svelte)
6. TypeScript
7. A backend: Node.js/Express or Python/Django/FastAPI, plus SQL and a database
8. Authentication, security and deployment
9. Testing, and performance and accessibility

PART 7

Building apps

Websites vs apps, mobile apps for iPhone and Android, desktop apps, web apps and games: what each one needs, how to build it and how to publish it.

In this part you will learn

- ✓ The real differences between a website, a web app and an app
- ✓ Native vs cross-platform mobile apps, and what each requires
- ✓ The same app written in SwiftUI, Kotlin, Flutter and React Native
- ✓ How to build and run your first mobile app on your own phone
- ✓ How to build desktop apps, starting with Python
- ✓ Progressive web apps (PWAs): apps you install from a website
- ✓ How games are made, with a small game in Python
- ✓ Everything an app needs, and how to publish to the App Store and Google Play

LESSON 41

Websites vs apps: what's the difference?

People use the word "app" for many different things. Here are the main kinds of software you can build, side by side:

	Website	Web app	Mobile app	Desktop app	Game
Example	A restaurant site, a portfolio	Gmail, Google Docs, Canva	WhatsApp, TikTok, Duolingo	VS Code, Photoshop, Spotify	Minecraft, Among Us
Runs in	A web browser	A web browser	iPhone / Android	Windows / Mac / Linux	Console, PC, phone or browser
Install?	No, just visit the URL	No (optionally installable as a PWA)	Yes, from an app store	Yes, download or store	Yes, usually
Main languages	HTML, CSS, JS	HTML, CSS, JS/TS + a backend	Swift, Kotlin, Dart, JS/TS	C#, C++, JS, Python, Swift	C#, C++, GDScript, Lua
Works offline?	Usually not	Partly (PWAs)	Often	Yes	Usually
Uses phone features (camera, GPS, notifications)	A few	Some	All of them	Computer features	Controllers, graphics card
Cost to publish	Free	Free to cheap	$25 once (Google) / $99 a year (Apple)	Free (own website) or store fee	Free (itch.io) or $100 per game (Steam)
Updating	Instantly for everyone	Instantly for everyone	New version, store review, users update	Users download updates	Patches

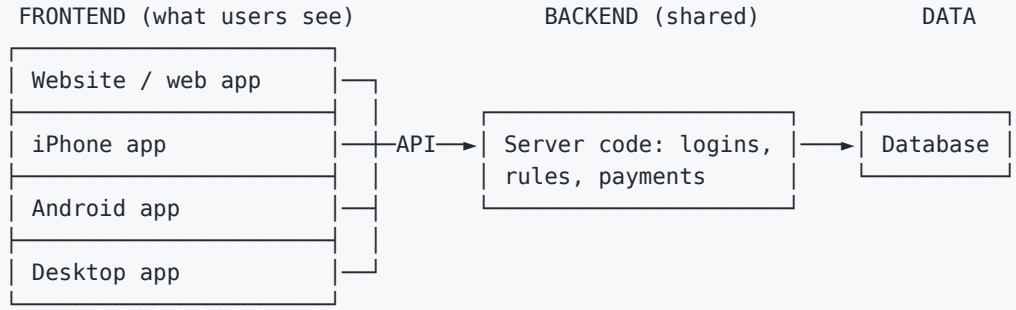
KEY IDEA

A website mainly shows information. **A web app** is a website you *use* to get something done, with accounts and saved data. **An app** usually means software **installed** on a phone or computer. Most big products (Spotify, Instagram, Discord) have all three, and they share the same **backend**.

What do they have in common?

Almost every modern app, website or game is made of the same layers you met in Part 5:

ONE BACKEND, MANY FRONTENDS



So everything you learnt about Python, APIs, databases and security is useful **whatever** you build. Only the frontend changes.

Which should I build first?

If you want...	Start with
Something online that anyone can open quickly	A **website** (Part 6)
A tool with accounts and saved data	A **web app**: website + backend
Something in people's pockets, using the camera or GPS	A **mobile app** (Lessons 42–44)
A tool for your own computer, or for offices	A **desktop app** (Lesson 45)
To make games	A **game engine** (Lesson 47)

> **TIP**
> For most beginners, the best order is **website first, then app**. Web skills (HTML, CSS, JavaScript) transfer directly to React Native and other app frameworks, and websites are free and instant to publish.

LESSON 42

Mobile apps: native vs cross-platform

There are two operating systems for phones: **iOS** (iPhone and iPad, by Apple) and **Android** (Samsung, Google Pixel, Xiaomi and most others). Worldwide, about 70% of phones run Android. In some countries, like the USA, iPhones are more common. You can build for them in two ways:

	Native apps	Cross-platform apps
What	A separate app for each platform, in its official language	One codebase that runs on both iOS and Android
Languages	**Swift** for iOS, **Kotlin** for Android	**Dart** (Flutter), **JavaScript/TypeScript** (React Native), **C#** (.NET MAUI), **Kotlin** (Kotlin Multiplatform)
Pros	Best performance; newest phone features first; feels exactly right on each phone	Write once, run on both; one team; faster and cheaper
Cons	Two codebases, so twice the work	Slightly less control; some features need extra plugins
Used by	Apple's own apps, many banking and big-company apps	Many startups; parts of apps from companies like Meta, Google, Microsoft and Shopify

What you need for each option

Option	Language	Editor / IDE	Computer needed	Test on
iOS native	Swift + SwiftUI	**Xcode** (free)	**A Mac** (Xcode only runs on macOS)	iOS Simulator, your iPhone
Android native	Kotlin + Jetpack Compose	**Android Studio** (free)	Windows, Mac or Linux	Android Emulator, your Android phone
Flutter	Dart	VS Code or Android Studio	Any (a Mac to build iOS versions)	Emulator, simulator or phone
React Native (with **Expo**)	JavaScript / TypeScript	VS Code	Any (Expo can build iOS apps in the cloud)	The **Expo Go** app on your own phone
.NET MAUI	C#	Visual Studio, VS Code	Windows or Mac	Emulator or phone
No-code / blocks	None (blocks or drag and drop)	MIT App Inventor, FlutterFlow, Thunkable	Any computer with a browser	A companion app on your phone

Which one should you choose?

- **You know JavaScript (after Part 6):** React Native with Expo. You can test on your own phone within minutes.
- **You want beautiful apps and a single, clean toolkit:** Flutter. Dart is easy if you know Python or JavaScript.
- **You only care about iPhones and have a Mac:** Swift with SwiftUI.
- **You only care about Android, or want an Android job:** Kotlin with Jetpack Compose.
- **You want to try app-making today, with no typing:** MIT App Inventor (appinventor.mit.edu), which uses blocks like Scratch.

LESSON 43

One app, four frameworks

Here is the **same app**, a counter with a button, written for each major mobile platform. Notice how similar the ideas are: a piece of **state** (`count`), a **text** that shows it, and a **button** that changes it. When the state changes, the screen updates automatically. This idea is called **declarative UI**, and every modern app framework uses it.

iOS: Swift and SwiftUI (in Xcode)

SWIFT

```swift
import SwiftUI

struct ContentView: View {
    @State private var count = 0

    var body: some View {
        VStack(spacing: 16) {
            Text("You tapped \(count) times")
                .font(.title)
            Button("Tap me") {
                count += 1
            }
            .buttonStyle(.borderedProminent)
        }
    }
}
```

Android: Kotlin and Jetpack Compose (in Android Studio)

KOTLIN

```kotlin
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }

    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text("You tapped $count times", fontSize = 24.sp)
        Button(onClick = { count++ }) {
            Text("Tap me")
        }
    }
}
```

Both platforms: Flutter and Dart

DART

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: CounterPage()));

class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int count = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Text('You tapped $count times',
            style: const TextStyle(fontSize: 24)),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() => count++),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Both platforms: React Native (JavaScript)

JAVASCRIPT · App.js (React Native)

```javascript
import { useState } from "react";
import { Button, StyleSheet, Text, View } from "react-native";

export default function App() {
  const [count, setCount] = useState(0);

  return (
    <View style={styles.container}>
      <Text style={styles.title}>You tapped {count} times</Text>
      <Button title="Tap me" onPress={() => setCount(count + 1)} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: "center", justifyContent: "center" },
  title: { fontSize: 24, marginBottom: 16 },
});
```

> **KEY IDEA**
> Once you understand **state + components + events** in one framework, the others are easy to pick up. That's why learning one well matters more than which one you pick.

LESSON 44

Your first mobile app, step by step

We'll use **React Native with Expo**, because it runs on Windows, Mac and Linux and you can test on your **real phone** (iPhone or Android) without installing Xcode or Android Studio.

Step 1: Install the tools

1. Install **Node.js** (the LTS version) from **nodejs.org**. This also installs `npm` and `npx`.
2. On your phone, install the free **Expo Go** app from the App Store or Google Play.

Step 2: Create the project

TERMINAL

```
npx create-expo-app@latest my-first-app --template blank
cd my-first-app
npx expo start
```

A **QR code** appears in the terminal. Make sure your phone and computer are on the **same Wi-Fi**. Then:

- **Android:** open Expo Go and scan the QR code.
- **iPhone:** open the Camera app, point it at the QR code and tap the link.

Your app opens on your phone!

Step 3: Change it

Open the project folder in VS Code. Replace everything in `App.js` with the React Native counter from Lesson 43 and save. The app on your phone **updates within a second**. This is called **fast refresh**.

Step 4: Grow it

Turn the counter into something useful, step by step. A great first app is a **to-do list**: a `TextInput` to type tasks, a button to add them, and a `FlatList` to show them. Use `useState` to keep the list. Then save the list on the phone with the `AsyncStorage` package, so tasks survive when the app closes. (Sound familiar? It's Project 2 again, as an app!)

Going further

Feature	What to look up
Several screens	**Expo Router** (navigation between screens)
Camera, location, notifications	`expo-camera`, `expo-location`, `expo-notifications`
Save data on the phone	`AsyncStorage`, or `expo-sqlite` for a real database
User accounts and cloud data	**Firebase** or **Supabase**
Build the app for the stores	**EAS Build** and **EAS Submit** (Expo's build service)

GOOD TO KNOW

The Flutter path is very similar: install the Flutter SDK (flutter.dev), run `flutter create my_app`, then `cd my_app` and `flutter run` with a phone or emulator connected, and edit `lib/main.dart`. Flutter's official "Get started" guide walks you through installing everything.

LESSON 45

Desktop apps

Desktop apps run on Windows, macOS or Linux. Many of the apps you use every day, like VS Code, Discord, Slack and Spotify, are built with **web technology** wrapped in a desktop window.

Tool	Language	Good for	Famous examples
tkinter	Python (built in)	Learning, simple tools	Small utilities, school projects
PySide6 / PyQt	Python	Professional Python apps	Scientific and engineering tools
Electron	HTML, CSS, JavaScript	Cross-platform apps using web skills	VS Code, Discord, Slack
Tauri	Web frontend + Rust	Like Electron but much smaller and lighter	A growing number of modern apps
WPF / WinUI / .NET MAUI	C#	Windows apps	Many business and office apps
SwiftUI / AppKit	Swift	Mac apps	Apple's own Mac apps
Qt	C++ (or Python)	Fast, cross-platform professional apps	Many design and engineering programs
Flutter	Dart	One codebase for mobile and desktop	Growing in popularity

Your first desktop app in Python

You already know enough Python to build a real window with buttons. `tkinter` comes with Python, so there is nothing to install:

PYTHON · tip_app.py

```python
import tkinter as tk


def calculate():
    try:
        bill = float(bill_entry.get())
        tip = bill * 0.15
        result_label.config(text=f"Tip: ${tip:.2f}    Total: ${bill + tip:.2f}")
    except ValueError:
        result_label.config(text="Please enter a number.")


window = tk.Tk()
window.title("Tip Calculator")
window.geometry("320x170")

tk.Label(window, text="Bill amount:").pack(pady=5)
bill_entry = tk.Entry(window)
bill_entry.pack()
tk.Button(window, text="Calculate 15% tip", command=calculate).pack(pady=10)
result_label = tk.Label(window, text="")
result_label.pack()

window.mainloop()
```

- `tk.Tk()` creates the window. `Label`, `Entry` (a text box) and `Button` are **widgets**.
- `.pack()` places each widget in the window, from top to bottom.
- `command=calculate` connects the button to your function. This is **event-driven programming**: your code runs when the user does something.
- `window.mainloop()` keeps the window open and waits for events. It's like a `while True` loop that is built in.

Turning it into a real program file

Friends without Python can't run a `.py` file. **PyInstaller** packages your program into one file that runs on its own:

TERMINAL

```
python -m pip install pyinstaller
pyinstaller --onefile --windowed tip_app.py
```

Your program appears in the `dist` folder (a `.exe` file on Windows). You must build on each system you want to support: on Windows for Windows, on a Mac for Mac.

GOOD TO KNOW

To share desktop apps widely, developers usually **code-sign** them, so Windows and macOS don't show scary "unknown developer" warnings. They then publish on their own website, the **Microsoft Store**, the **Mac App Store**, or Linux app stores like **Flathub**.

LESSON 46

Web apps and PWAs

A **progressive web app (PWA)** is a website that can be **installed** like an app. It gets its own icon on the home screen, opens full-screen without the browser bar, and can work offline. It's the cheapest way to get an "app" on every device, because it's one website, with no app stores and no fees.

To turn a website into a PWA, you add:

1. A **web app manifest** (a JSON file describing the app's name, icons and colours)
2. A **service worker** (a JavaScript file that can cache your files so the app works offline)
3. **HTTPS** (all the free hosts from Lesson 38 already provide it)

JSON · manifest.json

```
{
  "name": "My Tasks",
  "short_name": "Tasks",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#2563eb",
  "icons": [
    { "src": "icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

HTML

```
<link rel="manifest" href="manifest.json">
```

Users then install it from the browser: Chrome and Edge show an **Install** button, and on iPhone it's **Share > Add to Home Screen**.

	PWA	Native / cross-platform app
Cost	Free	Store fees, plus usually more development time
Updates	Instant: just update the website	Store review, then users update
Found in app stores?	Not by default (Android can list them with extra work)	Yes
Phone features	Many (camera, location, notifications), but fewer on iPhone	All of them
Best for	Tools, dashboards, simple apps, a first version	Apps that need full phone power or app-store discovery

LESSON 47

How games are made

Games are some of the most fun things to program, and some of the hardest, because they combine code, maths, art, sound and design.

Game engines

A **game engine** handles the hard parts for you: drawing graphics, physics, sound, input and exporting to different platforms.

Engine	Language	Cost	Known for
Unity	C#	Free for small creators	Mobile and indie games; *Hollow Knight*, *Among Us*, *Pokémon GO*
Unreal Engine	C++ and Blueprints (visual scripting)	Free; a share of revenue only for very successful games	Realistic 3D graphics; *Fortnite*
Godot	GDScript (very similar to Python), C#	Free and open source	2D and 3D indie games. Great for beginners
Roblox Studio	Luau (a version of Lua)	Free	Games inside Roblox. Huge young community
GameMaker	GML	Free to start	2D games; *Undertale*
Pygame	Python	Free	Learning how games work from scratch

Every game is a loop

All games, from Pong to Fortnite, run the same **game loop** about 60 times a second:

1. **Input:** what keys, buttons or touches did the player press?
2. **Update:** move things, check for collisions, update the score.
3. **Draw:** clear the screen and draw everything in its new position.

A tiny game in Python with Pygame

Install it with `python -m pip install pygame`, then run this. Use the arrow keys to move the square:

PYTHON · square_game.py

```python
import pygame

pygame.init()
screen = pygame.display.set_mode((640, 480))
pygame.display.set_caption("Move the square")
clock = pygame.time.Clock()
x, y = 300, 220

running = True
while running:
    # 1. input
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    keys = pygame.key.get_pressed()

    # 2. update
    if keys[pygame.K_LEFT]:
        x -= 5
    if keys[pygame.K_RIGHT]:
        x += 5
    if keys[pygame.K_UP]:
        y -= 5
    if keys[pygame.K_DOWN]:
        y += 5

    # 3. draw
    screen.fill((30, 30, 40))
    pygame.draw.rect(screen, (0, 200, 255), (x, y, 40, 40))
    pygame.display.flip()
    clock.tick(60)          # 60 frames per second

pygame.quit()
```

CHALLENGES

Turn it into a real game:

1. Stop the square leaving the screen (keep `x` between 0 and 600, and `y` between 0 and 440).
2. Add a red "coin" at a random position. When the square touches it (`pygame.Rect.colliderect`), add 1 to a score and move the coin.
3. Show the score on screen with `pygame.font.Font(None, 36).render(...)`.
4. Add an enemy that chases the player, and a game-over screen.

TIP

Want to make games seriously? After Pygame, try **Godot** (free, and GDScript feels like Python) and follow its official "Your first 2D game" tutorial. Game jams on **itch.io**, where you make a small game in a weekend, are a fantastic way to learn.

LESSON 48

Everything an app needs

Writing the code is only part of making an app. Here is everything a real app usually needs, from idea to launch:

Area	What it means	Common tools
1. Idea and users	Which problem? For whom? Talk to real people who might use it	Notes, interviews, surveys
2. Design (UI/UX)	Sketch every screen and how users move between them, before coding	Paper, **Figma**
3. Frontend	The screens users touch	React, SwiftUI, Compose, Flutter, React Native
4. Backend	Server code for anything shared: accounts, messages, payments	Python, Node.js, Go... or **Firebase** / **Supabase**
5. Database	Stores users and their data	PostgreSQL, SQLite, Firestore
6. Authentication	Sign-up, login, password reset, "Sign in with Google/Apple"	Firebase Auth, Supabase Auth, Auth0, Clerk
7. Hosting	Somewhere for the backend and website to run	Vercel, Render, AWS, Google Cloud
8. Testing	Automated tests plus testing on real, different-sized devices	pytest, Jest, XCTest, beta testers
9. Security and privacy	Protect user data; follow privacy laws like GDPR	Lesson 30's rules
10. Accessibility	Usable by people with disabilities: screen readers, contrast, text size	Platform accessibility checkers
11. Store listing	App name, icon, screenshots, description, **privacy policy** (required by the stores)	App Store Connect, Google Play Console
12. Analytics and crashes	See how people use the app and catch crashes	Firebase Analytics, Sentry
13. Updates and support	Fix bugs, answer users, add features	GitHub Issues, email

TIP

Start with an **MVP** (minimum viable product): the smallest version that is actually useful. Ship it, get feedback, improve. Most successful apps started very small.

LESSON 49

Publishing your app

Where	Cost	Requirements	Review
Apple App Store (iPhone, iPad, Mac)	**$99 per year** (Apple Developer Program)	Usually a Mac with Xcode to build and upload (cloud services like Expo EAS can help); app icons, screenshots and a privacy policy	Every version is reviewed by Apple. Usually takes a day or two
Google Play (Android)	**$25 once**	A Google Play Console account and identity verification. New personal accounts must first run a closed test with a group of testers (currently at least 12 testers for 14 days)	Automated plus human review. Usually a few hours to a few days
The web (website, web app, PWA)	Free to a few dollars a month	Hosting, plus an optional domain	None. You publish instantly
Desktop (your own website)	Free (code-signing certificates cost extra)	An installer or package for each system	None
Microsoft Store / Mac App Store	Low or no registration fee / included in Apple's $99	Store account and packaged app	Yes
Games: itch.io / Steam	Free / $100 per game on Steam	Game builds, store page, trailer	Steam reviews your store page and build

WATCH OUT

App store rules and prices change. Before you publish, always read the **current** official pages: Apple's *App Review Guidelines* and Google's *Play Console Help*.

How apps make money

- **Paid apps:** users pay once to download.
- **In-app purchases and subscriptions:** free to download, with paid extras or a monthly plan. This is the most common model today.
- **Ads:** free for users; you are paid by advertisers (for example through Google AdMob).
- **Store commission:** Apple and Google keep **15–30%** of digital sales (15% for most small developers).

LESSON 50

AI-powered apps

Many new apps include AI features: chat assistants, summarising text, translating, recognising images or generating content. You usually don't build the AI model yourself. Instead, your app **calls an AI model through an API** (Lesson 28), just like it would call a weather API:

1. Your app's **backend** sends a request to an AI provider (such as Anthropic's Claude, OpenAI or Google Gemini) with the user's text.
2. The model sends back its answer, usually as JSON.
3. Your backend checks it and sends it to your app's screen.

> **WATCH OUT**
> Keep AI API keys **on your backend**, never in your website's JavaScript or inside your mobile app, where anyone could find them and spend your money. Also check what your app sends: never send private user data without permission.

To go deeper into AI itself (training your own models), learn Python, NumPy, pandas, statistics and then **scikit-learn** and **PyTorch**. Part 9 shows the full AI/ML path.

PART 8

A tour of programming languages

20 languages: what each one is for, how hard it is, which tools it uses, and the same small program written in each, so you can compare.

In this part you will learn

- ✓ How languages differ: typing, compiling and what they're designed for
- ✓ The web languages: HTML/CSS, JavaScript, TypeScript, PHP, Ruby
- ✓ General-purpose languages: Python, Java, C#, Go
- ✓ Mobile languages: Swift, Kotlin, Dart
- ✓ Systems languages: C, C++, Rust and assembly
- ✓ Data and scripting languages: SQL, R, Bash, Lua and more
- ✓ Which language to learn for which goal

LESSON 51

The big picture

There is no single "best" language. Each was designed for certain jobs. Professional developers usually know **two or three** well and can read several more. Here are the ideas that make languages different:

Idea	Meaning	Examples
Compiled vs interpreted	Translated before running, or while running (Lesson 0.3)	Compiled: C, Rust, Go. Interpreted: Python, Ruby
Static vs dynamic typing	**Static:** you (or the compiler) fix each variable's type before running, so many mistakes are caught early. **Dynamic:** types are checked while running. More flexible, but some mistakes appear later	Static: Java, C#, TypeScript, Go, Rust. Dynamic: Python, JavaScript, Ruby
Memory management	**Garbage-collected:** the language cleans up unused memory for you. **Manual:** you manage it yourself (fast, but easy to get wrong)	GC: Python, Java, Go. Manual: C, C++. Rust: checked by the compiler
Paradigm	The style of thinking the language encourages: step by step (procedural), objects (OOP), or functions (functional)	Most modern languages mix all three
Ecosystem	Libraries, tools, community and jobs. Often more important than the language itself!	npm (JavaScript), PyPI (Python), crates.io (Rust)

All 20 languages at a glance

Language	Main uses	Beginner difficulty	Typing
Python	AI, data, automation, backends, education	Easy	Dynamic
JavaScript	Websites, web apps, servers, apps	Easy–medium	Dynamic
TypeScript	Large JavaScript projects	Medium	Static
HTML & CSS	Web page structure and style (not programming)	Easy	n/a
Java	Big company backends, Android, schools	Medium	Static
C#	Unity games, Windows apps, company backends	Medium	Static
Go	Cloud services, servers, tools	Medium	Static
Swift	iPhone, iPad and Mac apps	Medium	Static
Kotlin	Android apps, backends	Medium	Static
Dart	Flutter apps (mobile, web, desktop)	Easy–medium	Static
C	Operating systems, devices, microcontrollers	Hard	Static
C++	Games, engines, browsers, fast software	Hard	Static
Rust	Safe, fast systems software	Hard	Static
PHP	Websites, WordPress, Laravel	Easy–medium	Dynamic
Ruby	Web apps (Ruby on Rails)	Easy	Dynamic
SQL	Databases	Easy to start	Declarative
R	Statistics and data analysis	Medium	Dynamic
Bash / PowerShell	Automating the terminal and servers	Medium	Dynamic
Lua / Luau	Game scripting (Roblox), plugins	Easy	Dynamic
Scratch	Learning to think in code	Very easy	Blocks

LESSON 52

The web languages

Every example in this part does the same thing: it greets Sara and Omar. Every one of them prints:

```
OUTPUT
Hello, Sara!
Hello, Omar!
```

HTML and CSS

Covered in Part 6. They describe the structure (HTML) and style (CSS) of every web page. Every web developer needs them, whatever else they use.

JavaScript

Used for	Every interactive website; web apps; servers (Node.js); mobile apps (React Native); desktop apps (Electron)
Difficulty	Easy to start, with some confusing quirks
Tools	Any browser, VS Code, Node.js, npm
Who uses it	Practically every tech company

The only language that runs natively in web browsers, which makes it the most widely used language in the world. Created in 1995 in just 10 days.

JAVASCRIPT
```javascript
function greet(name) {
    return `Hello, ${name}!`;
}

for (const name of ["Sara", "Omar"]) {
    console.log(greet(name));
}
```

Run it with: `node greet.js`

TypeScript

Used for	The same things as JavaScript, in bigger and more serious projects
Difficulty	Medium (it's JavaScript plus types)
Tools	VS Code (which is itself written in TypeScript), Node.js, `tsc`
Who uses it	Microsoft (its creator), Google, Airbnb, Slack and most modern web teams

JavaScript with **types** added. The editor catches mistakes like passing a number where text was expected, before the code even runs. It is compiled into normal JavaScript. Most professional web projects use it today.

TYPESCRIPT
```typescript
function greet(name: string): string {
    return `Hello, ${name}!`;
}

const names: string[] = ["Sara", "Omar"];
for (const name of names) {
    console.log(greet(name));
}
```

Run it with: `npx tsc greet.ts && node greet.js`

PHP

Used for	Server-side websites; WordPress (which runs a large share of all websites); Laravel apps
Difficulty	Easy to medium
Tools	VS Code or PhpStorm; a local server like XAMPP or Laravel Herd
Who uses it	WordPress, Wikipedia, Facebook (originally), Slack (parts of its backend)

Often joked about, but it still powers a huge part of the web, and modern PHP with **Laravel** is pleasant and productive.

PHP
```php
<?php
function greet($name) {
    return "Hello, $name!";
}

$names = ["Sara", "Omar"];
foreach ($names as $name) {
    echo greet($name) . "\n";
}
```

Run it with: `php greet.php`

Ruby

Used for	Web apps with **Ruby on Rails**; scripting
Difficulty	Easy. It was designed to make programmers happy
Tools	VS Code or RubyMine, the `gem` package manager
Who uses it	GitHub, Shopify and Airbnb were built with Rails

Elegant and readable, a bit like Python. Rails lets small teams build complete web products very quickly.

RUBY
```ruby
def greet(name)
  "Hello, #{name}!"
end

["Sara", "Omar"].each do |name|
  puts greet(name)
end
```

Run it with: `ruby greet.rb`

LESSON 53

General-purpose languages

Python

Used for	AI and machine learning, data science, automation and scripting, web backends, science, education
Difficulty	Easy: the best first language
Tools	VS Code, PyCharm, Jupyter notebooks, pip
Who uses it	Google, Netflix, Instagram, NASA, nearly every AI company

You already know it! Its huge library ecosystem is why it dominates AI and data. Its weakness is speed, which is why heavy maths libraries are written in C underneath.

PYTHON

```python
def greet(name):
    return f"Hello, {name}!"


for name in ["Sara", "Omar"]:
    print(greet(name))
```

Run it with: `python3 greet.py`

Java

Used for	Large company systems (banks, airlines, shops), Android apps, big data tools, *Minecraft: Java Edition*
Difficulty	Medium: wordy, but clear and strict
Tools	**IntelliJ IDEA**, Eclipse, VS Code; Maven or Gradle for building
Who uses it	Amazon, Netflix, LinkedIn, most banks, and many schools (AP Computer Science uses Java)

"Write once, run anywhere": Java code runs on the **Java Virtual Machine** (JVM) on any system. Its strict rules make huge codebases with many developers manageable.

```java
public class Main {
    static String greet(String name) {
        return "Hello, " + name + "!";
    }

    public static void main(String[] args) {
        String[] names = {"Sara", "Omar"};
        for (String name : names) {
            System.out.println(greet(name));
        }
    }
}
```

Run it with: `javac Main.java && java Main`

C#

Used for	**Unity** games, Windows apps, company web backends (ASP.NET), cross-platform apps (.NET MAUI)
Difficulty	Medium: similar to Java, but a bit more modern
Tools	**Visual Studio**, VS Code with the C# Dev Kit, JetBrains Rider; the .NET SDK
Who uses it	Microsoft, Stack Overflow, countless game studios

Pronounced "C sharp". Created by Microsoft. The top choice if you want to make games with Unity.

```csharp
string Greet(string name) => $"Hello, {name}!";

string[] names = { "Sara", "Omar" };
foreach (var name in names)
{
    Console.WriteLine(Greet(name));
}
```

Run it with: `dotnet run`

Go

Used for	Cloud infrastructure, fast web servers, command-line tools
Difficulty	Medium. A deliberately small, simple language
Tools	VS Code with the Go extension, GoLand; `go` command
Who uses it	Google (its creator), Uber, Twitch; Docker and Kubernetes are written in Go

Also called Golang. Simple to read, very fast to compile, and great at doing many things at once (**concurrency**).

GO

```go
package main

import "fmt"

func greet(name string) string {
    return "Hello, " + name + "!"
}

func main() {
    names := []string{"Sara", "Omar"}
    for _, name := range names {
        fmt.Println(greet(name))
    }
}
```

Run it with: `go run greet.go`

LESSON 54

Mobile languages

Swift

Used for	Apps for iPhone, iPad, Mac, Apple Watch and Apple TV
Difficulty	Medium, with a friendly modern syntax
Tools	**Xcode** (Mac only); Swift Playgrounds on iPad and Mac is great for learning
Who uses it	Apple and every company with an iPhone app

Created by Apple in 2014 to replace Objective-C. Combined with **SwiftUI**, it's the official way to build Apple apps (Lesson 43).

SWIFT
```swift
func greet(_ name: String) -> String {
    return "Hello, \(name)!"
}

let names = ["Sara", "Omar"]
for name in names {
    print(greet(name))
}
```

Run it with: `swift greet.swift`

Kotlin

Used for	Android apps (Google's recommended language), backends, cross-platform apps with Kotlin Multiplatform
Difficulty	Medium: cleaner and shorter than Java
Tools	**Android Studio**, IntelliJ IDEA
Who uses it	Google, Netflix, Pinterest and most Android developers

Made by JetBrains. Runs on the Java Virtual Machine and works with all Java code, but it is safer and much less wordy.

KOTLIN
```kotlin
fun greet(name: String): String = "Hello, $name!"

fun main() {
    val names = listOf("Sara", "Omar")
    for (name in names) {
        println(greet(name))
    }
}
```

Run it with: `kotlinc greet.kt -include-runtime -d greet.jar && java -jar greet.jar`

Dart

Used for	**Flutter** apps for iOS, Android, web and desktop from one codebase
Difficulty	Easy to medium. Feels familiar if you know JavaScript or Java
Tools	VS Code or Android Studio with the Flutter plugin; DartPad (dartpad.dev) in the browser
Who uses it	Google (its creator), BMW, Alibaba, and many startups

Almost everyone who learns Dart does so to use Flutter.

DART

```dart
String greet(String name) => 'Hello, $name!';

void main() {
  final names = ['Sara', 'Omar'];
  for (final name in names) {
    print(greet(name));
  }
}
```

Run it with: `dart run greet.dart`

LESSON 55

Systems languages: close to the machine

C

Used for	Operating systems (Linux, the core of Windows and macOS), devices and microcontrollers, databases, other languages (Python itself is written in C)
Difficulty	Hard: you manage memory yourself
Tools	GCC or Clang compilers; VS Code, CLion; Arduino IDE for electronics
Who uses it	Everyone, indirectly: it's under almost everything

Created in 1972 and still everywhere. Learning C teaches you how computers really work: memory, pointers and the CPU. Many computer science degrees teach it for exactly that reason.

```c
#include <stdio.h>

void greet(const char *name) {
    printf("Hello, %s!\n", name);
}

int main(void) {
    const char *names[] = {"Sara", "Omar"};
    for (int i = 0; i < 2; i++) {
        greet(names[i]);
    }
    return 0;
}
```

Run it with: `gcc greet.c -o greet && ./greet`

C++

Used for	Game engines (Unreal), high-performance games, web browsers (Chrome), trading systems, robotics
Difficulty	Hard: a huge and powerful language
Tools	Visual Studio, CLion, VS Code; GCC, Clang or MSVC compilers
Who uses it	Game studios, Google, Adobe, Microsoft

C plus classes and much more. Chosen when every millisecond counts.

```cpp
#include <iostream>
#include <string>
#include <vector>

std::string greet(const std::string& name) {
    return "Hello, " + name + "!";
}

int main() {
    std::vector<std::string> names = {"Sara", "Omar"};
    for (const auto& name : names) {
        std::cout << greet(name) << "\n";
    }
    return 0;
}
```

Run it with: `g++ greet.cpp -o greet && ./greet`

Rust

Used for	Safe, fast systems software: browsers, operating systems, command-line tools, WebAssembly
Difficulty	Hard at first: the compiler is very strict
Tools	`cargo` (excellent built-in build tool), VS Code with rust-analyzer, RustRover
Who uses it	Mozilla (where it started), Microsoft, Amazon, Cloudflare; parts of the Linux and Windows kernels

As fast as C and C++, but its compiler **prevents whole categories of memory bugs** before the program runs. For years it has been voted the most-admired language in developer surveys.

```rust
fn greet(name: &str) -> String {
    format!("Hello, {}!", name)
}

fn main() {
    let names = ["Sara", "Omar"];
    for name in names {
        println!("{}", greet(name));
    }
}
```

Run it with: `cargo run` (or: `rustc greet.rs && ./greet`)

Assembly

Assembly language is one step above machine code: each line is one CPU instruction, such as "move this number into this register". Today it is mostly used for tiny devices, performance tricks, reverse engineering and security research. You don't need it to start, but it's fascinating once you're curious about how CPUs work.

LESSON 56

Data and scripting languages

SQL

Used for	Asking databases questions, and changing their data (Lesson 29)
Difficulty	Easy to start, deep to master
Tools	DB Browser for SQLite, pgAdmin, DBeaver, any database
Who uses it	Developers, data analysts, scientists, product managers. Almost everyone in tech

SQL is **declarative**: you describe *what* you want, and the database works out *how* to get it. It is one of the most valuable skills in tech, whatever your job.

```sql
SELECT name, score
FROM students
WHERE score >= 80
ORDER BY score DESC;
```

R

Used for	Statistics, data analysis and charts, especially in science, medicine and research
Difficulty	Medium. A little unusual, but very powerful for data
Tools	**RStudio** (Posit), the tidyverse and ggplot2 libraries
Who uses it	Universities, researchers, health organisations, data analysts

Python and R are the two main languages for data. R is especially strong in classic statistics and research.

```r
greet <- function(name) {
  paste0("Hello, ", name, "!")
}

names <- c("Sara", "Omar")
for (name in names) {
  cat(greet(name), "\n", sep = "")
}
```

Run it with: `Rscript greet.R`

Bash (and PowerShell)

Used for	Automating the terminal: running commands, managing servers, build scripts
Difficulty	Medium: the syntax is fiddly
Tools	Any terminal on Mac or Linux (and WSL on Windows); **PowerShell** is Windows' equivalent
Who uses it	DevOps engineers, back-end developers, system administrators

A **shell script** is a list of terminal commands saved in a file. If you find yourself typing the same commands every day, script them!

TERMINAL
```
#!/bin/bash
greet() {
    echo "Hello, $1!"
}

for name in Sara Omar; do
    greet "$name"
done
```

Run it with: `bash greet.sh`

Lua (and Luau)

Used for	Game scripting: **Roblox** (Luau), plugins and mods for games and apps
Difficulty	Easy: a small and simple language
Tools	**Roblox Studio** (Luau), VS Code, LÖVE (a 2D game framework)
Who uses it	Roblox creators, game studios, and app makers adding plugin systems

If you want to make Roblox games, this is the language you'll write, in its Luau version.

LUA
```
local function greet(name)
    return "Hello, " .. name .. "!"
end

for _, name in ipairs({"Sara", "Omar"}) do
    print(greet(name))
end
```

Run it with: `lua greet.lua`

Others for science and maths

MATLAB (engineering and science at universities, paid) and **Julia** (fast scientific computing, free) are common in research and engineering. Python with NumPy can do most of the same work.

LESSON 57

Block-based coding and other languages you'll hear about

Scratch and block coding

Scratch (scratch.mit.edu, from MIT) lets you build games and animations by snapping blocks together. It teaches loops, decisions, variables and events without any typing errors. **MIT App Inventor** uses the same idea for phone apps, and **Blockly** and **Code.org** for lessons. It's ideal for young learners, or for anyone who finds typed code scary at first.

Other languages, in one line each

Language	What it is
Scala	Functional + object-oriented on the JVM; used for big data (Apache Spark)
Elixir	Very reliable systems that handle millions of connections (built on Erlang, which powers WhatsApp-style messaging)
Haskell	A purely functional language; popular in universities and research
Objective-C	The older language for Apple apps, replaced by Swift
Perl	An older scripting language for processing text; still found on servers
COBOL	From 1959! Still runs many banks and government systems
Fortran	From 1957; still used for scientific supercomputing
Zig	A modern, simple alternative to C
Solidity	Smart contracts on the Ethereum blockchain
GDScript	Godot game engine's own language, very similar to Python
Visual Basic / VBA	Automating Microsoft Excel and Office

LESSON 58

Which language for which goal?

I want to...	Learn	Then add
Learn to program (any goal)	**Python**	Whatever your goal below needs
Build websites	**HTML, CSS, JavaScript**	TypeScript, React, then a backend
Build web apps with accounts and data	**JavaScript/TypeScript** or **Python**	SQL, Node.js or Django/FastAPI
Make iPhone apps	**Swift**	SwiftUI, Xcode
Make Android apps	**Kotlin**	Jetpack Compose, Android Studio
Make apps for both phones	**Dart** (Flutter) or **JavaScript** (React Native)	Firebase or Supabase
Make games	**C#** (Unity), **GDScript** (Godot), **Lua** (Roblox)	C++ for Unreal or engine work
Work in AI and machine learning	**Python**	NumPy, pandas, PyTorch, maths and statistics
Analyse data	**Python** or **R**, and **SQL**	Excel, Power BI or Tableau, statistics
Work in cybersecurity	**Python** and **Bash**	Networking, Linux, C for understanding exploits
Work in cloud and DevOps	**Bash** and **Python** or **Go**	Linux, Docker, Kubernetes, a cloud provider
Program robots and electronics	**C** / **C++** (Arduino) or **MicroPython**	Electronics basics, Raspberry Pi
Get a job at a big company	**Java**, **C#**, **Python** or **JavaScript**	Data structures and algorithms, SQL, Git
Pass a school computing exam	**Whatever your course uses**	Lots of practice with past papers

KEY IDEA

Your **first** language matters less than you think. The second one is learnt in weeks, and the third in days. What matters is **going deep** in one language first and **building real projects** with it.

PART 9

Your path forward

Careers in tech, a 12-month study plan, 30 project ideas, the best free resources, and how to use AI to learn faster.

In this part you will learn

- ✓ The main careers in software and what each one needs
- ✓ A month-by-month plan for your first year
- ✓ 30 project ideas, from beginner to advanced
- ✓ The best free courses, websites and communities
- ✓ How to use AI assistants to learn, without cheating yourself
- ✓ How to build a portfolio and get your first opportunity

LESSON 59

Career paths in software

Path	What you build	Languages	Key tools and skills
Front-end developer	The parts of websites and web apps that users see	HTML, CSS, JavaScript, TypeScript	React (or Vue/Svelte), Tailwind, accessibility, Figma
Back-end developer	Servers, APIs, databases, business logic	Python, JavaScript, Java, C#, Go	SQL, PostgreSQL, REST APIs, Docker, security
Full-stack developer	Both front-end and back-end	JavaScript/TypeScript + one backend language	React + Node.js or Django; databases; deployment
Mobile developer	iPhone and Android apps	Swift, Kotlin, Dart, JavaScript	SwiftUI, Jetpack Compose, Flutter, React Native
Game developer	Games for PC, console, mobile and the web	C#, C++, GDScript, Lua	Unity, Unreal, Godot, maths, physics
Data analyst	Reports and dashboards that answer business questions	SQL, Python or R	Excel, pandas, Power BI or Tableau, statistics
Data scientist / ML engineer	Models that predict, recommend and classify	Python	NumPy, pandas, scikit-learn, PyTorch, statistics, linear algebra
AI engineer	Products built on AI models: assistants, agents, search	Python, TypeScript	LLM APIs, prompt design, evaluation, vector databases
Cybersecurity	Protecting systems; finding weaknesses legally	Python, Bash, some C	Networking, Linux, Wireshark, Burp Suite, CTF practice
DevOps / cloud engineer	The systems that build, ship and run software	Bash, Python, Go	Linux, Docker, Kubernetes, AWS/Azure/GCP, CI/CD, Terraform
Embedded / robotics	Software inside devices, cars and robots	C, C++, Rust, MicroPython	Arduino, Raspberry Pi, electronics
QA / test engineer	Automated tests that keep products working	Python, JavaScript, Java	pytest, Playwright, Selenium, CI

TIP

You don't need to choose now. Finish Parts 2 to 6, build a few projects, and notice which ones you **enjoyed** most. Enjoyment is the best predictor of how far you'll go.

Do I need a university degree?

A computer science degree helps, especially for big companies and for AI research, and it teaches valuable theory. But many developers are **self-taught** or come from short courses. What every employer wants to see is the same: **proof that you can build things**. That means projects, a GitHub profile and the ability to explain your code.

LESSON 60

Your 12-month roadmap

Month	Focus	By the end you have
1	Parts 0–2: Python basics	The guessing game, and Quiz 1 passed
2	Part 3: lists, dicts, functions, files	The to-do app, and Quiz 2 passed
3	Part 4 + Git (Lesson 26)	The contact book, on GitHub, and Quiz 3 passed
4	Part 5: terminal, APIs, SQL, security	A Python program that uses a real API and a SQLite database
5	Part 6: HTML and CSS	2–3 small, responsive web pages
6	Part 6: JavaScript	Your portfolio website, live online
7	Part 7: apps (choose web app, mobile or games)	Your first app running on your phone or computer
8–9	Choose a path (Lesson 59) and go deep	2 medium projects in that path
10–11	One bigger project, done properly	A project with tests, a README, a live demo and real users (even 5 friends!)
12	Polish and share	A clean GitHub, an updated portfolio, and your first contribution to open source or a hackathon

A weekly routine that works

Day	What to do (30–60 minutes)
Mon–Thu	**10 min** review yesterday's topic · **25 min** a new lesson with exercises · **10 min** work on your project
Fri	A coding puzzle (Exercism, Codewars), and fix something in an old project
Weekend	**Build.** A longer session on your project. This is where skills really grow
Every 2 weeks	Look back: what did I learn? What confused me? Write it down

KEY IDEA

Consistency beats intensity. **One hour every day for a year** turns a complete beginner into someone who can build real software.

LESSON 61

30 project ideas, from easy to advanced

Beginner (Parts 2–3)

Project	Practises
Calculator with a menu	input, if/elif, functions
Rock, paper, scissors against the computer	random, if, loops, score
Unit converter (km/miles, °C/°F, kg/lb)	functions, maths
Times-table quiz with a score	loops, random, input
Password generator and strength checker	strings, random, conditions
Hangman	strings, lists, loops
Mad Libs story generator	input, f-strings
Dice-rolling statistics	random, dictionaries, counting
Countdown timer	time module, loops
Grade book (average, highest, lowest)	lists, functions, files

Intermediate (Parts 4–7)

Project	Practises
Expense tracker with CSV export and a chart	files, dictionaries, matplotlib
Weather app using a free API	requests/fetch, JSON, error handling
Flashcard study app	classes, JSON, random
Browser quiz game	HTML, CSS, JavaScript, the DOM
Web scraper that collects headlines	requests, BeautifulSoup
Snake or Breakout in Pygame	game loop, collisions
Discord or Telegram bot	APIs, events
Habit tracker as a PWA	JavaScript, localStorage, manifest
Notes app with Flask and SQLite	backend, SQL, HTML forms
Mobile to-do app with Expo	React Native, state, AsyncStorage

Advanced (after this guide)

Project	Practises
Blog with user accounts, comments and admin	Django or Next.js, auth, database
Real-time chat app	WebSockets, backend, deployment
Online shop with cart and test payments	full stack, Stripe test mode, security
Data dashboard for a public dataset	pandas, charts, a web framework
REST API with tests and CI	FastAPI or Express, pytest/Jest, GitHub Actions
Mobile app published to a store	the full publishing process (Lesson 49)
Machine learning model (for example, predicting house prices)	scikit-learn, data cleaning, evaluation
Multiplayer browser game	JavaScript, canvas, WebSockets
An AI assistant for one specific task	LLM APIs, prompt design, backend
A clone of an app you love (simplified)	everything!

TIP

The best project is one **you or someone you know would actually use**. Solving a real problem keeps you motivated, and it makes a great story in interviews.

LESSON 62

The best free learning resources

Resource	What it is	Best for
CS50 (cs50.harvard.edu)	Harvard's famous intro courses: CS50x, CS50P (Python), CS50W (web), CS50 AI. Free	A university-quality foundation
freeCodeCamp (freecodecamp.org)	Free interactive courses and certifications, and a huge YouTube channel	Web development, Python, data
The Odin Project (theodinproject.com)	A free, complete full-stack web curriculum	Becoming a web developer
MDN Web Docs (developer.mozilla.org)	The best reference for HTML, CSS and JavaScript	Looking things up while building
Official Python tutorial (docs.python.org)	Python's own tutorial and documentation	Accurate Python details
Automate the Boring Stuff with Python (automatetheboringstuff.com)	A free online book about practical Python automation	Useful scripts for real life
Exercism (exercism.org)	Free exercises in 70+ languages, with optional mentoring	Practice and feedback
Codewars, **HackerRank**, **LeetCode**	Coding puzzles and challenges	Problem solving, interview prep
Kaggle Learn (kaggle.com/learn)	Short, free courses on data science and machine learning	Data and AI
Scratch and **Code.org**	Visual, block-based coding	Very first steps
TryHackMe, **PicoCTF**	Legal, guided hacking practice	Cybersecurity
Roadmap.sh	Visual learning roadmaps for every developer role	Planning what to learn next

YouTube channels developers love

freeCodeCamp.org (full free courses), **CS50** (the Harvard lectures), **Corey Schafer** (clear Python tutorials), **Kevin Powell** (CSS), **Traversy Media** (web projects), **Fireship** (fast overviews of technologies), **Bro Code** (beginner courses in many languages) and **The Coding Train** (creative, fun coding).

Communities: you're not alone

- **Stack Overflow**: search it first, since most errors already have answers.
- **Reddit**: r/learnprogramming, r/learnpython, r/webdev.
- **Discord servers**: The Odin Project, freeCodeCamp, Python and many others.
- **GitHub**: read code from real projects, and later contribute to issues labelled *good first issue*.
- **Local and school clubs**, hackathons and coding meetups.

LESSON 63

Using AI as a learning partner

AI assistants like Claude can be amazing teachers: patient, available any time, and able to explain things in many different ways. But used wrongly, they can stop you from learning at all. The rule is simple:

> **KEY IDEA**
> Use AI to help you **understand**, not to do the thinking **for** you. If you can't explain every line of your code, you're not finished.

Great ways to use AI	Ways that hurt your learning
"Explain this error message like I'm a beginner."	Pasting the exercise and copying the answer
"Give me 5 practice problems about loops. Don't give me the answers."	Letting it write projects you don't understand
"Review my code. Tell me what to improve, but don't rewrite it."	Skipping the struggle. Struggling *is* how the brain learns
"Explain recursion in a different way. I didn't get the book's version."	Believing everything without testing it. AI can be wrong too
"Quiz me on Lesson 12 with 10 questions."	Pasting secrets, passwords or private data
"What's the difference between a list and a tuple, with examples?"	Using it in exams or homework where it isn't allowed

A good way to ask for help

EXAMPLE PROMPT

```
I'm a beginner learning Python. I'm trying to make my to-do app remove a task.
Here is my code:
   [paste the code]
When I type 0, it removes the last task instead of showing an error.
I expected it to say 'No task with that number'.
Please explain WHY this happens and give me a hint, not the full fix.
```

> **IN THE REAL WORLD**
> Professional developers use AI coding assistants every day, for writing tests, explaining unfamiliar code, finding bugs and drafting code. But they can do it well only **because** they understand the code it writes. That understanding is what this guide builds.

LESSON 64

Your portfolio and your first opportunity

Make your GitHub profile shine

- Pin your **3 to 6 best projects**. Quality beats quantity.
- Every project gets a **README.md** with: what it does, a screenshot or GIF, how to run it, what you learnt, and a **live demo link** if possible.
- Make a **profile README** (a repository with the same name as your username) that introduces you.
- Commit regularly, with clear messages. It shows steady growth.

Ways to get real experience

- **Build for real people:** a website for a family business, a club or a charity.
- **Hackathons:** build something in a weekend with a team. Many welcome beginners.
- **Open source:** fix typos in documentation, then small bugs. Look for *good first issue* labels.
- **Internships and apprenticeships:** apply early and often, even if you don't meet every requirement.
- **Freelancing small jobs** once your skills are solid.
- **Teach others:** write a blog post or make a video about what you learnt. Teaching proves understanding.

What technical interviews usually include

Part	How to prepare
Coding problems	Data structures and algorithms (Lessons 23–24); daily practice on LeetCode or HackerRank; think out loud
Talking about your projects	Be ready to explain *why* you made each decision, and what you'd improve
System design (more senior roles)	How you'd build a larger app: databases, APIs, caching, scaling
Behavioural questions	Real stories: a bug you fixed, a team problem you solved, something you learnt quickly

YOU MADE IT

Final words

If you've worked through this guide, look back at where you started: you barely knew what code was. Now you can write programs, build websites, understand how apps and the internet work, and you know what to learn next.

Every expert programmer was once exactly where you started. The difference between people who become developers and people who give up is not talent. It's that they **kept going**, one small step at a time, through the errors and the confusing days.

1. **Build something every week**, however small.
2. **Stay curious**: when something works, ask *why* it works.
3. **Be kind to yourself** on hard days. Confusion means you're learning.
4. **Help others** who are one step behind you.

> **KEY IDEA**
> Your next step: open your editor and type `print("Hello, world!")`. The journey starts there, and it starts today. Good luck!

APPENDICES

Answers, references and checklists

Answers to every exercise and quiz, a glossary, cheat sheets, a troubleshooting guide, keyboard shortcuts, FAQs and your progress checklist.

In this part you will learn

- ✓ Appendix A: answers to all exercises and quizzes
- ✓ Appendix B: glossary of 90+ terms
- ✓ Appendix C: Python cheat sheet
- ✓ Appendix D: HTML, CSS and JavaScript cheat sheet
- ✓ Appendix E: troubleshooting: when things don't work
- ✓ Appendix F: keyboard shortcuts
- ✓ Appendix G: frequently asked questions
- ✓ Appendix H: your progress checklist

APPENDIX A

Answers to exercises and quizzes

There is usually **more than one correct answer**. If your code works and is readable, it's right! Compare with these answers to learn other ways of doing it. For the project challenges, there's no single answer. Share your code with a teacher, a friend or an AI assistant for a review.

Lesson 1: Your first program

1. Three `print` lines:

PYTHON
```
print("Sara")
print(16)
print("Cairo")
```

OUTPUT
```
Sara
16
Cairo
```

2.

```python
print(12 * 7)
```

```
84
```

3. Removing a quote gives an error like `SyntaxError: unterminated string literal (detected at line 1)`. Python means: a string started, but never ended. Put the quote back to fix it.

4.

```python
print("*****")
print("*   *")
print("*****")
```

```
*****
*   *
*****
```

Lesson 2: Variables

1.

```python
city = "Cairo"
country = "Egypt"
print("I live in " + city + ", " + country)
```

```
I live in Cairo, Egypt
```

2.

```python
price = 4
quantity = 3
print(price * quantity)
```

```
12
```

3.

```python
coins = 50
coins -= 20
coins += 35
print(coins)
```

```
65
```

4. Use a temporary variable. Python also has a one-line trick: `a, b = b, a`.

```python
a = 1
b = 2
temp = a
a = b
b = temp
print(a, b)
```

```
2 1
```

Lesson 3: Data types and maths

1.

```python
print(60 * 60 * 24)
```

```
86400
```

2.

```python
c = 100
print(c * 9 / 5 + 32)
```

```
212.0
```

3.

```python
minutes = 135
print(minutes // 60, "hours and", minutes % 60, "minutes")
```

```
2 hours and 15 minutes
```

4. `2 ** 10` is `1024`, and `17 % 5` is `2` (17 = 3 × 5 + **2**).

Lesson 4: Input

1.

```python
a = float(input("First number: "))
b = float(input("Second number: "))
print("Sum:", a + b)
print("Difference:", a - b)
print("Product:", a * b)
print("Division:", a / b)
```

2.

```python
year = int(input("What year were you born? "))
print("This year you turn", 2026 - year)
```

3.

```python
total = int(input("Minutes: "))
print(total // 60, "hours and", total % 60, "minutes")
```

4.

```python
name = input("Your name: ")
food = input("Favourite food: ")
print(name + " would eat " + food + " for breakfast, lunch and dinner!")
```

Lesson 5: Strings

1.

```python
name = input("Your name: ")
print(name.upper())
print(f"Your name has {len(name)} letters")
```

2.

```python
word = input("A word: ")
print("First:", word[0])
print("Last:", word[-1])
```

3.

```python
print("=" * 30)
print("MY AMAZING PROGRAM")
print("=" * 30)
```

4.

```python
name = "Sara"
score = 91
total = 100
percent = score * 100 // total
print(f"{name} scored {score} points out of {total} ({percent}%)")
```

```
Sara scored 91 points out of 100 (91%)
```

Lesson 6: Decisions

1.

```python
number = float(input("A number: "))
if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

2.

```python
password = input("Password: ")
if password == "open123":
    print("Access granted")
else:
    print("Access denied")
```

3.

```python
a = float(input("First: "))
b = float(input("Second: "))
c = float(input("Third: "))

biggest = a
if b > biggest:
    biggest = b
if c > biggest:
    biggest = c
print("Biggest:", biggest)
```

4. Change `year` to test each one: 2024 and 2000 are leap years, but 1900 is not (divisible by 100, but not by 400).

PYTHON
```python
year = 2024
if year % 4 == 0 and (year % 100 != 0 or year % 400 == 0):
    print(year, "is a leap year")
else:
    print(year, "is not a leap year")
```

OUTPUT
```
2024 is a leap year
```

Lesson 7: while loops

1.

PYTHON
```python
n = 2
while n <= 20:
    print(n)
    n += 2
```

2.

PYTHON
```python
total = 0
number = int(input("Number (0 to stop): "))
while number != 0:
    total += number
    number = int(input("Number (0 to stop): "))
print("Total:", total)
```

3.

PYTHON
```python
i = 1
while i <= 10:
    print(f"7 x {i} = {7 * i}")
    i += 1
```

4.

PYTHON
```python
balance = 1000
years = 0
while balance <= 2000:
    balance *= 1.05
    years += 1
print(years, "years")
```

OUTPUT
```
15 years
```

Lesson 8: for loops

1.

```python
for i in range(10, 0, -1):
    print(i)
print("Happy New Year!")
```

2. `letter in "aeiou"` checks whether the letter appears in that string.

```python
word = input("A word: ").lower()
count = 0
for letter in word:
    if letter in "aeiou":
        count += 1
print(count, "vowels")
```

3. Check for 15 **first** (multiples of both 3 and 5). Otherwise, 15 would print just Fizz.

```python
for n in range(1, 31):
    if n % 15 == 0:
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
    else:
        print(n)
```

4.

```python
n = int(input("Height: "))
for row in range(1, n + 1):
    print("#" * row)
```

Quiz 1

1. `3`
2. `str` (it's in quotes, so it's text)
3. It uses `=` (store) instead of `==` (compare). Correct: `if x == 5:`
4. `2`, `4`, `6` (8 is not included)
5. Always a string (`str`)
6. `30` (10 + 5 = 15, then 15 × 2 = 30)
7. `=` stores a value in a variable. `==` compares two values and gives `True` or `False`.
8. `"ababab"`
9. Press **Ctrl + C** in the terminal (or the editor's Stop button)
10. `11` (the space counts as a character)

Lesson 9: Lists

1.

```python
items = []
while True:
    item = input("Item (or done): ")
    if item == "done":
        break
    items.append(item)

for number, item in enumerate(items, start=1):
    print(number, item)
```

2.

```python
numbers = [4, 19, 7, 33, 2]
biggest = numbers[0]
for n in numbers:
    if n > biggest:
        biggest = n
print(biggest)
```

```
33
```

3.

```python
scores = [55, 82, 71, 90, 48, 67]
count = 0
for s in scores:
    if s >= 70:
        count += 1
print(count)
```

```
3
```

4.

```python
numbers = [1, 2, 2, 3, 1, 4]
unique = []
for n in numbers:
    if n not in unique:
        unique.append(n)
print(unique)
```

```
[1, 2, 3, 4]
```

Lesson 11: Dictionaries

1.

```python
phone_book = {"Sara": "0101234567", "Omar": "0119876543", "Lina": "0125556677"}
name = input("Name: ")
print(phone_book.get(name, "Not found"))
```

2.

```python
sentence = input("A sentence: ").lower()
counts = {}
for word in sentence.split():
    counts[word] = counts.get(word, 0) + 1
for word, count in counts.items():
    print(word, count)
```

3.

```python
stock = {"pens": 10, "books": 4}
prices = {"pens": 1.5, "books": 12}
total = 0
for item, amount in stock.items():
    total += amount * prices[item]
print(total)
```

```
OUTPUT
63.0
```

4.

```python
students = [
    {"name": "Sara", "score": 91},
    {"name": "Omar", "score": 78},
    {"name": "Lina", "score": 85},
]
best = students[0]
for s in students:
    if s["score"] > best["score"]:
        best = s
print(best["name"])
```

```
OUTPUT
Sara
```

Lesson 12: Functions

1.

PYTHON
```python
def is_even(n):
    return n % 2 == 0

print(is_even(4), is_even(7))
```

OUTPUT
```
True False
```

2.

PYTHON
```python
def rectangle_area(width, height):
    return width * height


def circle_area(radius):
    return 3.14159 * radius ** 2


print(rectangle_area(3, 4))
print(circle_area(2))
```

OUTPUT
```
12
12.56636
```

3.

PYTHON
```python
def count_vowels(text):
    count = 0
    for letter in text.lower():
        if letter in "aeiou":
            count += 1
    return count

print(count_vowels("Programming"))
```

OUTPUT
```
3
```

4. A faster version only checks divisors up to the square root of `n`, but this one is easier to understand.

PYTHON
```python
def is_prime(n):
    if n < 2:
        return False
    for divisor in range(2, n):
        if n % divisor == 0:
            return False
    return True


primes = []
for n in range(50):
    if is_prime(n):
        primes.append(n)
print(primes)
```

OUTPUT
```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

Lesson 13: Strings in depth

1.

PYTHON
```python
sentence = input("A sentence: ")
print(len(sentence.split()), "words")
```

2.

PYTHON
```python
def acronym(phrase):
    letters = ""
    for word in phrase.split():
        letters += word[0].upper()
    return letters

print(acronym("portable network graphics"))
```

OUTPUT
```
PNG
```

3.

PYTHON
```python
def is_palindrome(text):
    cleaned = text.replace(" ", "").lower()
    return cleaned == cleaned[::-1]

print(is_palindrome("Never odd or even"))
```

OUTPUT
```
True
```

4.

```python
email = input("Email: ")
print(email.split("@")[0])
```

Lesson 14: Errors

1.

```python
def get_number(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Please type a number.")
```

2.

```python
fruits = ["apple", "banana", "cherry"]
try:
    position = int(input("Position: "))
    print(fruits[position])
except ValueError:
    print("Please type a whole number.")
except IndexError:
    print(f"Please choose 0 to {len(fruits) - 1}.")
```

3. `total = n` **replaces** the total each time, so after the loop it's just the last number (6), and 6 / 3 = 2.0. The fix is `total += n`. Lesson 20 shows how to find this kind of bug.

Lesson 15: Files

1.

```python
from datetime import date

entry = input("Today's entry: ")
with open("diary.txt", "a") as file:
    file.write(f"{date.today()}: {entry}\n")
print("Saved!")
```

2.

```python
count = 0
with open("notes.txt") as file:
    for line in file:
        count += 1
print(count, "lines")
```

3.

```python
def load_high_score():
    try:
        with open("highscore.txt") as file:
            return int(file.read())
    except (FileNotFoundError, ValueError):
        return 0


def save_high_score(score):
    with open("highscore.txt", "w") as file:
        file.write(str(score))


high = load_high_score()
score = int(input("Your score: "))
if score > high:
    save_high_score(score)
    print("New high score!")
else:
    print("The high score is still", high)
```

Lesson 16: Modules

1.

```python
import random

count = int(input("How many dice? "))
total = 0
for i in range(count):
    roll = random.randint(1, 6)
    print("Rolled", roll)
    total += roll
print("Total:", total)
```

2. For real passwords, use the `secrets` module (`secrets.choice`) instead of `random`. It is designed to be unpredictable, which matters for security.

```python
import random
import string

characters = string.ascii_letters + string.digits + "!@#$%"
password = ""
for i in range(12):
    password += random.choice(characters)
print(password)
```

3. Change 5 and 17 to your birthday's month and day.

```python
from datetime import date

today = date.today()
birthday = date(today.year, 5, 17)
if birthday < today:
    birthday = date(today.year + 1, 5, 17)
print((birthday - today).days, "days to go!")
```

4.

```python
import turtle

pen = turtle.Turtle()
for i in range(5):
    pen.forward(150)
    pen.right(144)
turtle.done()
```

Quiz 2

1. `20` (index 1 is the second item)
2. `items.sort()` changes the list itself (and returns `None`). `sorted(items)` returns a **new** sorted list and leaves the original alone.
3. A `set`
4. `my_dict.get("key", default_value)`
5. `print` shows a value on the screen. `return` sends a value back to the code that called the function, so it can be stored and used.
6. `['a', 'b', 'c']`
7. `"nohtyP"`
8. `"w"` (write)
9. From the bottom up: the last line gives the error type and message
10. `python -m pip install requests` (`python3` on Mac and Linux)

Lesson 17: Classes

1.

PYTHON

```python
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)


r = Rectangle(3, 4)
print(r.area(), r.perimeter())
```

OUTPUT

```
12 14
```

2.

PYTHON

```python
class Student:
    def __init__(self, name):
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        self.grades.append(grade)

    def average(self):
        if not self.grades:
            return 0
        return sum(self.grades) / len(self.grades)


s = Student("Lina")
s.add_grade(90)
s.add_grade(80)
print(s.name, s.average())
```

OUTPUT

```
Lina 85.0
```

3.

PYTHON

```python
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
        self.history = []

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit must be positive.")
            return
        self.balance += amount
        self.history.append(f"+{amount}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Not enough money!")
            return
        self.balance -= amount
        self.history.append(f"-{amount}")

    def show_history(self):
        for item in self.history:
            print(item)


account = BankAccount("Sara", 100)
account.deposit(50)
account.withdraw(30)
account.show_history()
```

OUTPUT

```
+50
-30
```

Quiz 3

1. A class is the blueprint. An object is one thing built from it (an *instance*).
2. The particular object the method was called on (in `rex.bark()`, `self` is `rex`).
3. Encapsulation, abstraction, inheritance and polymorphism.
4. `[n * n for n in range(11) if n % 2 == 0]` gives `[0, 4, 16, 36, 64, 100]`.
5. A marker on a line where the debugger pauses the program so you can inspect the variables.
6. It raises an `AssertionError` and stops the program, telling you the test failed.
7. O(log n). The list must be **sorted**.
8. A stack (Last In, First Out).
9. A **base case** that stops the recursion.
10. Bugs love boundaries: it's easy to write `>` when you meant `>=`. Testing the exact values where behaviour changes catches these "off-by-one" mistakes.

Lesson 34: HTML

3. Link the two pages to each other with relative links:

```HTML
<!-- in index.html -->
<a href="games.html">My favourite games</a>

<!-- in games.html -->
<a href="index.html">Back to home</a>
```

4.

```HTML
<form>
    <label for="name">Name</label>
    <input type="text" id="name" required>

    <label for="email">Email</label>
    <input type="email" id="email" required>

    <label for="password">Password</label>
    <input type="password" id="password" minlength="8" required>

    <label for="dob">Date of birth</label>
    <input type="date" id="dob">

    <label>
        <input type="checkbox" required> I agree to the rules
    </label>

    <button type="submit">Sign up</button>
</form>
```

Lesson 35: CSS

2.

```CSS
.card {
    padding: 24px;
    border-radius: 12px;
    background: white;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
    transition: transform 0.2s, box-shadow 0.2s;
}

.card:hover {
    transform: translateY(-6px);
    box-shadow: 0 10px 24px rgba(0, 0, 0, 0.15);
}
```

3.

```css
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 16px 32px;
}

.navbar a {
    margin-left: 16px;
}
```

4.

```css
.gallery {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 16px;
}

@media (max-width: 600px) {
    .gallery {
        grid-template-columns: 1fr;
    }
}
```

Lesson 36: JavaScript

1.

```html
<input id="name-input" placeholder="Your name">
<button id="greet-btn">Greet me</button>
<p id="greeting"></p>
```

```javascript
const nameInput = document.querySelector("#name-input");
const greeting = document.querySelector("#greeting");

document.querySelector("#greet-btn").addEventListener("click", () => {
    greeting.textContent = `Hello, ${nameInput.value}!`;
});
```

2. With a button `<button id="color-btn">New colour</button>`:

```javascript
document.querySelector("#color-btn").addEventListener("click", () => {
    const r = Math.floor(Math.random() * 256);
    const g = Math.floor(Math.random() * 256);
    const b = Math.floor(Math.random() * 256);
    document.body.style.backgroundColor = `rgb(${r}, ${g}, ${b})`;
});
```

3. Add the CSS rule `.done { text-decoration: line-through; }` too.

HTML

```
<input id="task-input" placeholder="New task">
<button id="add-btn">Add</button>
<ul id="task-list"></ul>
```

JAVASCRIPT

```
const taskInput = document.querySelector("#task-input");
const taskList = document.querySelector("#task-list");

document.querySelector("#add-btn").addEventListener("click", () => {
    const text = taskInput.value.trim();
    if (text === "") {
        return;
    }
    const li = document.createElement("li");
    li.textContent = text;
    li.addEventListener("click", () => li.classList.toggle("done"));
    taskList.append(li);
    taskInput.value = "";
});
```

4.

HTML

```
<input id="a" type="number"> <input id="b" type="number">
<button data-op="+">+</button> <button data-op="-">-</button>
<button data-op="*">x</button> <button data-op="/">/</button>
<p id="result"></p>
```

JAVASCRIPT

```
const result = document.querySelector("#result");

for (const button of document.querySelectorAll("button[data-op]")) {
    button.addEventListener("click", () => {
        const a = Number(document.querySelector("#a").value);
        const b = Number(document.querySelector("#b").value);
        const op = button.dataset.op;
        let answer;
        if (op === "+") {
            answer = a + b;
        } else if (op === "-") {
            answer = a - b;
        } else if (op === "*") {
            answer = a * b;
        } else {
            answer = b === 0 ? "You can't divide by zero" : a / b;
        }
        result.textContent = answer;
    });
}
```

APPENDIX B

Glossary

Term	Meaning
Algorithm	A step-by-step method for solving a problem
API	A way for programs to talk to each other, usually by sending requests and receiving JSON
App store	A shop for apps: Apple App Store, Google Play, Microsoft Store
Argument	A value you pass into a function when calling it: in `greet("Sara")`, `"Sara"` is the argument
Array	An ordered list of values (called a list in Python)
Assembly	A low-level language where each line is one CPU instruction
Authentication	Checking who a user is (logging in)
Backend	The part of an app that runs on a server: logic, data and security
Big-O	A way to describe how an algorithm's work grows as its input grows
Binary	The number system of 0s and 1s that computers use
Boolean	A value that is either `True` or `False`
Branch (Git)	A separate line of work in a repository
Breakpoint	A marker where the debugger pauses your program
Browser	The app that shows websites: Chrome, Firefox, Safari, Edge
Bug	A mistake in a program
Cache	A store of data kept close by, so it can be reused quickly
Class	A blueprint for creating objects
CLI	Command-line interface: a program you use by typing in the terminal
Client	The program asking for something, usually the browser or app
Cloud	Computers you rent over the internet (AWS, Azure, Google Cloud)
Commit	A saved snapshot of your project in Git
Compiler	A program that translates all of your code into machine code before it runs
Component	A reusable piece of user interface, in frameworks like React or Flutter
Concatenation	Joining strings together
Container	A packaged app with everything it needs to run (Docker)
CRUD	Create, Read, Update, Delete: the four basic data operations
CSS	The language that styles web pages
Database	Software for storing and searching large amounts of data
Debugging	Finding and fixing bugs

Term	Meaning
Deploy	Put your software live, for real users
Dictionary	A collection of key: value pairs
DNS	The internet's phone book: it turns domain names into IP addresses
DOM	The browser's tree of objects for a web page, which JavaScript can change
Domain name	A website's human-friendly address, like `example.com`
Edge case	An unusual input at the limits, like an empty list or zero
Event	Something that happens, like a click or key press, that code can react to
Exception	An error that happens while a program runs
Framework	A large library that gives your project structure (React, Django, Flutter)
Frontend	The part of an app that users see and touch
Full-stack	Working on both frontend and backend
Function	A named, reusable block of code
Git	A version control system that tracks the history of your code
GitHub	A website for hosting Git repositories and collaborating
Hosting	A service that keeps your website or app running online
HTML	The language for the structure and content of web pages
HTTP / HTTPS	The protocol browsers and servers use to talk; the S means encrypted
IDE	Integrated Development Environment: an editor with built-in tools
Index	The position of an item in a sequence, starting at 0
Infinite loop	A loop that never stops
Inheritance	A class reusing the code of another class
Interpreter	A program that runs code line by line
IP address	A number that identifies a computer on a network
JSON	A text format for data, which looks like Python dictionaries and lists
Keyword	A word reserved by the language, like `if`, `for` or `def`
Library / package	Reusable code that someone else wrote
Linter	A tool that warns you about likely mistakes and style problems
Loop	Code that repeats
Merge	Combining changes from one branch into another
Method	A function that belongs to an object
Module	A file of code that can be imported
MVP	Minimum viable product: the smallest useful version of an app
Native app	An app built with a platform's own language and tools
Node.js	A program that runs JavaScript outside the browser

Term	Meaning
npm / pip	Package managers for JavaScript / Python
Object	A thing built from a class, with attributes and methods
Open source	Software whose code is public, and that anyone can study and contribute to
Operating system	The main software that runs a computer: Windows, macOS, Linux, Android, iOS
Parameter	A variable in a function definition that receives an argument
Pull request (PR)	A request to review and merge your changes
PWA	Progressive web app: a website that can be installed like an app
Query	A question asked to a database, usually in SQL
RAM	The computer's fast, short-term memory
Recursion	A function calling itself
Refactor	Improving code's structure without changing what it does
Repository (repo)	A project tracked by Git
Responsive design	Websites that adapt to any screen size
Return value	The value a function sends back
Runtime	The time while a program is running, or the system that runs it
Scope	Where in the code a variable exists and can be used
SDK	Software Development Kit: tools for building for a platform
Server	A computer that waits for requests and responds to them
Snake_case	Naming style with lowercase words joined by underscores
Source code	The human-written text of a program
SQL	The language for working with relational databases
SQL injection	An attack that sneaks SQL code into user input
Stack trace / traceback	The error report showing where a program crashed
State	Data that can change while an app runs, like a counter's value
String	Text data
Syntax	The grammar rules of a programming language
Terminal	A window for typing commands
Test	Code that checks that other code works
TypeScript	JavaScript with types
UI / UX	User interface (how it looks) / user experience (how it feels to use)
URL	A web address
Variable	A named place that stores a value
Version control	Tracking changes to code over time (Git)
XSS	Cross-site scripting: an attack that injects JavaScript into a web page

APPENDIX C

Python cheat sheet

PYTHON · BASICS

```
# ---------- basics ----------
name = "Sara"                # str
age = 16                     # int
height = 1.62                # float
is_student = True            # bool
nothing = None

print(f"{name} is {age}")    # f-string
age = int(input("Age: "))    # input is always a str

# ---------- maths ----------
#  +  -  *  /   //  (floor)   %  (remainder)   **  (power)
x += 1                       # x = x + 1

# ---------- comparisons and logic ----------
#  ==  !=  <  >  <=  >=      and  or  not      in

# ---------- decisions ----------
if age >= 18:
    print("adult")
elif age >= 13:
    print("teen")
else:
    print("child")

# ---------- loops ----------
for i in range(5):           # 0 1 2 3 4
    print(i)
for item in my_list:
    print(item)
while count < 10:
    count += 1
# break: leave the loop      continue: skip to the next round
```

PYTHON · COLLECTIONS AND STRINGS

```
# ---------- lists ----------
nums = [3, 1, 2]
nums.append(4); nums.insert(0, 9); nums.remove(1); nums.pop()
nums.sort(); sorted(nums); len(nums); sum(nums); max(nums); min(nums)
nums[0]; nums[-1]; nums[1:3]; 3 in nums
squares = [n * n for n in nums if n > 1]

# ---------- dictionaries ----------
person = {"name": "Sara", "age": 16}
person["city"] = "Cairo"
person.get("phone", "none")
for key, value in person.items():
    print(key, value)

# ---------- tuples and sets ----------
point = (3, 4)
```

```
x, y = point
unique = set([1, 1, 2])            # {1, 2}

# ---------- strings ----------
s = "  Hello World  "
s.strip(); s.lower(); s.upper(); s.title(); s.replace("World", "You")
s.split(); ", ".join(["a", "b"]); s.find("o"); s.startswith("H")
s[0]; s[-1]; s[::-1]; len(s)
f"{3.14159:.2f}"                    # '3.14'
```

PYTHON · FUNCTIONS, ERRORS, FILES, CLASSES

```
# ---------- functions ----------
def add(a, b=0):
    """Docstring: what the function does."""
    return a + b

square = lambda n: n * n

# ---------- errors ----------
try:
    number = int(text)
except ValueError:
    print("Not a number")
finally:
    print("Always runs")

# ---------- files ----------
with open("notes.txt", "w") as f:      # "r" read, "w" write, "a" append
    f.write("Hello\n")
with open("notes.txt") as f:
    for line in f:
        print(line.strip())

# ---------- classes ----------
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return f"{self.name} says woof"

class Puppy(Dog):                      # inheritance
    def bark(self):
        return super().bark() + " (quietly)"

# ---------- modules ----------
import math, random, json
from datetime import date
# terminal:  python -m pip install requests

if __name__ == "__main__":
    main()
```

APPENDIX D

HTML, CSS and JavaScript cheat sheet

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Page title</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header><nav><a href="#about">About</a></nav></header>
    <main>
        <section id="about">
            <h1>Main heading</h1>   <h2>Subheading</h2>
            <p>Text with <strong>bold</strong> and <em>italic</em>.</p>
            <a href="https://example.com">A link</a>
            <img src="photo.jpg" alt="Description of the photo">
            <ul><li>Bullet item</li></ul>   <ol><li>Numbered item</li></ol>
            <div class="box">A generic box</div> <span>inline text</span>
            <button id="btn">Click</button>
            <input type="text" placeholder="Type here">
        </section>
    </main>
    <footer>&copy; 2026</footer>
    <script src="script.js"></script>
</body>
</html>
```

CSS

```
/* selectors */
p { }   .class { }   #id { }   nav a { }   a:hover { }   h1, h2 { }

/* text */
color: #333;  font-family: Arial, sans-serif;  font-size: 1.2rem;
font-weight: bold;  text-align: center;  line-height: 1.6;

/* box model */
width: 300px;  max-width: 100%;  padding: 16px;  margin: 0 auto;
border: 1px solid #ddd;  border-radius: 8px;  box-sizing: border-box;
background: #f5f5f5;  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);

/* flexbox */
display: flex;  flex-direction: row;  justify-content: space-between;
align-items: center;  gap: 16px;  flex-wrap: wrap;

/* grid */
display: grid;  grid-template-columns: repeat(3, 1fr);  gap: 16px;

/* responsive */
@media (max-width: 700px) { .grid { grid-template-columns: 1fr; } }

/* animation */
```

```
transition: transform 0.2s;   transform: scale(1.05);
```

JAVASCRIPT

```
// variables
const name = "Sara";   let score = 0;

// decisions and loops
if (score > 10 && name === "Sara") { } else if (score > 5) { } else { }
for (let i = 0; i < 5; i++) { }
for (const item of items) { }
while (running) { }

// functions
function add(a, b) { return a + b; }
const multiply = (a, b) => a * b;

// arrays and objects
const nums = [1, 2, 3];
nums.push(4);  nums.length;  nums.map(n => n * 2);  nums.filter(n => n > 1);
const user = { name: "Sara", age: 16 };   user.name;

// the DOM
const el = document.querySelector("#btn");
el.textContent = "Hi";   el.classList.toggle("dark");   input.value;
el.addEventListener("click", () => { console.log("clicked"); });
const li = document.createElement("li");   list.append(li);

// fetching data
const response = await fetch("https://api.example.com/data");
const data = await response.json();

// saving in the browser
localStorage.setItem("theme", "dark");   localStorage.getItem("theme");
```

APPENDIX E

Troubleshooting: when things don't work

Problem	Likely cause	Fix
`'python' is not recognized` (Windows)	Python isn't on the PATH	Try `py`. Or reinstall Python with "Add python.exe to PATH" ticked
`command not found: python` (Mac)	On Macs it's called `python3`	Use `python3` and `python3 -m pip`
`ModuleNotFoundError: No module named 'requests'`	The package isn't installed, or it's installed for a different Python	`python -m pip install requests`, using the same `python` command you run programs with
VS Code says a module is missing, but the terminal works	VS Code is using a different Python	Ctrl + Shift + P, then **Python: Select Interpreter**, and choose the right one
`IndentationError`	Spaces and tabs mixed, or wrong indentation	Use 4 spaces. In VS Code, use *Convert Indentation to Spaces*
`FileNotFoundError` when the file is right there	The program runs from a different folder	Open the project **folder** in VS Code, or run the program from that folder in the terminal
My program doesn't wait for input / closes instantly	It was double-clicked, so the window closes at the end	Run it from the editor or the terminal instead
Infinite loop, and the program won't stop	The loop condition never becomes false	Ctrl + C, then check that the loop changes its variable
CSS changes don't show	Wrong file path, a typo in the selector, or browser cache	Check `<link href>`, check the class name, then hard refresh (Ctrl + Shift + R)
JavaScript: `Cannot read properties of null`	`querySelector` found nothing: wrong selector, or the script ran before the HTML loaded	Check the id or class, and put `<script>` just before `</body>`
The website works locally but images are missing online	File names are case-sensitive online: `Photo.JPG` is not `photo.jpg`	Use lowercase file names, with no spaces
`git push` rejected	GitHub has changes you don't have	`git pull`, fix any conflicts, then `git push`
Git merge conflict	Two branches changed the same lines	Open the file, keep the right parts, remove the `<<<<<<<` markers, then add and commit
`npm` or `npx` not found	Node.js isn't installed	Install the LTS version from nodejs.org, then restart the terminal
Expo app won't load on the phone	Phone and computer are on different networks	Use the same Wi-Fi, or run `npx expo start --tunnel`

TIP

The universal debugging steps: read the whole error message, search for its last line, check spelling and capital letters, make the smallest program that shows the problem, then ask for help with the code and the full error.

APPENDIX F

Keyboard shortcuts

On a Mac, use **Cmd** instead of **Ctrl** (and **Option** instead of **Alt**).

Shortcut	Where	What it does
Ctrl + S	Everywhere	Save
Ctrl + Z / Ctrl + Shift + Z	Everywhere	Undo / redo
Ctrl + C / Ctrl + V / Ctrl + X	Everywhere	Copy / paste / cut
Ctrl + F	Everywhere	Find
Ctrl + /	VS Code	Comment or uncomment the selected lines
Ctrl + Shift + P	VS Code	Command palette: search every command
Ctrl + P	VS Code	Quickly open a file by name
Ctrl + `	VS Code	Open or close the terminal
Alt + Up / Down	VS Code	Move the current line up or down
Shift + Alt + Down	VS Code	Copy the current line below
Ctrl + D	VS Code	Select the next match of the selected word (edit several at once)
Shift + Alt + F	VS Code	Format the document
F5 / F10 / F11	VS Code	Start debugging / step over / step into
F12 or Ctrl + Shift + I	Browser	Open DevTools
Ctrl + Shift + R	Browser	Hard refresh (ignore the cache)
Ctrl + C	Terminal	Stop the running program
Tab	Terminal	Auto-complete file and folder names
Up arrow	Terminal	Previous command

APPENDIX G

Frequently asked questions

How long does it take to learn to code?

You can write simple programs within weeks. Building real projects comfortably usually takes 6 to 12 months of regular practice. Getting job-ready often takes 1 to 2 years. You never stop learning, and that's part of the fun.

Am I too young or too old?

No. People start coding as children and in their 60s. What matters is regular practice.

Do I need an expensive computer?

No. Any computer from the last several years works for learning. 8 GB of RAM is fine to start. Only iPhone app development needs something specific: a Mac.

Should I learn several languages at once?

No. Go deep in one first (Python), then add JavaScript for the web. Learning two at the very start mixes up the syntax and slows you down.

I understand the lessons but can't write code on my own. What's wrong?

Nothing. It's the most common stage of learning! Reading and writing are different skills. Close the book and rebuild the examples from memory, do more exercises, and build small projects of your own. It gets easier.

I keep forgetting things. Is that normal?

Completely. Professionals look things up all day. You don't need to memorise everything, just know that it exists and where to find it. Repetition through projects makes the important things stick.

Will AI replace programmers?

AI is changing how programming is done: it writes a lot of routine code. But someone still needs to understand problems, design solutions, check that the code is correct and safe, and take responsibility for it. People who understand code *and* use AI well are more valuable than ever.

Is it OK to use AI to help me learn?

Yes, if you use it to understand rather than to copy (see Lesson 63). Always follow your school's rules for homework and exams.

How do I stay motivated?

Build things you care about, track your progress (Appendix H), learn with a friend or community, and celebrate small wins. Your first working program is a big deal!

Is coding only for people who are good at maths?

No. Most programming uses logic and basic arithmetic. Specific areas, like 3D games, graphics and AI, use more maths, and you can learn it when you need it.

APPENDIX H

Your progress checklist

Tick each box when you have finished the lesson **and** done its exercises. Print this page or copy it into a notebook.

Parts 0–1: Getting started

Lesson	Date finished
☐ 0.1–0.5 Welcome lessons	
☐ 1.1–1.5 Tools set up	
☐ Ran my first `.py` file	

Part 2: Python Stage 1

Lesson	Date finished
☐ Lesson 1: First program	
☐ Lesson 2: Variables	
☐ Lesson 3: Data types and maths	
☐ Lesson 4: Input	
☐ Lesson 5: Strings	
☐ Lesson 6: Decisions	
☐ Lesson 7: while loops	
☐ Lesson 8: for loops	
☐ Project 1: Guessing game	
☐ Quiz 1 (8/10 or better)	

Part 3: Python Stage 2

Lesson	Date finished
☐ Lesson 9: Lists	
☐ Lesson 10: Tuples and sets	
☐ Lesson 11: Dictionaries	
☐ Lesson 12: Functions	
☐ Lesson 13: Strings in depth	
☐ Lesson 14: Errors	
☐ Lesson 15: Files	
☐ Lesson 16: Modules	
☐ Project 2: To-do app	
☐ Quiz 2 (8/10 or better)	

Part 4: Python Stage 3

Lesson	Date finished
☐ Lesson 17: Classes	
☐ Lesson 18: Inheritance	
☐ Lesson 19: Shortcuts	
☐ Lesson 20: Debugging	
☐ Lesson 21: Testing	
☐ Lesson 22: Clean code	
☐ Lesson 23: Algorithms	
☐ Lesson 24: Data structures	
☐ Project 3: Contact book	
☐ Quiz 3 (8/10 or better)	

Part 5: Developer essentials

Lesson	Date finished
☐ Lesson 25: Terminal	
☐ Lesson 26: Git and GitHub (first repo pushed!)	
☐ Lesson 27: How the internet works	
☐ Lesson 28: APIs and JSON	
☐ Lesson 29: Databases and SQL	
☐ Lesson 30: Security	
☐ Lesson 31: Teamwork	
☐ Lesson 32: Cloud and Docker	

Part 6: Websites

Lesson	Date finished
☐ Lesson 33: What a website needs	
☐ Lesson 34: HTML	
☐ Lesson 35: CSS	
☐ Lesson 36: JavaScript	
☐ Project 4: Portfolio website	
☐ Lesson 38: Published online!	
☐ Lesson 39: Backends	
☐ Lesson 40: Next steps	

Part 7: Apps

Lesson	Date finished
☐ Lesson 41: Websites vs apps	
☐ Lesson 42: Native vs cross-platform	
☐ Lesson 43: One app, four frameworks	
☐ Lesson 44: First mobile app on my phone	
☐ Lesson 45: Desktop app	
☐ Lesson 46: PWAs	
☐ Lesson 47: Games	
☐ Lessons 48–50: Requirements, publishing, AI apps	

Parts 8–9: Languages and your path

Lesson	Date finished
☐ Lessons 51–58: Language tour	
☐ Chose my path (Lesson 59)	
☐ Made my 12-month plan (Lesson 60)	
☐ Started a project from Lesson 61	